

JS 章节

内置类型

JS 中分为七种内置类型，七种内置类型又分为两大类型：基本类型和对象（Object）。

基本类型有六种： `null`，`undefined`，`boolean`，`number`，`string`，`symbol`。

其中 JS 的数字类型是浮点类型的，没有整型。并且浮点类型基于 IEEE 754标准实现，在使用中会遇到某些 Bug。NaN 也属于 `number` 类型，并且 NaN 不等于自身。

对于基本类型来说，如果使用字面量的方式，那么这个变量只是个字面量，只有在必要的时候才会转换为对应的类型

```
let a = 111 // 这只是字面量，不是 number 类型
a.toString() // 使用时候才会转换为对象类型
```

对象（Object）是引用类型，在使用过程中会遇到浅拷贝和深拷贝的问题。

```
let a = { name: 'FE' }
let b = a
b.name = 'EF'
console.log(a.name) // EF
```

Typeof

`typeof` 对于基本类型，除了 `null` 都可以显示正确的类型

```
typeof 1 // 'number'
typeof '1' // 'string'
typeof undefined // 'undefined'
typeof true // 'boolean'
typeof Symbol() // 'symbol'
typeof b // b 没有声明, 但是还会显示 undefined
```

typeof 对于对象, 除了函数都会显示 object

```
typeof [] // 'object'
typeof {} // 'object'
typeof console.log // 'function'
```

对于 null 来说, 虽然它是基本类型, 但是会显示 object, 这是一个存在很久了 Bug

```
typeof null // 'object'
```

PS: 为什么会出现这种情况呢? 因为在 JS 的最初版本中, 使用的是 32 位系统, 为了性能考虑使用低位存储了变量的类型信息, 000 开头代表是对象, 然而 null 表示为全零, 所以将它错误的判断为 object。虽然现在的内部类型判断代码已经改变了, 但是对于这个 Bug 却是一直流传下来。

如果我们想获得一个变量的正确类型, 可以通过 `Object.prototype.toString.call(xx)`。这样我们就可以获得类似 `[object Type]` 的字符串。

```
let a
// 我们也可以这样判断 undefined
a === undefined
// 但是 undefined 不是保留字，能够在低版本浏览器被赋值
let undefined = 1
// 这样判断就会出错
// 所以可以用下面的方式来判断，并且代码量更少
// 因为 void 后面随便跟上一个组成表达式
// 返回就是 undefined
a === void 0
```

类型转换

转Boolean

在条件判断时，除了 `undefined`，`null`，`false`，`NaN`，`''`，`0`，`-0`，其他所有值都转为 `true`，包括所有对象。

对象转基本类型

对象在转换基本类型时，首先会调用 `valueOf` 然后调用 `toString`。并且这两个方法你是可以重写的。

```
let a = {
  valueOf() {
    return 0
  }
}
```

当然你也可以重写 `Symbol.toPrimitive`，该方法在转基本类型时调用优先级最高。

```
let a = {
  valueOf() {
    return 0;
  },
  toString() {
    return '1';
  },
  [Symbol.toPrimitive]() {
    return 2;
  }
}
1 + a // => 3
'1' + a // => '12'
```

四则运算符

只有当加法运算时，其中一方是字符串类型，就会把另一个也转为字符串类型。其他运算只要其中一方是数字，那么另一方就转为数字。并且加法运算会触发三种类型转换：将值转换为原始值，转换为数字，转换为字符串。

```
1 + '1' // '11'
2 * '2' // 4
[1, 2] + [2, 1] // '1,22,1'
// [1, 2].toString() -> '1,2'
// [2, 1].toString() -> '2,1'
// '1,2' + '2,1' = '1,22,1'
```

对于加号需要注意这个表达式 `'a' + + 'b'`

```
'a' + + 'b' // -> "NaN"
// 因为 + 'b' -> NaN
// 你也许在一些代码中看到过 + '1' -> 1
```

== 操作符

比较运算 $x==y$, 其中 x 和 y 是值, 产生`true`或者`false`。这样的比较按如下方式进行:

1. 若`Type(x)`与`Type(y)`相同, 则
 - a. 若`Type(x)`为`Undefined`, 返回`true`。
 - b. 若`Type(x)`为`Null`, 返回`true`。
 - c. 若`Type(x)`为`Number`, 则
 - i. 若 x 为`NaN`, 返回`false`。
 - ii. 若 y 为`NaN`, 返回`false`。
 - iii. 若 x 与 y 为相等数值, 返回`true`。
 - iv. 若 x 为`+0`且 y 为`-0`, 返回`true`。
 - v. 若 x 为`-0`且 y 为`+0`, 返回`true`。
 - vi. 返回`false`。
 - d. 若`Type(x)`为`String`, 则当 x 和 y 为完全相同的字符序列(长度相等且相同字符在相同位置)时返回`true`。否则, 返回`false`。
 - e. 若`Type(x)`为`Boolean`, 当 x 和 y 同为`true`或者同为`false`时返回`true`。否则, 返回`false`。
 - f. 当 x 和 y 为引用同一对象时返回`true`。否则, 返回`false`。
2. 若 x 为`null`且 y 为`undefined`, 返回`true`。
3. 若 x 为`undefined`且 y 为`null`, 返回`true`。
4. 若`Type(x)`为`Number`且`Type(y)`为`String`, 返回`comparison x == ToNumber(y)`的结果。
5. 若`Type(x)`为`String`且`Type(y)`为`Number`,
6. 返回比较`ToNumber(x) == y`的结果。
7. 若`Type(x)`为`Boolean`, 返回比较`ToNumber(x) == y`的结果。
8. 若`Type(y)`为`Boolean`, 返回比较`x == ToNumber(y)`的结果。
9. 若`Type(x)`为`String`或`Number`, 且`Type(y)`为`Object`, 返回比较`x == ToPrimitive(y)`的结果。
10. 若`Type(x)`为`Object`且`Type(y)`为`String`或`Number`, 返回比较`ToPrimitive(x) == y`的结果。
11. 返回`false`。

上图中的 `toPrimitive` 就是对象转基本类型。

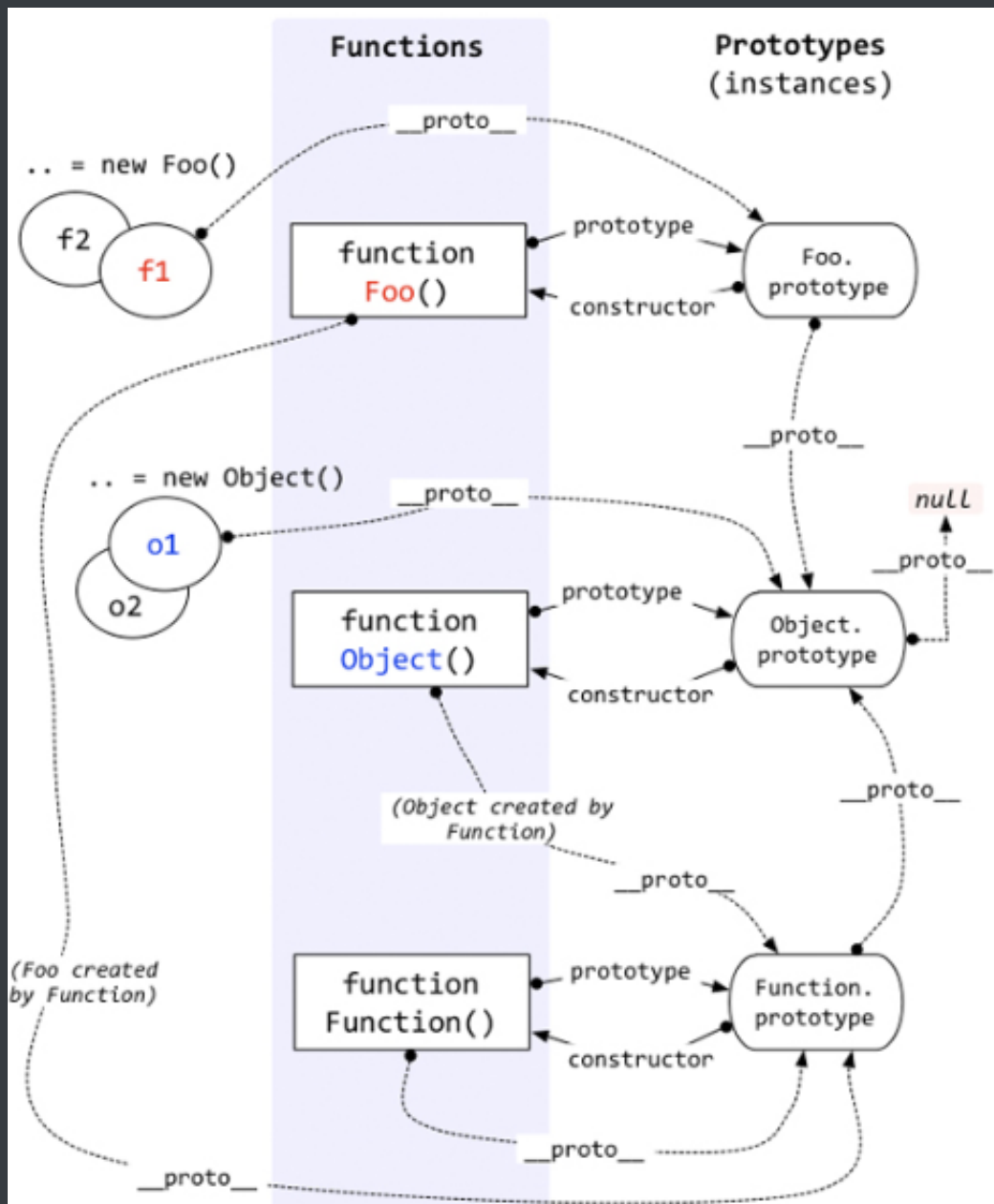
这里来解析一道题目 `[] == ![] // -> true`, 下面是这个表达式为何为 `true` 的步骤

```
// [] 转成 true, 然后取反变成 false
[] == false
// 根据第 8 条得出
[] == ToNumber(false)
[] == 0
// 根据第 10 条得出
ToPrimitive([]) == 0
// [].toString() -> ''
'' == 0
// 根据第 6 条得出
0 == 0 // -> true
```

比较运算符

1. 如果是对象，就通过 toPrimitive 转换对象
2. 如果是字符串，就通过 unicode 字符索引来比较

原型



每个函数都有 `prototype` 属性，除了 `Function.prototype.bind()`，该属性指向原型。

每个对象都有 `__proto__` 属性，指向了创建该对象的构造函数的原型。其实这个属性指向了 `[[prototype]]`，但是 `[[prototype]]` 是内部属性，我们并不能访问到，所以使用 `__proto__` 来访问。

对象可以通过 `__proto__` 来寻找不属于该对象的属性，`__proto__` 将对象连接起来组成了原型链。

如果你想更进一步的了解原型，可以仔细阅读 [深度解析原型中的各个难点](#)。

new

1. 新生成了一个对象
2. 链接到原型
3. 绑定 this
4. 返回新对象

在调用 new 的过程中会发生以上四件事情，我们也可以试着来自己实现一个 new

```
function create() {  
  // 创建一个空的对象  
  let obj = new Object()  
  // 获得构造函数  
  let Con = [].shift.call(arguments)  
  // 链接到原型  
  obj.__proto__ = Con.prototype  
  // 绑定 this, 执行构造函数  
  let result = Con.apply(obj, arguments)  
  // 确保 new 出来的是个对象  
  return typeof result === 'object' ? result : obj  
}
```

对于实例对象来说，都是通过 new 产生的，无论是 function Foo() 还是 let a = { b : 1 } 。

对于创建一个对象来说，更推荐使用字面量的方式创建对象（无论性能上还是可读性）。因为你使用 new Object() 的方式创建对象需要通过作用域链一层层找到 Object，但是你使用字面量的方式就没这个问题。

```
function Foo() {}  
// function 就是个语法糖  
// 内部等同于 new Function()  
let a = { b: 1 }  
// 这个字面量内部也是使用了 new Object()
```


对于 new 来说，还需要注意下运算符优先级。

```
function Foo() {
  return this;
}
Foo.getName = function () {
  console.log('1');
};
Foo.prototype.getName = function () {
  console.log('2');
};

new Foo.getName();    // -> 1
new Foo().getName(); // -> 2
```

Precedence	Operator type	Associativity	Individual operators
20	Grouping	n/a	(...)
19	Member Access	left-to-right	
	Computed Member Access	left-to-right	... [...]
	new (with argument list)	n/a	new ... (...)
	Function Call	left-to-right	... (...)
18	new (without argument list)	right-to-left	new ...

从上图可以看出， new Foo() 的优先级大于 new Foo ，所以对于上述代码来说可以这样划分执行顺序

```
new (Foo.getName());
(new Foo()).getName();
```

对于第一个函数来说，先执行了 Foo.getName() ，所以结果为 1；对于后者来说，先执行 new Foo() 产生了一个实例，然后通过原型链找到了 Foo 上的 getName 函数，所以结果为 2。

instanceof

`instanceof` 可以正确的判断对象的类型，因为内部机制是通过判断对象的原型链中是不是能找到类型的 `prototype`。

我们也可以试着实现一下 `instanceof`

```
function instanceof(left, right) {  
    // 获得类型的原型  
    let prototype = right.prototype  
    // 获得对象的原型  
    left = left.__proto__  
    // 判断对象的类型是否等于类型的原型  
    while (true) {  
        if (left === null)  
            return false  
        if (prototype === left)  
            return true  
        left = left.__proto__  
    }  
}
```

this

`this` 是很多人会混淆的概念，但是其实他一点都不难，你只需要记住几个规则就可以了。

```
function foo() {  
    console.log(this.a)  
}  
  
var a = 1  
foo()  
  
var obj = {  
    a: 2,  
    foo: foo
```

```
}  
obj.foo()
```

// 以上两者情况 `this` 只依赖于调用函数前的对象，优先级是第二个情况大于第一个情况

// 以下情况是优先级最高的，`this` 只会绑定在 `c` 上，不会被任何方式修改 `this` 指向

```
var c = new foo()  
c.a = 3  
console.log(c.a)
```

// 还有种就是利用 `call`, `apply`, `bind` 改变 `this`，这个优先级仅次于 `new`

以上几种情况明白了，很多代码中的 `this` 应该就没什么问题了，下面让我们看看箭头函数中的 `this`

```
function a() {  
  return () => {  
    return () => {  
      console.log(this)  
    }  
  }  
}  
  
console.log(a()()())
```

箭头函数其实是没有 `this` 的，这个函数中的 `this` 只取决于他外面的第一个不是箭头函数的函数的 `this`。在这个例子中，因为调用 `a` 符合前面代码中的第一个情况，所以 `this` 是 `window`。并且 `this` 一旦绑定了上下文，就不会被任何代码改变。

执行上下文

当执行 JS 代码时，会产生三种执行上下文

- 全局执行上下文
- 函数执行上下文

- eval 执行上下文

每个执行上下文中都有三个重要的属性

- 变量对象 (VO) ， 包含变量、函数声明和函数的形参，该属性只能在全局上下文中访问
- 作用域链 (JS 采用词法作用域，也就是说变量的作用域是在定义时就决定了)
- this

```
var a = 10
function foo(i) {
  var b = 20
}
foo()
```

对于上述代码，执行栈中有两个上下文：全局上下文和函数 foo 上下文。

```
stack = [
  globalContext,
  fooContext
]
```

对于全局上下文来说，VO 大概是这样的

```
globalContext.VO === globe
globalContext.VO = {
  a: undefined,
  foo: <Function>,
}
```

对于函数 foo 来说，VO 不能访问，只能访问到活动对象 (AO)

```

fooContext.V0 === foo.A0
fooContext.A0 {
  i: undefined,
  b: undefined,
  arguments: <>
}
// arguments 是函数独有的对象(箭头函数没有)
// 该对象是一个伪数组，有 `length` 属性且可以通过下标访问元素
// 该对象中的 `callee` 属性代表函数本身
// `caller` 属性代表函数的调用者

```

对于作用域链，可以把它理解成包含自身变量对象和上级变量对象的列表，通过 `[[Scope]]` 属性查找上级变量

```

fooContext. [[Scope]] = [
  globalContext.V0
]
fooContext.Scope = fooContext. [[Scope]] + fooContext.V0
fooContext.Scope = [
  fooContext.V0,
  globalContext.V0
]

```

接下来让我们看一个老生常谈的例子， `var`

```

b() // call b
console.log(a) // undefined

var a = 'Hello world'

function b() {
  console.log('call b')
}

```

想必以上的输出大家肯定都已经明白了，这是因为函数和变量提升的原因。通常提升的解释是说将声明的代码移动到了顶部，这其实没有什么错误，便于大家理解。但是更准确的解释应该是：在生成执行上下文时，会有两个阶段。第一个阶段是创建的阶段（具体步骤是创建 VO），JS 解释器会找出需要提升的变量和函数，并且给他们提前在内存中开辟好空间，函数的话会将整个函数存入内存中，变量只声明并且赋值为 undefined，所以在第二个阶段，也就是代码执行阶段，我们可以直接提前使用。

在提升的过程中，相同的函数会覆盖上一个函数，并且函数优先于变量提升

```
b() // call b second

function b() {
  console.log('call b fist')
}
function b() {
  console.log('call b second')
}
var b = 'Hello world'
```

var 会产生很多错误，所以在 ES6 中引入了 let。let 不能在声明前使用，但是这并不是常说的 let 不会提升，let 提升了声明但没有赋值，因为临时死区导致了并不能在声明前使用。

对于非匿名的立即执行函数需要注意以下一点

```
var foo = 1
(function foo() {
  foo = 10
  console.log(foo)
})(); // -> f foo() { foo = 10 ; console.log(foo) }
```

因为当 JS 解释器在遇到非匿名的立即执行函数时，会创建一个辅助的特定对象，然后将函数名称作为这个对象的属性，因此函数内部才可以访问到 foo，但是这个值又是只读的，所以对它的赋值并不生效，所以打印的结果还是这个函数，并且外部的值也没有发生更改。

```
specialObject = {};  
  
Scope = specialObject + Scope;  
  
foo = new FunctionExpression;  
foo.[[Scope]] = Scope;  
specialObject.foo = foo; // {DontDelete}, {ReadOnly}  
  
delete Scope[0]; // remove specialObject from the front of scope chain
```

闭包

闭包的定义很简单：函数 A 返回了一个函数 B，并且函数 B 中使用了函数 A 的变量，函数 B 就被称为闭包。

```
function A() {  
  let a = 1  
  function B() {  
    console.log(a)  
  }  
  return B  
}
```

你是否会疑惑，为什么函数 A 已经弹出调用栈了，为什么函数 B 还能引用到函数 A 中的变量。因为函数 A 中的变量这时候是存储在堆上的。现在的 JS 引擎可以通过逃逸分析辨别出哪些变量需要存储在堆上，哪些需要存储在栈上。

经典面试题，循环中使用闭包解决 var 定义函数的问题

```
for ( var i=1; i<=5; i++) {  
  setTimeout( function timer() {  
    console.log( i );  
  }, i*1000 );  
}
```

首先因为 `setTimeout` 是个异步函数，所有会先把循环全部执行完毕，这时候 `i` 就是 6 了，所以会输出一堆 6。

解决办法两种，第一种使用闭包

```
for (var i = 1; i <= 5; i++) {  
  (function(j) {  
    setTimeout(function timer() {  
      console.log(j);  
    }, j * 1000);  
  })(i);  
}
```

第二种就是使用 `setTimeout` 的第三个参数

```
for ( var i=1; i<=5; i++) {  
  setTimeout( function timer(j) {  
    console.log( j );  
  }, i*1000, i);  
}
```

第三种就是使用 `let` 定义 `i` 了

```
for ( let i=1; i<=5; i++) {  
  setTimeout( function timer() {  
    console.log( i );  
  }, i*1000 );  
}
```

因为对于 `let` 来说，他会创建一个块级作用域，相当于

```
{ // 形成块级作用域  
  let i = 0  
  {
```



```

    let ii = i
    setTimeout( function timer() {
        console.log( ii );
    }, i*1000 );
}
i++
{
    let ii = i
}
i++
{
    let ii = i
}
...
}

```

深浅拷贝

```

let a = {
    age: 1
}
let b = a
a.age = 2
console.log(b.age) // 2

```

从上述例子中我们可以发现，如果给一个变量赋值一个对象，那么两者的值会是同一个引用，其中一方改变，另一方也会相应改变。

通常在开发中我们不希望出现这样的问题，我们可以使用浅拷贝来解决这个问题。

浅拷贝

首先可以通过 `Object.assign` 来解决这个问题。

```
let a = {  
  age: 1  
}  
let b = Object.assign({}, a)  
a.age = 2  
console.log(b.age) // 1
```

当然我们也可以通过展开运算符 (...) 来解决

```
let a = {  
  age: 1  
}  
let b = {...a}  
a.age = 2  
console.log(b.age) // 1
```

通常浅拷贝就能解决大部分问题了，但是当我们遇到如下情况就需要使用到深拷贝了

```
let a = {  
  age: 1,  
  jobs: {  
    first: 'FE'  
  }  
}  
let b = {...a}  
a.jobs.first = 'native'  
console.log(b.jobs.first) // native
```

浅拷贝只解决了第一层的问题，如果接下去的值中还有对象的话，那么就又回到刚开始的话题了，两者享有相同的引用。要解决这个问题，我们需要引入深拷贝。

深拷贝

这个问题通常可以通过 `JSON.parse(JSON.stringify(object))` 来解决。

```
let a = {
  age: 1,
  jobs: {
    first: 'FE'
  }
}
let b = JSON.parse(JSON.stringify(a))
a.jobs.first = 'native'
console.log(b.jobs.first) // FE
```

但是该方法也是有局限性的：

- 会忽略 `undefined`
- 会忽略 `symbol`
- 不能序列化函数
- 不能解决循环引用的对象

```
let obj = {
  a: 1,
  b: {
    c: 2,
    d: 3,
  },
}
obj.c = obj.b
obj.e = obj.a
obj.b.c = obj.c
obj.b.d = obj.b
obj.b.e = obj.b.c
let newObj = JSON.parse(JSON.stringify(obj))
console.log(newObj)
```

如果你有这么一个循环引用对象，你会发现你 cannot 通过该方法深拷贝

```
✖ ▶ Uncaught TypeError: Converting circular structure to JSON  
    at JSON.stringify (<anonymous>)  
    at <anonymous>:13:30
```

在遇到函数、`undefined` 或者 `symbol` 的时候，该对象也不能正常的序列化

```
let a = {  
  age: undefined,  
  sex: Symbol('male'),  
  jobs: function() {},  
  name: 'yck'  
}  
let b = JSON.parse(JSON.stringify(a))  
console.log(b) // {name: "yck"}
```

你会发现在上述情况中，该方法会忽略掉函数和 `undefined`。

但是在通常情况下，复杂数据都是可以序列化的，所以这个函数可以解决大部分问题，并且该函数是内置函数中处理深拷贝性能最快的。当然如果你的数据中含有以上三种情况下，可以使用 [lodash](#) 的深拷贝函数。

如果你所需拷贝的对象含有内置类型并且不包含函数，可以使用 `MessageChannel`

```
function structuralClone(obj) {  
  return new Promise(resolve => {  
    const {port1, port2} = new MessageChannel();  
    port2.onmessage = ev => resolve(ev.data);  
    port1.postMessage(obj);  
  });  
}  
  
var obj = {a: 1, b: {  
  c: b
```

```
}}  
// 注意该方法是异步的  
// 可以处理 undefined 和循环引用对象  
(async () => {  
  const clone = await structuralClone(obj)  
})()
```

模块化

在有 Babel 的情况下，我们可以直接使用 ES6 的模块化

```
// file a.js  
export function a() {}  
export function b() {}  
// file b.js  
export default function() {}  
  
import {a, b} from './a.js'  
import XXX from './b.js'
```

CommonJS

CommonJs 是 Node 独有的规范，浏览器中使用就需要用到 Browserify 解析了。

```
// a.js  
module.exports = {  
  a: 1  
}  
// or  
exports.a = 1  
  
// b.js  
var module = require('./a.js')  
module.a // -> log 1
```

在上述代码中，`module.exports` 和 `exports` 很容易混淆，让我们来看看大致内部实现

```
var module = require('./a.js')
module.a
// 这里其实就是包装了一层立即执行函数，这样就不会污染全局变量了，
// 重要的是 module 这里，module 是 Node 独有的一个变量
module.exports = {
  a: 1
}
// 基本实现
var module = {
  exports: {} // exports 就是个空对象
}
// 这个是为什么 exports 和 module.exports 用法相似的原因
var exports = module.exports
var load = function (module) {
  // 导出的东西
  var a = 1
  module.exports = a
  return module.exports
};
```

再来说说 `module.exports` 和 `exports`，用法其实是相似的，但是不能对 `exports` 直接赋值，不会有任何效果。

对于 CommonJS 和 ES6 中的模块化的两者区别是：

- 前者支持动态导入，也就是 `require(`${path}/xx.js`)`，后者目前不支持，但是已有提案
- 前者是同步导入，因为用于服务端，文件都在本地，同步导入即使卡住主线程影响也不大。而后者是异步导入，因为用于浏览器，需要下载文件，如果也采用同步导入会对渲染有很大影响
- 前者在导出时都是值拷贝，就算导出的值变了，导入的值也不会改变，所以如果想更新值，必须重新导入一次。但是后者采用实时绑定的方式，导入导出的值都指向同一个内存地址，所以导入值会跟随导出值变化

- 后者会编译成 `require/exports` 来执行的

AMD

AMD 是由 RequireJS 提出的

```
// AMD
define(['./a', './b'], function(a, b) {
    a.do()
    b.do()
})
define(function(require, exports, module) {
    var a = require('./a')
    a.doSomething()
    var b = require('./b')
    b.doSomething()
})
```

防抖

你是否在日常开发中遇到一个问题，在滚动事件中需要做个复杂计算或者实现一个按钮的防二次点击操作。

这些需求都可以通过函数防抖动来实现。尤其是第一个需求，如果在频繁的事件回调中做复杂计算，很有可能导致页面卡顿，不如将多次计算合并为一次计算，只在一个精确点做操作。

PS：防抖和节流的作用都是防止函数多次调用。区别在于，假设一个用户一直触发这个函数，且每次触发函数的间隔小于wait，防抖的情况下只会调用一次，而节流的情况会每隔一定时间（参数wait）调用函数。

我们先来看一个袖珍版的防抖理解一下防抖的实现：

```
// func是用户传入需要防抖的函数
```

```

// wait是等待时间
const debounce = (func, wait = 50) => {
  // 缓存一个定时器id
  let timer = 0
  // 这里返回的函数是每次用户实际调用的防抖函数
  // 如果已经设定过定时器了就清空上一次的定时器
  // 开始一个新的定时器，延迟执行用户传入的方法
  return function(...args) {
    if (timer) clearTimeout(timer)
    timer = setTimeout(() => {
      func.apply(this, args)
    }, wait)
  }
}
// 不难看出如果用户调用该函数的间隔小于wait的情况下，上一次的时间还未到就被清除了，并不会执行函数

```

这是一个简单版的防抖，但是有缺陷，这个防抖只能在最后调用。一般的防抖会有immediate选项，表示是否立即调用。这两者的区别，举个栗子来说：

- 例如在搜索引擎搜索问题的时候，我们当然是希望用户输入完最后一个字才调用查询接口，这个时候适用 延迟执行 的防抖函数，它总是在一连串（间隔小于wait的）函数触发之后调用。
- 例如用户给interviewMap点star的时候，我们希望用户点第一下的时候就去调用接口，并且成功之后改变star按钮的样子，用户就可以立马得到反馈是否star成功了，这个情况适用 立即执行 的防抖函数，它总是在第一次调用，并且下一次调用必须与前一次调用的时间间隔大于wait才会触发。

下面我们来实现一个带有立即执行选项的防抖函数

```

// 这个是用来获取当前时间戳的
function now() {
  return +new Date()
}
/**

```



```

* 防抖函数，返回函数连续调用时，空闲时间必须大于或等于 wait，func 才会执行
*
* @param {function} func      回调函数
* @param {number} wait        表示时间窗口的间隔
* @param {boolean} immediate   设置为ture时，是否立即调用函数
* @return {function}          返回客户调用函数
*/

function debounce (func, wait = 50, immediate = true) {
  let timer, context, args

  // 延迟执行函数
  const later = () => setTimeout(() => {
    // 延迟函数执行完毕，清空缓存的定时器序号
    timer = null
    // 延迟执行的情况下，函数会在延迟函数中执行
    // 使用到之前缓存的参数和上下文
    if (!immediate) {
      func.apply(context, args)
      context = args = null
    }
  }, wait)

  // 这里返回的函数是每次实际调用的函数
  return function(...params) {
    // 如果没有创建延迟执行函数 (later)，就创建一个
    if (!timer) {
      timer = later()
      // 如果是立即执行，调用函数
      // 否则缓存参数和调用上下文
      if (immediate) {
        func.apply(this, params)
      } else {
        context = this
        args = params
      }
    }
    // 如果已有延迟执行函数 (later)，调用的时候清除原来的并重新设定一个
  }
}

```

```

    // 这样做延迟函数会重新计时
  } else {
    clearTimeout(timer)
    timer = later()
  }
}
}
}

```

整体函数实现的不难，总结一下。

- 对于按钮防点击来说的实现：如果函数是立即执行的，就立即调用，如果函数是延迟执行的，就缓存上下文和参数，放到延迟函数中去执行。一旦我开始一个定时器，只要我定时器还在，你每次点击我都重新计时。一旦你点累了，定时器时间到，定时器重置为 `null`，就可以再次点击了。
- 对于延时执行函数来说的实现：清除定时器ID，如果是延迟调用就调用函数

节流

防抖动和节流本质是不一样的。防抖动是将多次执行变为最后一次执行，节流是将多次执行变成每隔一段时间执行。

```

/**
 * underscore 节流函数，返回函数连续调用时，func 执行频率限定为 次 / wait
 *
 * @param {function} func      回调函数
 * @param {number} wait        表示时间窗口的间隔
 * @param {object} options     如果想忽略开始函数的的调用，传入{leading:
false}。
 *                               如果想忽略结尾函数的调用，传入{trailing:
false}
 *                               两者不能共存，否则函数不能执行
 * @return {function}          返回客户调用函数
 */
_.throttle = function(func, wait, options) {
  var context, args, result;

```

```
var timeout = null;
// 之前的时间戳
var previous = 0;
// 如果 options 没传则设为空对象
if (!options) options = {};
// 定时器回调函数
var later = function() {
    // 如果设置了 leading, 就将 previous 设为 0
    // 用于下面函数的第一个 if 判断
    previous = options.leading === false ? 0 : _.now();
    // 置空一是为了防止内存泄漏, 二是为了下面的定时器判断
    timeout = null;
    result = func.apply(context, args);
    if (!timeout) context = args = null;
};
return function() {
    // 获得当前时间戳
    var now = _.now();
    // 首次进入前者肯定为 true
    // 如果需要第一次不执行函数
    // 就将上次时间戳设为当前的
    // 这样在接下来计算 remaining 的值时会大于0
    if (!previous && options.leading === false) previous = now;
    // 计算剩余时间
    var remaining = wait - (now - previous);
    context = this;
    args = arguments;
    // 如果当前调用已经大于上次调用时间 + wait
    // 或者用户手动调了时间
    // 如果设置了 trailing, 只会进入这个条件
    // 如果没有设置 leading, 那么第一次会进入这个条件
    // 还有一点, 你可能会觉得开启了定时器那么应该不会进入这个 if 条件了
    // 其实还是会进入的, 因为定时器的延时
    // 并不是准确的时间, 很可能你设置了2秒
    // 但是他需要2.2秒才触发, 这时候就会进入这个条件
    if (remaining <= 0 || remaining > wait) {
```

```

    // 如果存在定时器就清理掉否则会调用二次回调
    if (timeout) {
        clearTimeout(timeout);
        timeout = null;
    }
    previous = now;
    result = func.apply(context, args);
    if (!timeout) context = args = null;
} else if (!timeout && options.trailing !== false) {
    // 判断是否设置了定时器和 trailing
    // 没有的话就开启一个定时器
    // 并且不能同时设置 leading 和 trailing
    timeout = setTimeout(later, remaining);
}
return result;
};
};

```

继承

在 ES5 中，我们可以使用如下方式解决继承的问题

```

function Super() {}
Super.prototype.getNumber = function() {
    return 1
}

function Sub() {}
let s = new Sub()
Sub.prototype = Object.create(Super.prototype, {
    constructor: {
        value: Sub,
        enumerable: false,
        writable: true,
        configurable: true
    }
})

```

```
}  
})
```

以上继承实现思路就是将子类的原型设置为父类的原型

在 ES6 中，我们可以通过 `class` 语法轻松解决这个问题

```
class MyDate extends Date {  
  test() {  
    return this.getTime()  
  }  
}  
  
let myDate = new MyDate()  
myDate.test()
```

但是 ES6 不是所有浏览器都兼容，所以我们需要使用 Babel 来编译这段代码。

如果你使用编译过得代码调用 `myDate.test()` 你会惊奇地发现出现了报错

```
✖ ▶ Uncaught TypeError: this is not a Date object.  
    at MyDate.getDate (<anonymous>)  
    at MyDate.test (<anonymous>:23:19)  
    at <anonymous>:1:8
```

因为在 JS 底层有限制，如果不是由 `Date` 构造出来的实例的话，是不能调用 `Date` 里的函数的。所以这也侧面的说明了：**ES6 中的 `class` 继承与 ES5 中的一般继承写法是不同的。**

既然底层限制了实例必须由 `Date` 构造出来，那么我们可以改变下思路实现继承

```
function MyData() {  
  
}  
MyData.prototype.test = function () {  
  return this.getTime()  
}  
let d = new Date()  
Object.setPrototypeOf(d, MyData.prototype)  
Object.setPrototypeOf(MyData.prototype, Date.prototype)
```

以上继承实现思路：**先创建父类实例** => 改变实例原先的 `__proto__` 转而连接到子类的 `prototype` => 子类的 `prototype` 的 `__proto__` 改为父类的 `prototype`。

通过以上方法实现的继承就可以完美解决 JS 底层的这个限制。

call, apply, bind 区别

首先说下前两者的区别。

`call` 和 `apply` 都是为了解决改变 `this` 的指向。作用都是相同的，只是传参的方式不同。

除了第一个参数外，`call` 可以接收一个参数列表，`apply` 只接受一个参数数组。

```
let a = {  
  value: 1  
}  
function getValue(name, age) {  
  console.log(name)  
  console.log(age)  
  console.log(this.value)  
}  
getValue.call(a, 'yck', '24')  
getValue.apply(a, ['yck', '24'])
```

模拟实现 call 和 apply

可以从以下几点来考虑如何实现

- 不传入第一个参数，那么默认为 window
- 改变了 this 指向，让新的对象可以执行该函数。那么思路是否可以变成给新的对象添加一个函数，然后在执行完以后删除？

```
Function.prototype.myCall = function (context) {  
  var context = context || window  
  // 给 context 添加一个属性  
  // getValue.call(a, 'yck', '24') => a.fn = getValue  
  context.fn = this  
  // 将 context 后面的参数取出来  
  var args = [...arguments].slice(1)  
  // getValue.call(a, 'yck', '24') => a.fn('yck', '24')  
  var result = context.fn(...args)  
  // 删除 fn  
  delete context.fn  
  return result  
}
```

以上就是 call 的思路，apply 的实现也类似

```
Function.prototype.myApply = function (context) {  
  var context = context || window  
  context.fn = this  
  
  var result  
  // 需要判断是否存储第二个参数  
  // 如果存在，就将第二个参数展开  
  if (arguments[1]) {  
    result = context.fn(...arguments[1])  
  } else {  
    result = context.fn()  
  }  
}
```

```
}

    delete context.fn
    return result
}
```

`bind` 和其他两个方法作用也是一致的，只是该方法会返回一个函数。并且我们可以通过 `bind` 实现柯里化。

同样的，也来模拟实现下 `bind`

```
Function.prototype.myBind = function (context) {
    if (typeof this !== 'function') {
        throw new TypeError('Error')
    }
    var _this = this
    var args = [...arguments].slice(1)
    // 返回一个函数
    return function F() {
        // 因为返回了一个函数，我们可以 new F()，所以需要判断
        if (this instanceof F) {
            return new _this(...args, ...arguments)
        }
        return _this.apply(context, args.concat(...arguments))
    }
}
```

Promise 实现

Promise 是 ES6 新增的语法，解决了回调地狱的问题。

可以把 Promise 看成一个状态机。初始是 `pending` 状态，可以通过函数 `resolve` 和 `reject`，将状态转变为 `resolved` 或者 `rejected` 状态，状态一旦改变就不能再次变化。

then 函数会返回一个 Promise 实例，并且该返回值是一个新的实例而不是之前的实例。因为 Promise 规范规定除了 pending 状态，其他状态是不可以改变的，如果返回的是一个相同实例的话，多个 then 调用就失去意义了。

对于 then 来说，本质上可以把它看成是 flatMap

```
// 三种状态
const PENDING = "pending";
const RESOLVED = "resolved";
const REJECTED = "rejected";
// promise 接收一个函数参数，该函数会立即执行
function MyPromise(fn) {
  let _this = this;
  _this.currentState = PENDING;
  _this.value = undefined;
  // 用于保存 then 中的回调，只有当 promise
  // 状态为 pending 时才会缓存，并且每个实例至多缓存一个
  _this.resolvedCallbacks = [];
  _this.rejectedCallbacks = [];

  _this.resolve = function (value) {
    if (value instanceof MyPromise) {
      // 如果 value 是个 Promise，递归执行
      return value.then(_this.resolve, _this.reject)
    }
    setTimeout(() => { // 异步执行，保证执行顺序
      if (_this.currentState === PENDING) {
        _this.currentState = RESOLVED;
        _this.value = value;
        _this.resolvedCallbacks.forEach(cb => cb());
      }
    })
  };

  _this.reject = function (reason) {
```

```

    setTimeout(() => { // 异步执行，保证执行顺序
      if (_this.currentState === PENDING) {
        _this.currentState = REJECTED;
        _this.value = reason;
        _this.rejectedCallbacks.forEach(cb => cb());
      }
    })
  }
}
// 用于解决以下问题
// new Promise(() => throw Error('error'))
try {
  fn(_this.resolve, _this.reject);
} catch (e) {
  _this.reject(e);
}
}

```

```

MyPromise.prototype.then = function (onResolved, onRejected) {
  var self = this;
  // 规范 2.2.7, then 必须返回一个新的 promise
  var promise2;
  // 规范 2.2.onResolved 和 onRejected 都为可选参数
  // 如果类型不是函数需要忽略，同时也实现了透传
  // Promise.resolve(4).then().then((value) => console.log(value))
  onResolved = typeof onResolved === 'function' ? onResolved : v => v;
  onRejected = typeof onRejected === 'function' ? onRejected : r =>
  throw r;

```

```

  if (self.currentState === RESOLVED) {
    return (promise2 = new MyPromise(function (resolve, reject) {
      // 规范 2.2.4, 保证 onFulfilled, onRejected 异步执行
      // 所以用了 setTimeout 包裹下
      setTimeout(function () {
        try {
          var x = onResolved(self.value);
          resolutionProcedure(promise2, x, resolve, reject);

```

```
        } catch (reason) {
            reject(reason);
        }
    });
}));
}
```

```
if (self.currentState === REJECTED) {
    return (promise2 = new MyPromise(function (resolve, reject) {
        setTimeout(function () {
            // 异步执行onRejected
            try {
                var x = onRejected(self.value);
                resolutionProcedure(promise2, x, resolve, reject);
            } catch (reason) {
                reject(reason);
            }
        });
    }));
}
```

```
if (self.currentState === PENDING) {
    return (promise2 = new MyPromise(function (resolve, reject) {
        self.resolvedCallbacks.push(function () {
            // 考虑到可能会有报错，所以使用 try/catch 包裹
            try {
                var x = onResolved(self.value);
                resolutionProcedure(promise2, x, resolve, reject);
            } catch (r) {
                reject(r);
            }
        });
    }));
```

```
    self.rejectedCallbacks.push(function () {
        try {
            var x = onRejected(self.value);
```

```

        resolutionProcedure(promise2, x, resolve, reject);
    } catch (r) {
        reject(r);
    }
});
}));
}
};

// 规范 2.3
function resolutionProcedure(promise2, x, resolve, reject) {
    // 规范 2.3.1, x 不能和 promise2 相同, 避免循环引用
    if (promise2 === x) {
        return reject(new TypeError("Error"));
    }
    // 规范 2.3.2
    // 如果 x 为 Promise, 状态为 pending 需要继续等待否则执行
    if (x instanceof MyPromise) {
        if (x.currentState === PENDING) {
            x.then(function (value) {
                // 再次调用该函数是为了确认 x resolve 的
                // 参数是什么类型, 如果是基本类型就再次 resolve
                // 把值传给下个 then
                resolutionProcedure(promise2, value, resolve, reject);
            }, reject);
        } else {
            x.then(resolve, reject);
        }
        return;
    }
    // 规范 2.3.3.3
    // reject 或者 resolve 其中一个执行过得话, 忽略其他的
    let called = false;
    // 规范 2.3.3, 判断 x 是否为对象或者函数
    if (x !== null && (typeof x === "object" || typeof x === "function"))
    {
        // 规范 2.3.3.2, 如果不能取出 then, 就 reject
    }
}

```

```
try {
  // 规范 2.3.3.1
  let then = x.then;
  // 如果 then 是函数, 调用 x.then
  if (typeof then === "function") {
    // 规范 2.3.3.3
    then.call(
      x,
      y => {
        if (called) return;
        called = true;
        // 规范 2.3.3.3.1
        resolutionProcedure(promise2, y, resolve, reject);
      },
      e => {
        if (called) return;
        called = true;
        reject(e);
      }
    );
  } else {
    // 规范 2.3.3.4
    resolve(x);
  }
} catch (e) {
  if (called) return;
  called = true;
  reject(e);
}
} else {
  // 规范 2.3.4, x 为基本类型
  resolve(x);
}
}
```

以上就是根据 Promise / A+ 规范来实现的代码，可以通过 `promises-aplus-tests` 的完整测试

```
The value is `true` with `Boolean.prototype` modified to have a `then` method
✓ already-fulfilled
✓ immediately-fulfilled
✓ eventually-fulfilled
✓ already-rejected
✓ immediately-rejected
✓ eventually-rejected
The value is `1` with `Number.prototype` modified to have a `then` method
✓ already-fulfilled
✓ immediately-fulfilled
✓ eventually-fulfilled
✓ already-rejected
✓ immediately-rejected
✓ eventually-rejected

872 passing (15s)

t yuchengkai at yuchengkaideiMac.local in ~/Desktop/test [18:45:53]
> promises-aplus-tests index.js
```

Generator 实现

Generator 是 ES6 中新增的语法，和 Promise 一样，都可以用来异步编程

```
// 使用 * 表示这是一个 Generator 函数
// 内部可以通过 yield 暂停代码
// 通过调用 next 恢复执行
function* test() {
  let a = 1 + 2;
  yield 2;
  yield 3;
}
let b = test();
console.log(b.next()); // > { value: 2, done: false }
console.log(b.next()); // > { value: 3, done: false }
console.log(b.next()); // > { value: undefined, done: true }
```

从以上代码可以发现，加上 `*` 的函数执行后拥有了 `next` 函数，也就是说函数执行后返回了一个对象。每次调用 `next` 函数可以继续执行被暂停的代码。以下是 Generator 函数的简单实现

```
// cb 也就是编译过的 test 函数
function generator(cb) {
  return (function() {
    var object = {
      next: 0,
      stop: function() {}
    };

    return {
      next: function() {
        var ret = cb(object);
        if (ret === undefined) return { value: undefined, done: true };
        return {
          value: ret,
          done: false
        };
      }
    };
  })();
}

// 如果你使用 babel 编译后可以发现 test 函数变成了这样
function test() {
  var a;
  return generator(function(_context) {
    while (1) {
      switch ((_context.prev = _context.next)) {
        // 可以发现通过 yield 将代码分割成几块
        // 每次执行 next 函数就执行一块代码
        // 并且表明下次需要执行哪块代码
        case 0:
          a = 1 + 2;

```

```

        _context.next = 4;
        return 2;
    case 4:
        _context.next = 6;
        return 3;
    // 执行完毕
    case 6:
    case "end":
        return _context.stop();
    }
}
});
}

```

Map、FlatMap 和 Reduce

Map 作用是生成一个新数组，遍历原数组，将每个元素拿出来做一些变换然后 append 到新的数组中。

```

[1, 2, 3].map((v) => v + 1)
// -> [2, 3, 4]

```

Map 有三个参数，分别是当前索引元素，索引，原数组

```

['1', '2', '3'].map(parseInt)
// parseInt('1', 0) -> 1
// parseInt('2', 1) -> NaN
// parseInt('3', 2) -> NaN

```

FlatMap 和 map 的作用几乎是相同的，但是对于多维数组来说，会将原数组降维。可以将 FlatMap 看成是 map + flatten，目前该函数在浏览器中还不支持。

```

[1, [2], 3].flatMap((v) => v + 1)
// -> [2, 3, 4]

```


如果想将一个多维数组彻底的降维，可以这样实现

```
const flattenDeep = (arr) => Array.isArray(arr)
  ? arr.reduce( (a, b) => [...a, ...flattenDeep(b)] , [])
  : [arr]

flattenDeep([1, [[2], [3, [4]], 5]])
```

Reduce 作用是数组中的值组合起来，最终得到一个值

```
function a() {
  console.log(1);
}

function b() {
  console.log(2);
}

[a, b].reduce((a, b) => a(b()))
// -> 2 1
```

async 和 await

一个函数如果加上 `async`，那么该函数就会返回一个 `Promise`

```
async function test() {
  return "1";
}

console.log(test()); // -> Promise {<resolved>: "1"}
```

可以把 `async` 看成将函数返回值使用 `Promise.resolve()` 包裹了下。

`await` 只能在 `async` 函数中使用

```

function sleep() {
  return new Promise(resolve => {
    setTimeout(() => {
      console.log('finish')
      resolve("sleep");
    }, 2000);
  });
}
async function test() {
  let value = await sleep();
  console.log("object");
}
test()

```

上面代码会先打印 finish 然后再打印 object 。因为 await 会等待 sleep 函数 resolve ，所以即使后面是同步代码，也不会先去执行同步代码再来执行异步代码。

async 和 await 相比直接使用 Promise 来说，优势在于处理 then 的调用链，能够更清晰准确的写出代码。缺点在于滥用 await 可能会导致性能问题，因为 await 会阻塞代码，也许之后的异步代码并不依赖于前者，但仍然需要等待前者完成，导致代码失去了并行性。

下面来看一个使用 await 的代码。

```

var a = 0
var b = async () => {
  a = a + await 10
  console.log('2', a) // -> '2' 10
  a = (await 10) + a
  console.log('3', a) // -> '3' 20
}
b()
a++
console.log('1', a) // -> '1' 1

```

对于以上代码你可能会有疑惑，这里说明下原理

- 首先函数 b 先执行，在执行到 `await 10` 之前变量 a 还是 0，因为在 `await` 内部实现了 `generators`，`generators` 会保留堆栈中东西，所以这时候 `a = 0` 被保存了下来
- 因为 `await` 是异步操作，遇到 `await` 就会立即返回一个 `pending` 状态的 `Promise` 对象，暂时返回执行代码的控制权，使得函数外的代码得以继续执行，所以会先执行 `console.log('1', a)`
- 这时候同步代码执行完毕，开始执行异步代码，将保存下来的值拿出来使用，这时候 `a = 10`
- 然后后面就是常规执行代码了

Proxy

Proxy 是 ES6 中新增的功能，可以用来自定义对象中的操作

```
let p = new Proxy(target, handler);  
// `target` 代表需要添加代理的对象  
// `handler` 用来自定义对象中的操作
```

可以很方便的使用 Proxy 来实现一个数据绑定和监听

```
let onWatch = (obj, setBind, getLogger) => {  
  let handler = {  
    get(target, property, receiver) {  
      getLogger(target, property)  
      return Reflect.get(target, property, receiver);  
    },  
    set(target, property, value, receiver) {  
      setBind(value);  
      return Reflect.set(target, property, value);  
    }  
  };  
  return new Proxy(obj, handler);  
};  
  
let obj = { a: 1 }
```

```

let value
let p = onWatch(obj, (v) => {
  value = v
}, (target, property) => {
  console.log(`Get '${property}' = ${target[property]}`);
})
p.a = 2 // bind `value` to `2`
p.a // -> Get 'a' = 2

```

为什么 $0.1 + 0.2 \neq 0.3$

因为 JS 采用 IEEE 754 双精度版本（64位），并且只要采用 IEEE 754 的语言都有该问题。

我们都知道计算机表示十进制是采用二进制表示的，所以 0.1 在二进制表示为

```

// (0011) 表示循环
0.1 =  $2^{-4} * 1.10011(0011)$ 

```

那么如何得到这个二进制的呢，我们可以来演算下

$$\begin{array}{r}
 0.1 \\
 \times 2 \\
 \hline
 0.2 \quad 0 \\
 \times 2 \\
 \hline
 0.4 \quad 0 \\
 \times 2 \\
 \hline
 0.8 \quad 0
 \end{array}$$

$$\begin{array}{r}
 \times \quad 2 \\
 \hline
 0.1 \quad 1
 \end{array}$$

$$\begin{array}{r}
 \times \quad 2 \\
 \hline
 0.2 \quad 1
 \end{array}$$

$$\begin{array}{r}
 \times \quad 2 \\
 \hline
 0.4 \quad 0
 \end{array}$$

$$\begin{array}{r}
 \times \quad 2 \\
 \hline
 0.8 \quad 0
 \end{array}$$

$$\begin{array}{r}
 \times \quad 2 \\
 \hline
 1.6 \quad 1
 \end{array}$$

小数算二进制和整数不同。乘法计算时，只计算小数位，整数位用作每一位的二进制，并且得到的第一位为最高位。所以我们得出 $0.1 = 2^{-4} * 1.10011(0011)$ ，那么 0.2 的演算也基本如上所示，只需要去掉第一步乘法，所以得出 $0.2 = 2^{-3} * 1.10011(0011)$ 。

回来继续说 IEEE 754 双精度。六十四位中符号位占一位，整数位占十一位，其余五十二位都为小数位。因为 0.1 和 0.2 都是无限循环的二进制了，所以在小数位末尾处需要判断是否进位（就和十进制的四舍五入一样）。

所以 $2^{-4} * 1.10011...001$ 进位后就变成了 $2^{-4} * 1.10011(0011 * 12次)010$ 。那么把这两个二进制加起来会得出 $2^{-2} * 1.0011(0011 * 11次)0100$ ，这个值算成十进制就是 0.300000000000000004

下面说一下原生解决办法，如下代码所示

```
parseFloat((0.1 + 0.2).toFixed(10))
```

正则表达式

元字符

元字符	作用
.	匹配任意字符除了换行符和回车符
[]	匹配方括号内的任意字符。比如 [0-9] 就可以用来匹配任意数字
^	^9，这样使用代表匹配以 9 开头。[^ 9]，这样使用代表不匹配方括号内除了 9 的字符
{1, 2}	匹配 1 到 2 位字符
(yck)	只匹配和 yck 相同字符串
	匹配 前后任意字符
\	转义
*	只匹配出现 0 次及以上 * 前的字符
+	只匹配出现 1 次及以上 + 前的字符
?	? 之前字符可选

修饰语

修饰语	作用
i	忽略大小写
g	全局搜索
m	多行

字符简写

简写	作用
\w	匹配字母数字或下划线
\W	和上面相反
\s	匹配任意的空白符
\S	和上面相反
\d	匹配数字
\D	和上面相反
\b	匹配单词的开始或结束
\B	和上面相反

V8 下的垃圾回收机制

V8 实现了准确式 GC，GC 算法采用了分代式垃圾回收机制。因此，V8 将内存（堆）分为新生代和老生代两部分。

新生代算法

新生代中的对象一般存活时间较短，使用 Scavenge GC 算法。

在新生代空间中，内存空间分为两部分，分别为 From 空间和 To 空间。在这两个空间中，必定有一个空间是使用的，另一个空间是空闲的。新分配的对象会被放入 From 空间中，当 From 空间被占满时，新生代 GC 就会启动了。算法会检查 From 空间中存活的对象并复制到 To 空间中，如果有失活的对象就会销毁。当复制完成后将 From 空间和 To 空间互换，这样 GC 就结束了。

老生代算法

老生代中的对象一般存活时间较长且数量也多，使用了两个算法，分别是标记清除算法和标记压缩算法。

在讲算法前，先来说下什么情况下对象会出现在老生代空间中：

- 新生代中的对象是否已经经历过一次 Scavenge 算法，如果经历过的话，会将对象从新生代空间移到老年代空间中。
- To 空间的对象占比大小超过 25 %。在这种情况下，为了不影响到内存分配，会将对象从新生代空间移到老年代空间中。

老年代中的空间很复杂，有如下几个空间

```
enum AllocationSpace {  
    // TODO(v8:7464): Actually map this space's memory as read-only.  
    RO_SPACE,      // 不变的对象空间  
    NEW_SPACE,     // 新生代用于 GC 复制算法的空间  
    OLD_SPACE,     // 老年代常驻对象空间  
    CODE_SPACE,    // 老年代代码对象空间  
    MAP_SPACE,     // 老年代 map 对象  
    LO_SPACE,      // 老年代大空间对象  
    NEW_LO_SPACE,  // 新生代大空间对象  
  
    FIRST_SPACE = RO_SPACE,  
    LAST_SPACE = NEW_LO_SPACE,  
    FIRST_GROWABLE_PAGED_SPACE = OLD_SPACE,  
    LAST_GROWABLE_PAGED_SPACE = MAP_SPACE  
};
```

在老年代中，以下情况会先启动标记清除算法：

- 某一个空间没有分块的时候
- 空间中被对象超过一定限制
- 空间不能保证新生代中的对象移动到老年代中

在这个阶段中，会遍历堆中所有的对象，然后标记活的对象，在标记完成后，销毁所有没有被标记的对象。在标记大型对内存时，可能需要几百毫秒才能完成一次标记。这就会导致一些性能上的问题。为了解决这个问题，2011 年，V8 从 stop-the-world 标记切换到增量标志。在增量标记期间，GC 将标记工作分解为更小的模块，可以让 JS 应用逻辑在模块间隙执行一会，从而不至于让应用出现停顿情况。但在 2018 年，GC 技术又有了一个重大突破，这项技术名为并发标记。该技术可以让 GC 扫描和标记对象时，同时允许 JS 运行，你可以点击

[该博客](#) 详细阅读。

清除对象后会造成堆内存出现碎片的情况，当碎片超过一定限制后会启动压缩算法。在压缩过程中，将活的对象像一端移动，直到所有对象都移动完成然后清理掉不需要的内存。

网络章节

事件机制

事件触发三阶段

事件触发有三个阶段

- window 往事件触发处传播，遇到注册的捕获事件会触发
- 传播到事件触发处时触发注册的事件
- 从事件触发处往 window 传播，遇到注册的冒泡事件会触发

事件触发一般来说会按照上面的顺序进行，但是也有特例，如果给一个目标节点同时注册冒泡和捕获事件，事件触发会按照注册的顺序执行。

```
// 以下会先打印冒泡然后是捕获
node.addEventListener('click',(event) =>{
  console.log('冒泡')
},false);
node.addEventListener('click',(event) =>{
  console.log('捕获 ')
},true)
```

注册事件

通常我们使用 `addEventListener` 注册事件，该函数的第三个参数可以是布尔值，也可以是对象。对于布尔值 `useCapture` 参数来说，该参数默认值为 `false`。 `useCapture` 决定了注册的事件是捕获事件还是冒泡事件。对于对象参数来说，可以使用以下几个属性

- `capture`，布尔值，和 `useCapture` 作用一样
- `once`，布尔值，值为 `true` 表示该回调只会调用一次，调用后会移除监听
- `passive`，布尔值，表示永远不会调用 `preventDefault`

一般来说，我们只希望事件只触发在目标上，这时候可以使用 `stopPropagation` 来阻止事件的进一步传播。通常我们认为 `stopPropagation` 是用来阻止事件冒泡的，其实该函数也可以阻止捕获事件。`stopImmediatePropagation` 同样也能实现阻止事件，但是还能阻止该事件目标执行别的注册事件。

```
node.addEventListener('click',(event) =>{
  event.stopImmediatePropagation()
  console.log('冒泡')
},false);
// 点击 node 只会执行上面的函数，该函数不会执行
node.addEventListener('click',(event) => {
  console.log('捕获 ')
},true)
```

事件代理

如果一个节点中的子节点是动态生成的，那么子节点需要注册事件的话应该注册在父节点上

```
<ul id="ul">
  <li>1</li>
  <li>2</li>
  <li>3</li>
  <li>4</li>
  <li>5</li>
</ul>
<script>
  let ul = document.querySelector('#ul')
  ul.addEventListener('click', (event) => {
    console.log(event.target);
  })
</script>
```

事件代理的方式相对于直接给目标注册事件来说，有以下优点

- 节省内存
- 不需要给子节点注销事件

跨域

因为浏览器出于安全考虑，有同源策略。也就是说，如果协议、域名或者端口有一个不同就是跨域，Ajax 请求会失败。

我们可以通过以下几种常用方法解决跨域的问题

JSONP

JSONP 的原理很简单，就是利用 `<script>` 标签没有跨域限制的漏洞。通过 `<script>` 标签指向一个需要访问的地址并提供一个回调函数来接收数据当需要通讯时。

```
<script src="http://domain/api?param1=a&param2=b&callback=jsonp">
</script>
<script>
    function jsonp(data) {
        console.log(data)
    }
</script>
```

JSONP 使用简单且兼容性不错，但是只限于 get 请求。

在开发中可能会遇到多个 JSONP 请求的回调函数名是相同的，这时候就需要自己封装一个 JSONP，以下是简单实现

```
function jsonp(url, jsonpCallback, success) {
    let script = document.createElement("script");
    script.src = url;
    script.async = true;
    script.type = "text/javascript";
    window[jsonpCallback] = function(data) {
        success && success(data);
    };
    document.body.appendChild(script);
}
jsonp(
    "http://xxx",
    "callback",
    function(value) {
        console.log(value);
    }
);
```

CORS

CORS需要浏览器和后端同时支持。IE 8 和 9 需要通过 XDomainRequest 来实现。

浏览器会自动进行 CORS 通信，实现CORS通信的关键是后端。只要后端实现了 CORS，就实现了跨域。

服务端设置 Access-Control-Allow-Origin 就可以开启 CORS。该属性表示哪些域名可以访问资源，如果设置通配符则表示所有网站都可以访问资源。

document.domain

该方式只能用于二级域名相同的情况下，比如 a.test.com 和 b.test.com 适用于该方式。

只需要给页面添加 document.domain = 'test.com' 表示二级域名都相同就可以实现跨域

postMessage

这种方式通常用于获取嵌入页面中的第三方页面数据。一个页面发送消息，另一个页面判断来源并接收消息

```
// 发送消息端
window.parent.postMessage('message', 'http://test.com');
// 接收消息端
var mc = new MessageChannel();
mc.addEventListener('message', (event) => {
    var origin = event.origin || event.originalEvent.origin;
    if (origin === 'http://test.com') {
        console.log('验证通过')
    }
});
```

Event loop

众所周知 JS 是一门非阻塞单线程语言，因为在最初 JS 就是为了和浏览器交互而诞生的。如果 JS 是一门多线程的语言的话，我们在多个线程中处理 DOM 就可能会发生问题（一个线程中新加节点，另一个线程中删除节点），当然可以引入读写锁解决这个问题。

JS 在运行的过程中会产生执行环境，这些执行环境会被顺序的加入到执行栈中。如果遇到异步的代码，会被挂起并加入到 Task（有多种 task）队列中。一旦执行栈为空，Event Loop 就会从 Task 队列中拿出需要执行的代码并放入执行栈中执行，所以本质上来说 JS 中的异步还是同步行为。

```
console.log('script start');

setTimeout(function() {
  console.log('setTimeout');
}, 0);

console.log('script end');
```

以上代码虽然 `setTimeout` 延时为 0，其实还是异步。这是因为 HTML5 标准规定这个函数第二个参数不得小于 4 毫秒，不足会自动增加。所以 `setTimeout` 还是会在 `script end` 之后打印。

不同的任务源会被分配到不同的 Task 队列中，任务源可以分为微任务（microtask）和宏任务（macrotask）。在 ES6 规范中，microtask 称为 `jobs`，macrotask 称为 `task`。

```
console.log('script start');

setTimeout(function() {
  console.log('setTimeout');
}, 0);

new Promise((resolve) => {
  console.log('Promise')
  resolve()
})
```

```
}).then(function() {
  console.log('promise1');
}).then(function() {
  console.log('promise2');
});

console.log('script end');
// script start => Promise => script end => promise1 => promise2 =>
setTimeout
```

以上代码虽然 `setTimeout` 写在 `Promise` 之前，但是因为 `Promise` 属于微任务而 `setTimeout` 属于宏任务，所以会有以上的打印。

微任务包括 `process.nextTick` , `promise` , `Object.observe` , `MutationObserver`

宏任务包括 `script` , `setTimeout` , `setInterval` , `setImmediate` , `I/O` , `UI rendering`

很多人有个误区，认为微任务快于宏任务，其实是错误的。因为宏任务中包括了 `script` , 浏览器会先执行一个宏任务，接下来有异步代码的话就先执行微任务。

所以正确的一次 Event loop 顺序是这样的

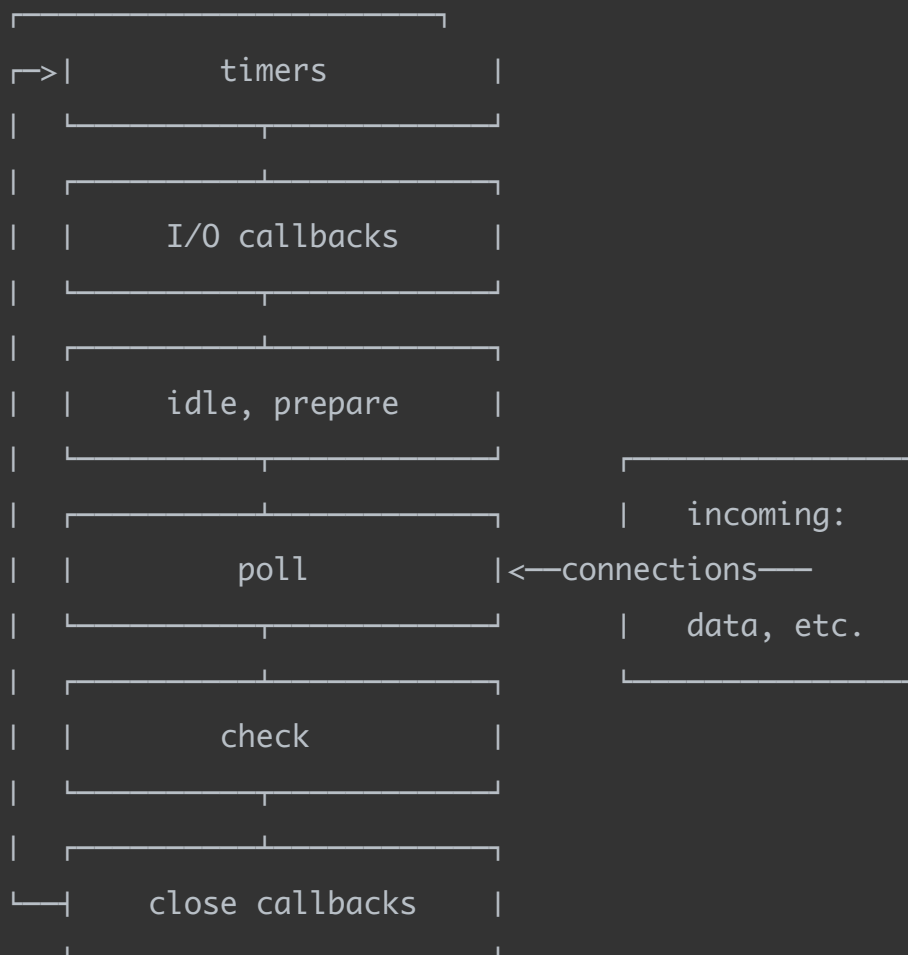
1. 执行同步代码，这属于宏任务
2. 执行栈为空，查询是否有微任务需要执行
3. 执行所有微任务
4. 必要的话渲染 UI
5. 然后开始下一轮 Event loop，执行宏任务中的异步代码

通过上述的 Event loop 顺序可知，如果宏任务中的异步代码有大量的计算并且需要操作 DOM 的话，为了更快的 界面响应，我们可以把操作 DOM 放入微任务中。

Node 中的 Event loop

Node 中的 Event loop 和浏览器中的不相同。

Node 的 Event loop 分为6个阶段，它们会按照顺序反复运行



timer

timers 阶段会执行 `setTimeout` 和 `setInterval`

一个 timer 指定的时间并不是准确时间，而是在达到这个时间后尽快执行回调，可能会因为系统正在执行别的事务而延迟。

下限的时间有一个范围： `[1, 2147483647]`，如果设定的时间不在这个范围，将被设置为 1。

I/O

I/O 阶段会执行除了 close 事件，定时器和 `setImmediate` 的回调

idle, prepare

idle, prepare 阶段内部实现

poll

poll 阶段很重要，这一阶段中，系统会做两件事情

1. 执行到点的定时器
2. 执行 poll 队列中的事件

并且当 poll 中没有定时器的情况下，会发现以下两件事情

- 如果 poll 队列不为空，会遍历回调队列并同步执行，直到队列为空或者系统限制
- 如果 poll 队列为空，会有两件事发生
 - 如果有 `setImmediate` 需要执行，poll 阶段会停止并且进入到 check 阶段执行 `setImmediate`
 - 如果没有 `setImmediate` 需要执行，会等待回调被加入到队列中并立即执行回调

如果有别的定时器需要被执行，会回到 timer 阶段执行回调。

check

check 阶段执行 `setImmediate`

close callbacks

close callbacks 阶段执行 close 事件

并且在 Node 中，有些情况下的定时器执行顺序是随机的

```

setTimeout(() => {
  console.log('setTimeout');
}, 0);
setImmediate(() => {
  console.log('setImmediate');
})
// 这里可能会输出 setTimeout, setImmediate
// 可能也会相反的输出, 这取决于性能
// 因为可能进入 event loop 用了不到 1 毫秒, 这时候会执行 setImmediate
// 否则会执行 setTimeout

```

当然在这种情况下, 执行顺序是相同的

```

var fs = require('fs')

fs.readFile(__filename, () => {
  setTimeout(() => {
    console.log('timeout');
  }, 0);
  setImmediate(() => {
    console.log('immediate');
  });
});
// 因为 readFile 的回调在 poll 中执行
// 发现有 setImmediate, 所以会立即跳到 check 阶段执行回调
// 再去 timer 阶段执行 setTimeout
// 所以以上输出一定是 setImmediate, setTimeout

```

上面介绍的都是 macrotask 的执行情况, microtask 会在以上每个阶段完成后立即执行。

```

setTimeout(()=>{
  console.log('timer1')

  Promise.resolve().then(function() {

```

```

        console.log('promise1')
    })
}, 0)

setTimeout(()=>{
    console.log('timer2')

    Promise.resolve().then(function() {
        console.log('promise2')
    })
}, 0)

```

// 以上代码在浏览器和 node 中打印情况是不同的
 // 浏览器中一定打印 timer1, promise1, timer2, promise2
 // node 中可能打印 timer1, timer2, promise1, promise2
 // 也可能打印 timer1, promise1, timer2, promise2

Node 中的 `process.nextTick` 会先于其他 microtask 执行。

```

setTimeout(() => {
    console.log("timer1");

    Promise.resolve().then(function() {
        console.log("promise1");
    });
}, 0);

process.nextTick(() => {
    console.log("nextTick");
});
// nextTick, timer1, promise1

```

存储

cookie, localStorage, sessionStorage, indexDB

特性	cookie	localStorage	sessionStorage	indexDB
数据生命周期	一般由服务器生成，可以设置过期时间	除非被清理，否则一直存在	页面关闭就清理	除非被清理，否则一直存在
数据存储大小	4K	5M	5M	无限
与服务端通信	每次都会携带在 header 中，对于请求性能影响	不参与	不参与	不参与

从上表可以看到， cookie 已经不建议用于存储。如果没有大量数据存储需求的话，可以使用 localStorage 和 sessionStorage 。对于不怎么改变的数据尽量使用 localStorage 存储，否则可以用 sessionStorage 存储。

对于 cookie ，我们还需要注意安全性。

属性	作用
value	如果用于保存用户登录态，应该将该值加密，不能使用明文的用户标识
http-only	不能通过 JS 访问 Cookie，减少 XSS 攻击
secure	只能在协议为 HTTPS 的请求中携带
same-site	规定浏览器不能在跨域请求中携带 Cookie，减少 CSRF 攻击

Service Worker

Service workers 本质上充当Web应用程序与浏览器之间的代理服务器，也可以在网络可用时作为浏览器和网络间的代理。它们旨在（除其他之外）使得能够创建有效的离线体验，拦截网络请求并基于网络是否可用以及更新的资源是否驻留在服务器上采取适当的动作。他们还允许访问推送通知和后台同步API。

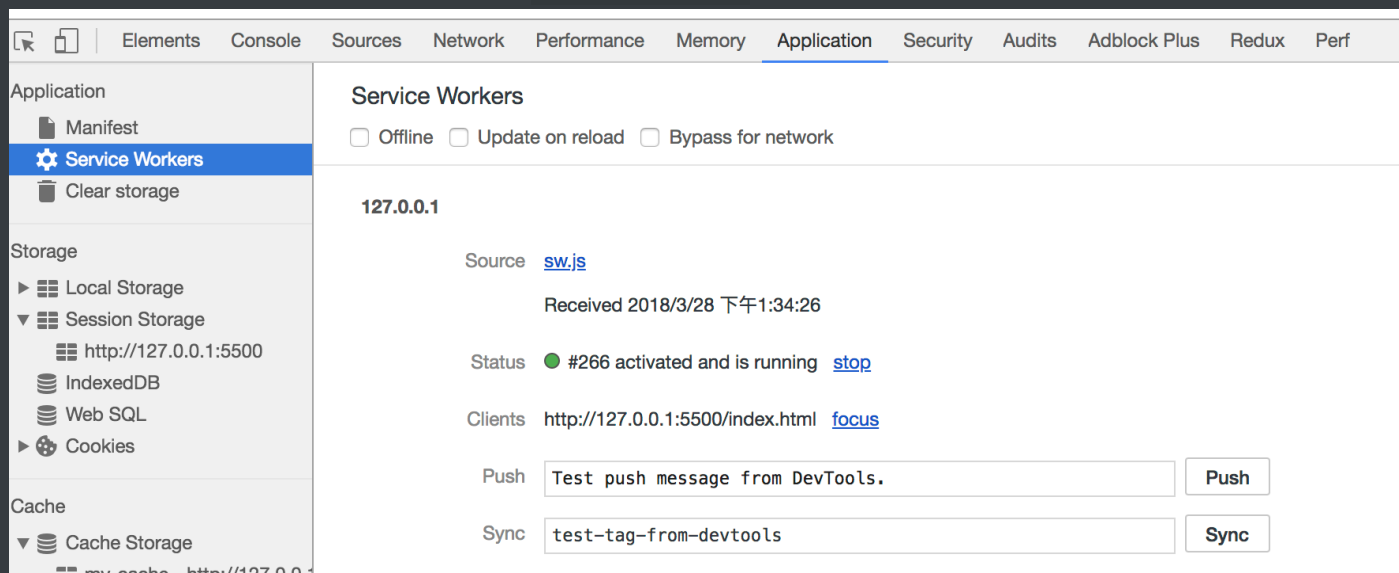
目前该技术通常用来做缓存文件，提高首屏速度，可以试着来实现这个功能。

```
// index.js
if (navigator.serviceWorker) {
  navigator.serviceWorker
    .register("sw.js")
    .then(function(registration) {
      console.log("service worker 注册成功");
    })
    .catch(function(err) {
      console.log("servcie worker 注册失败");
    });
}

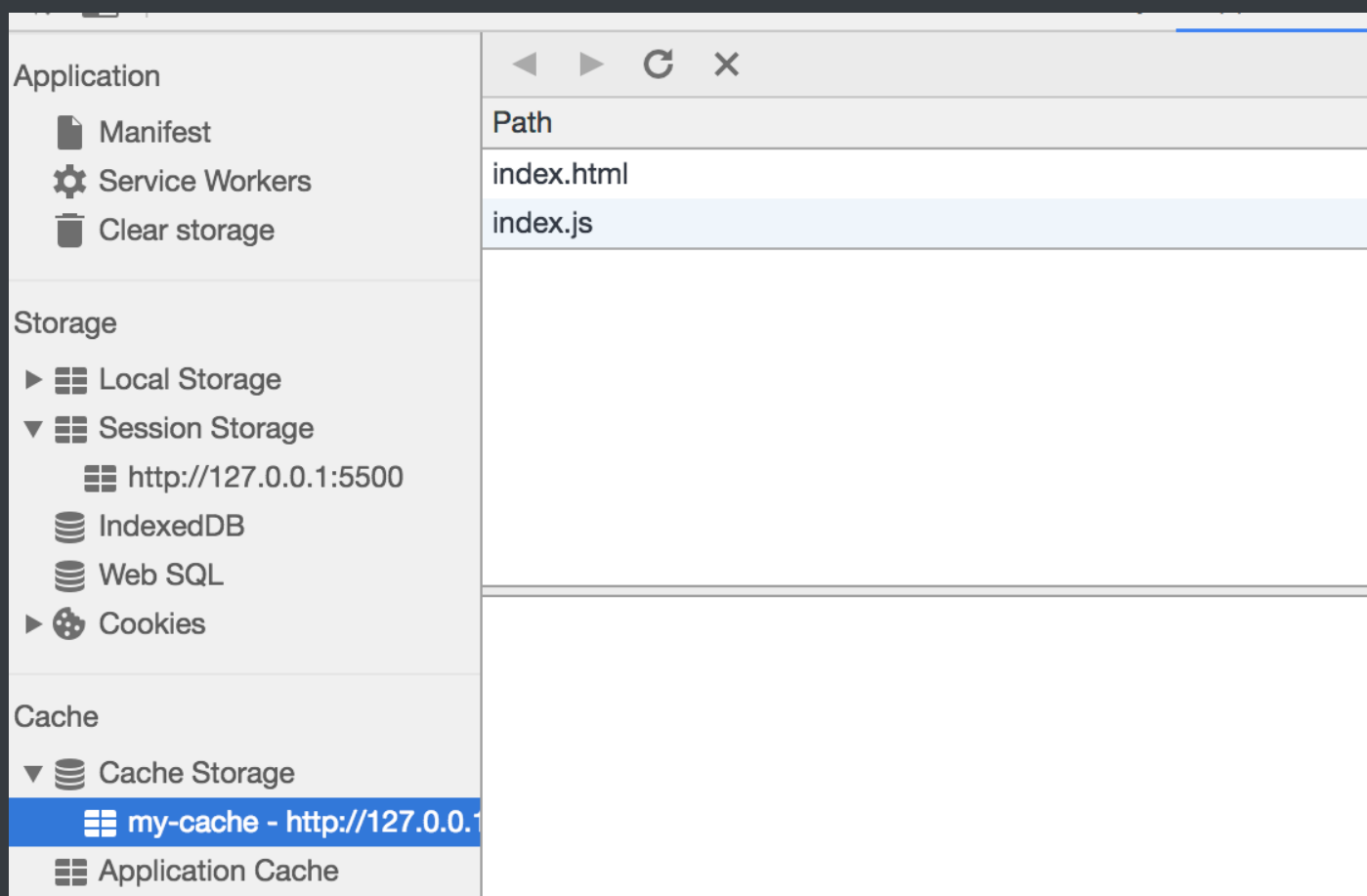
// sw.js
// 监听 `install` 事件，回调中缓存所需文件
self.addEventListener("install", e => {
  e.waitUntil(
    caches.open("my-cache").then(function(cache) {
      return cache.addAll(["./index.html", "./index.js"]);
    })
  );
});

// 拦截所有请求事件
// 如果缓存中已经有请求的数据就直接用缓存，否则去请求数据
self.addEventListener("fetch", e => {
  e.respondWith(
    caches.match(e.request).then(function(response) {
      if (response) {
        return response;
      }
      console.log("fetch source");
    })
  );
});
```

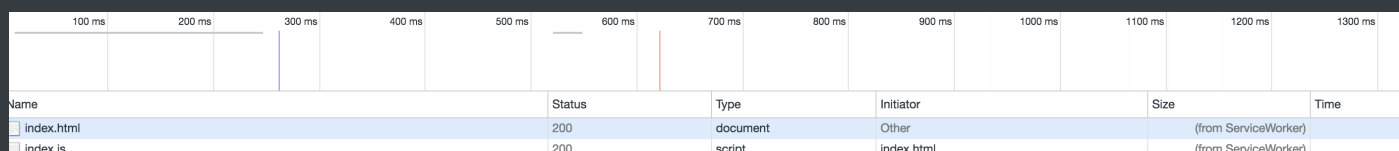
打开页面，可以在开发者工具中的 Application 看到 Service Worker 已经启动了



在 Cache 中也可以发现我们所需的文件已被缓存



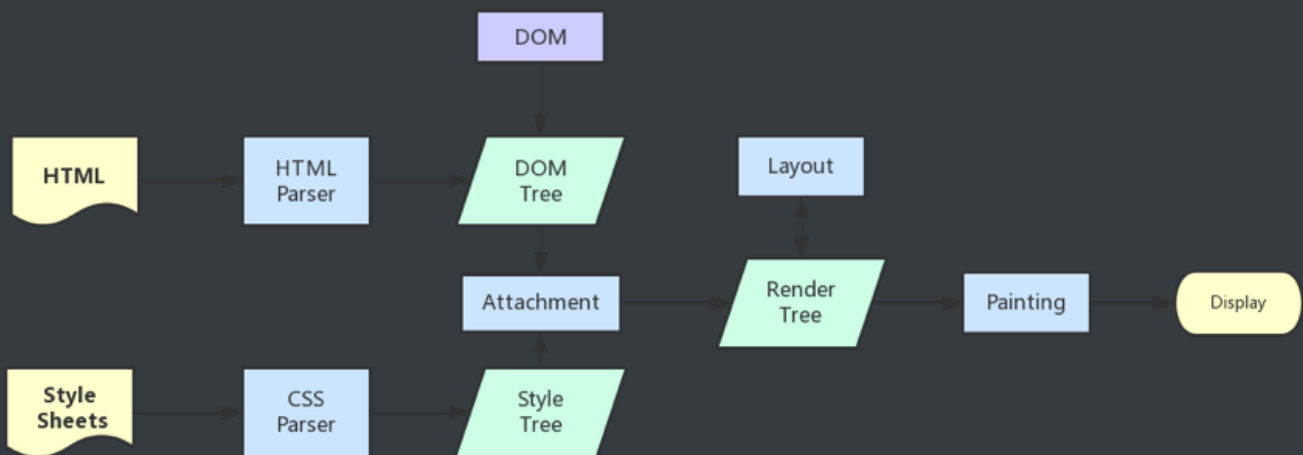
当我们重新刷新页面可以发现我们缓存的数据是从 Service Worker 中读取的



渲染机制

浏览器的渲染机制一般分为以下几个步骤

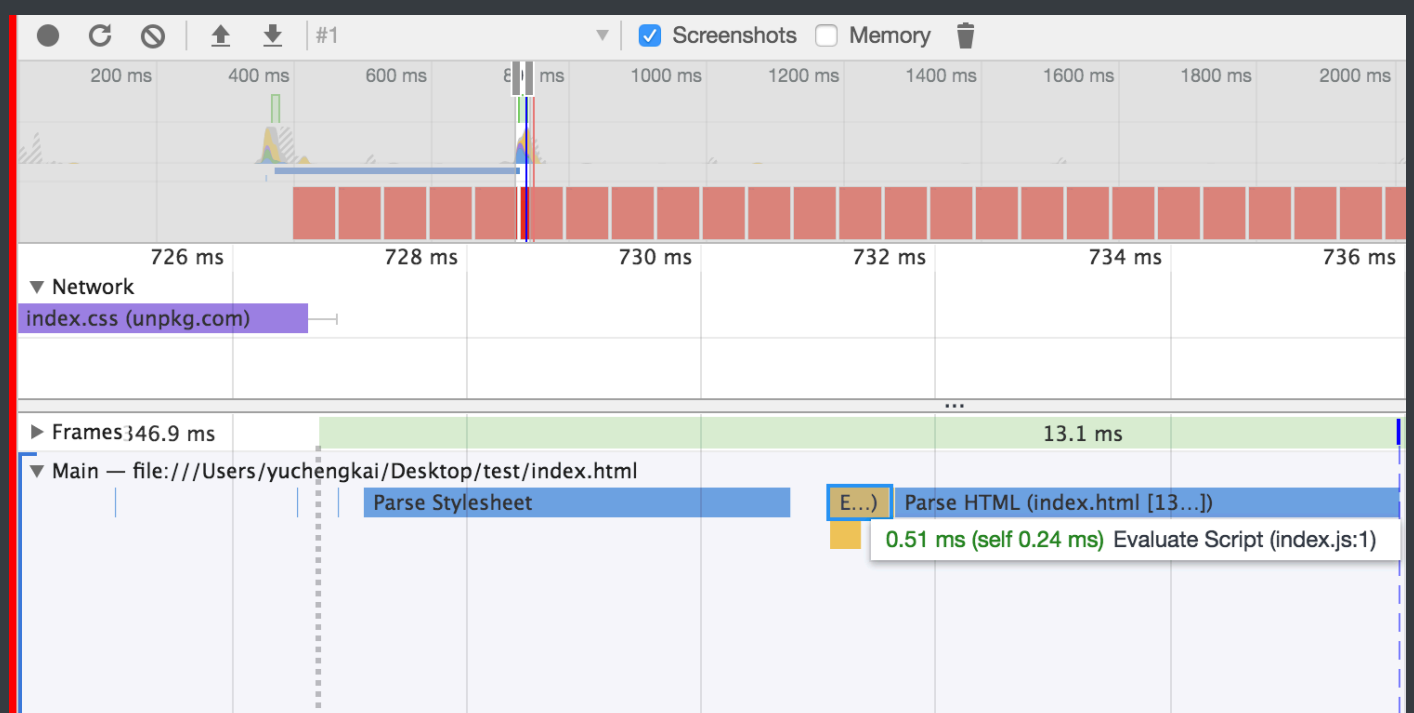
1. 处理 HTML 并构建 DOM 树。
2. 处理 CSS 构建 CSSOM 树。
3. 将 DOM 与 CSSOM 合并成一个渲染树。
4. 根据渲染树来布局，计算每个节点的位置。
5. 调用 GPU 绘制，合成图层，显示在屏幕上。



在构建 CSSOM 树时，会阻塞渲染，直至 CSSOM 树构建完成。并且构建 CSSOM 树是一个十分消耗性能的过程，所以应该尽量保证层级扁平，减少过度层叠，越是具体的 CSS 选择器，执行速度越慢。

当 HTML 解析到 script 标签时，会暂停构建 DOM，完成后才会从暂停的地方重新开始。也就是说，如果你想首屏渲染的越快，就越不应该在首屏就加载 JS 文件。并且 CSS 也会影响 JS 的执行，只有当解析完样式表才会执行 JS，所以也可以认为这种情况下，CSS 也会暂停构建 DOM。

```
3
4
5 <head>
6   <meta charset="UTF-8">
7   <meta name="viewport" content="width=device-width, initial-scale=1.0">
8   <meta http-equiv="X-UA-Compatible" content="ie=edge">
9   <title>Document</title>
10  <link rel="stylesheet" href="https://unpkg.com/element-ui/lib/theme-chalk/index.css">
11 </head>
12
13 <body>
14   <script src="./index.js"></script>
15   <div>123</div>
16   <a>Button1111</a>
17   <a href="">111</a>
18   <!-- <script src="/myvm.js"></script> -->
19 </body>
```



Load 和 DOMContentLoaded 区别

Load 事件触发代表页面中的 DOM, CSS, JS, 图片已经全部加载完毕。

DOMContentLoaded 事件触发代表初始的 HTML 被完全加载和解析, 不需要等待 CSS, JS, 图片加载。

图层

一般来说，可以把普通文档流看成一个图层。特定的属性可以生成一个新的图层。**不同的图层渲染互不影响**，所以对于某些频繁需要渲染的建议单独生成一个新图层，提高性能。**但不能生成过多的图层，会引起反作用。**

通过以下几个常用属性可以生成新图层

- 3D 变换: `translate3d`、`translateZ`
- `will-change`
- `video`、`iframe` 标签
- 通过动画实现的 `opacity` 动画转换
- `position: fixed`

重绘 (Repaint) 和回流 (Reflow)

重绘和回流是渲染步骤中的一小节，但是这两个步骤对于性能影响很大。

- 重绘是当节点需要更改外观而不会影响布局的，比如改变 `color` 就叫称为重绘
- 回流是布局或者几何属性需要改变就称为回流。

回流必定会发生重绘，重绘不一定会引发回流。回流所需的成本比重绘高的多，改变深层次的节点很可能导致父节点的一系列回流。

所以以下几个动作可能会导致性能问题：

- 改变 window 大小
- 改变字体
- 添加或删除样式
- 文字改变
- 定位或者浮动
- 盒模型

很多人不知道的是，重绘和回流其实和 Event loop 有关。

1. 当 Event loop 执行完 Microtasks 后，会判断 document 是否需要更新。因为浏览器是

60Hz 的刷新率，每 16ms 才会更新一次。

2. 然后判断是否有 `resize` 或者 `scroll`，有的话会去触发事件，所以 `resize` 和 `scroll` 事件也是至少 16ms 才会触发一次，并且自带节流功能。
3. 判断是否触发了 `media query`
4. 更新动画并且发送事件
5. 判断是否有全屏操作事件
6. 执行 `requestAnimationFrame` 回调
7. 执行 `IntersectionObserver` 回调，该方法用于判断元素是否可见，可以用于懒加载上，但是兼容性不好
8. 更新界面
9. 以上就是一帧中可能会做的事情。如果在一帧中有空闲时间，就会去执行 `requestIdleCallback` 回调。

以上内容来自于 [HTML 文档](#)

减少重绘和回流

- 使用 `translate` 替代 `top`

```
<div class="test"></div>
<style>
.test {
  position: absolute;
  top: 10px;
  width: 100px;
  height: 100px;
  background: red;
}
</style>
<script>
  setTimeout(() => {
    // 引起回流
    document.querySelector('.test').style.top = '100px'
  }, 1000)
</script>
```

- 使用 `visibility` 替换 `display: none`，因为前者只会引起重绘，后者会引发回流（改变了布局）
- 把 DOM 离线后修改，比如：先把 DOM 给 `display:none` (有一次 Reflow)，然后你修改100次，然后再把它显示出来
- 不要把 DOM 结点的属性值放在一个循环里当成循环里的变量

```
for(let i = 0; i < 1000; i++) {
    // 获取 offsetTop 会导致回流，因为需要去获取正确的值
    console.log(document.querySelector('.test').style.offsetTop)
}
```

- 不要使用 table 布局，可能很小的一个小改动会造成整个 table 的重新布局
- 动画实现的速度的选择，动画速度越快，回流次数越多，也可以选择使用 `requestAnimationFrame`
- CSS 选择符从右往左匹配查找，避免 DOM 深度过深
- 将频繁运行的动画变为图层，图层能够阻止该节点回流影响别的元素。比如对于 `video` 标签，浏览器会自动将该节点变为图层。

The screenshot shows the Chrome DevTools interface with the 'Layers' panel open. The 'video' element is highlighted in the DOM tree on the left. The 'Details' pane on the right shows the following information:

Details	
Size	1180 x 664 (at 0,0)
Compositing Reasons	Composition due to association with a <video> element.
Memory estimate	3.0 MB
Paint count	154
Slow scroll regions	
Sticky position constraint	

性能章节

DNS 预解析

DNS 解析也是需要时间的，可以通过预解析的方式来预先获得域名所对应的 IP。

```
<link rel="dns-prefetch" href="//yuchengkai.cn">
```

缓存

缓存对于前端性能优化来说是个很重要的点，良好的缓存策略可以降低资源的重复加载提高网页的整体加载速度。

通常浏览器缓存策略分为两种：强缓存和协商缓存。

强缓存

实现强缓存可以通过两种响应头实现：Expires 和 Cache-Control。强缓存表示在缓存期间不需要请求，state code 为 200

```
Expires: Wed, 22 Oct 2018 08:41:00 GMT
```

Expires 是 HTTP / 1.0 的产物，表示资源会在 Wed, 22 Oct 2018 08:41:00 GMT 后过期，需要再次请求。并且 Expires 受限于本地时间，如果修改了本地时间，可能会造成缓存失效。

```
Cache-control: max-age=30
```

Cache-Control 出现于 HTTP / 1.1，优先级高于 Expires。该属性表示资源会在 30 秒后过期，需要再次请求。

协商缓存

如果缓存过期了，我们就可以使用协商缓存来解决问题。协商缓存需要请求，如果缓存有效会返回 304。

协商缓存需要客户端和服务端共同实现，和强缓存一样，也有两种实现方式。

Last-Modified 和 If-Modified-Since

Last-Modified 表示本地文件最后修改日期，If-Modified-Since 会将 Last-Modified 的值发送给服务器，询问服务器在该日期后资源是否有更新，有更新的话就会将新的资源发送回来。

但是如果在本地打开缓存文件，就会造成 Last-Modified 被修改，所以在 HTTP / 1.1 出现了 ETag 。

ETag 和 If-None-Match

ETag 类似于文件指纹，If-None-Match 会将当前 ETag 发送给服务器，询问该资源 ETag 是否变动，有变动的话就将新的资源发送回来。并且 ETag 优先级比 Last-Modified 高。

选择合适的缓存策略

对于大部分的场景都可以使用强缓存配合协商缓存解决，但是在一些特殊的地方可能需要选择特殊的缓存策略

- 对于某些不需要缓存的资源，可以使用 Cache-control: no-store ，表示该资源不需要缓存
- 对于频繁变动的资源，可以使用 Cache-Control: no-cache 并配合 ETag 使用，表示该资源已被缓存，但是每次都会发送请求询问资源是否更新。
- 对于代码文件来说，通常使用 Cache-Control: max-age=31536000 并配合策略缓存使用，然后对文件进行指纹处理，一旦文件名变动就会立刻下载新的文件。

使用 HTTP / 2.0

因为浏览器会有并发请求限制，在 HTTP / 1.1 时代，每个请求都需要建立和断开，消耗了好几个 RTT 时间，并且由于 TCP 慢启动的原因，加载体积大的文件会需要更多的时间。

在 HTTP / 2.0 中引入了多路复用，能够让多个请求使用同一个 TCP 链接，极大的加快了网页的加载速度。并且还支持 Header 压缩，进一步的减少了请求的数据大小。

更详细的内容你可以查看 [该小节](#)

预加载

在开发中，可能会遇到这样的情况。有些资源不需要马上用到，但是希望尽早获取，这时候就可以使用预加载。

预加载其实是声明式的 `fetch`，强制浏览器请求资源，并且不会阻塞 `onload` 事件，可以使用以下代码开启预加载

```
<link rel="preload" href="http://example.com">
```

预加载可以一定程度上降低首屏的加载时间，因为可以将一些不影响首屏但重要的文件延后加载，唯一缺点就是兼容性不好。

预渲染

可以通过预渲染将下载的文件预先在后台渲染，可以使用以下代码开启预渲染

```
<link rel="prerender" href="http://example.com">
```

预渲染虽然可以提高页面的加载速度，但是要确保该页面百分百会被用户在之后打开，否则就白白浪费资源去渲染

优化渲染过程

对于代码层面的优化，你可以查阅浏览器系列中的 [相关内容](#)。

懒执行

懒执行就是将某些逻辑延迟到使用时再计算。该技术可以用于首屏优化，对于某些耗时逻辑并不需要在首屏就使用的，就可以使用懒执行。懒执行需要唤醒，一般可以通过定时器或者事件的调用来唤醒。

懒加载

懒加载就是将不关键的资源延后加载。

懒加载的原理就是只加载自定义区域（通常是可视区域，但也可以是即将进入可视区域）内需要加载的东西。对于图片来说，先设置图片标签的 `src` 属性为一张占位图，将真实的图片资源放入一个自定义属性中，当进入自定义区域时，就将自定义属性替换为 `src` 属性，这样图片就会去下载资源，实现了图片懒加载。

懒加载不仅可以用于图片，也可以使用在别的资源上。比如进入可视区域才开始播放视频等等。

文件优化

图片优化

计算图片大小

对于一张 $100 * 100$ 像素的图片来说，图像上有 10000 个像素点，如果每个像素的值是 RGBA 存储的话，那么也就是说每个像素有 4 个通道，每个通道 1 个字节（8 位 = 1 个字节），所以该图片大小大概为 39KB（ $10000 * 1 * 4 / 1024$ ）。

但是在实际项目中，一张图片可能并不需要使用那么多颜色去显示，我们可以通过减少每个像素的调色板来相应缩小图片的大小。

了解了如何计算图片大小的知识，那么对于如何优化图片，想必大家已经有 2 个思路了：

- 减少像素点
- 减少每个像素点能够显示的颜色

图片加载优化

1. 不用图片。很多时候会使用到很多修饰类图片，其实这类修饰图片完全可以用 CSS 去代替。
2. 对于移动端来说，屏幕宽度就那么点，完全没有必要去加载原图浪费带宽。一般图片都用 CDN 加载，可以计算出适配屏幕的宽度，然后去请求相应裁剪好的图片。
3. 小图使用 base64 格式
4. 将多个图标文件整合到一张图片中（雪碧图）
5. 选择正确的图片格式：
 - 对于能够显示 WebP 格式的浏览器尽量使用 WebP 格式。因为 WebP 格式具有更好的图像数据压缩算法，能带来更小的图片体积，而且拥有肉眼识别无差异的图像质量，缺点就是兼容性并不好
 - 小图使用 PNG，其实对于大部分图标这类图片，完全可以使用 SVG 代替
 - 照片使用 JPEG

其他文件优化

- CSS 文件放在 head 中
- 服务端开启文件压缩功能
- 将 script 标签放在 body 底部，因为 JS 文件执行会阻塞渲染。当然也可以把 script 标签放在任意位置然后加上 defer，表示该文件会并行下载，但是会放到 HTML 解析完成后顺序执行。对于没有任何依赖的 JS 文件可以加上 async，表示加载和渲染后续文档元素的过程将和 JS 文件的加载与执行并行无序进行。
- 执行 JS 代码过长会卡住渲染，对于需要很多时间计算的代码可以考虑使用 Webworker。Webworker 可以让我们另开一个线程执行脚本而不影响渲染。

CDN

静态资源尽量使用 CDN 加载，由于浏览器对于单个域名有并发请求上限，可以考虑使用多个 CDN 域名。对于 CDN 加载静态资源需要注意 CDN 域名要与主站不同，否则每次请求都会带上主站的 Cookie。

其他

使用 Webpack 优化项目

- 对于 Webpack4，打包项目使用 production 模式，这样会自动开启代码压缩
- 使用 ES6 模块来开启 tree shaking，这个技术可以移除没有使用的代码
- 优化图片，对于小图可以使用 base64 的方式写入文件中
- 按照路由拆分代码，实现按需加载
- 给打包出来的文件名添加哈希，实现浏览器缓存文件

监控

对于代码运行错误，通常的办法是使用 `window.onerror` 拦截报错。该方法能拦截到大部分的详细报错信息，但是也有例外

- 对于跨域的代码运行错误会显示 `Script error`。对于这种情况我们需要给 `script` 标签添加 `crossorigin` 属性
- 对于某些浏览器可能不会显示调用栈信息，这种情况可以通过 `arguments.callee.caller` 来做栈递归

对于异步代码来说，可以使用 `catch` 的方式捕获错误。比如 `Promise` 可以直接使用 `catch` 函数，`async await` 可以使用 `try catch`

但是要注意线上运行的代码都是压缩过的，需要在打包时生成 `sourceMap` 文件便于 debug。

对于捕获的错误需要上传给服务器，通常可以通过 `img` 标签的 `src` 发起一个请求。

面试题

如何渲染几万条数据并不卡住界面

这道题考察了如何在不卡住页面的情况下渲染数据，也就是说不能一次性将几万条都渲染出来，而应该一次渲染部分 DOM，那么就可以通过 `requestAnimationFrame` 来每 16 ms 刷新一次。

```
<!DOCTYPE html>
```

```
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <meta http-equiv="X-UA-Compatible" content="ie=edge">
  <title>Document</title>
</head>
<body>
  <ul>控件</ul>
  <script>
    setTimeout(() => {
      // 插入十万条数据
      const total = 100000
      // 一次插入 20 条，如果觉得性能不好就减少
      const once = 20
      // 渲染数据总共需要几次
      const loopCount = total / once
      let countOfRender = 0
      let ul = document.querySelector("ul");
      function add() {
        // 优化性能，插入不会造成回流
        const fragment = document.createDocumentFragment();
        for (let i = 0; i < once; i++) {
          const li = document.createElement("li");
          li.innerText = Math.floor(Math.random() * total);
          fragment.appendChild(li);
        }
        ul.appendChild(fragment);
        countOfRender += 1;
        loop();
      }
      function loop() {
        if (countOfRender < loopCount) {
          window.requestAnimationFrame(add);
        }
      }
    });
  </script>
</body>
</html>
```

```
    }  
    loop();  
  }, 0);  
</script>  
</body>  
</html>
```

框架基本原理篇

MVVM

MVVM 由以下三个内容组成

- View：界面
- Model：数据模型
- ViewModel：作为桥梁负责沟通 View 和 Model

在 JQuery 时期，如果需要刷新 UI 时，需要先取到对应的 DOM 再更新 UI，这样数据和业务的逻辑就和页面有强耦合。

在 MVVM 中，UI 是通过数据驱动的，数据一旦改变就会相应的刷新对应的 UI，UI 如果改变，也会改变对应的数据。这种方式就可以在业务处理中只关心数据的流转，而无需直接和页面打交道。ViewModel 只关心数据和业务的处理，不关心 View 如何处理数据，在这种情况下，View 和 Model 都可以独立出来，任何一方改变了也不一定需要改变另一方，并且可以将一些可复用的逻辑放在一个 ViewModel 中，让多个 View 复用这个 ViewModel。

在 MVVM 中，最核心的也就是数据双向绑定，例如 Angular 的脏数据检测，Vue 中的数据劫持。

脏数据检测

当触发了指定事件后会进入脏数据检测，这时会调用 `$digest` 循环遍历所有的数据观察者，判断当前值是否和先前的值有区别，如果检测到变化的话，会调用 `$watch` 函数，然后再次调用 `$digest` 循环直到发现没有变化。循环至少为二次，至多为十次。

脏数据检测虽然存在低效的问题，但是不关心数据是通过什么方式改变的，都可以完成任务，但是这在 Vue 中的双向绑定是存在问题的。并且脏数据检测可以实现批量检测出更新的值，再去统一更新 UI，大大减少了操作 DOM 的次数。所以低效也是相对的，这就仁者见仁智者见智了。

数据劫持

Vue 内部使用了 `Object.defineProperty()` 来实现双向绑定，通过这个函数可以监听到 `set` 和 `get` 的事件。

```
var data = { name: 'yck' }
observe(data)
let name = data.name // -> get value
data.name = 'yyy' // -> change value

function observe(obj) {
  // 判断类型
  if (!obj || typeof obj !== 'object') {
    return
  }
  Object.keys(obj).forEach(key => {
    defineReactive(obj, key, obj[key])
  })
}

function defineReactive(obj, key, val) {
  // 递归子属性
  observe(val)
  Object.defineProperty(obj, key, {
    enumerable: true,
```

```

    configurable: true,
    get: function reactiveGetter() {
        console.log('get value')
        return val
    },
    set: function reactiveSetter(newVal) {
        console.log('change value')
        val = newVal
    }
})
}

```

以上代码简单的实现了如何监听数据的 set 和 get 的事件，但是仅仅如此是不够的，还需要在适当的时候给属性添加发布订阅

```

<div>
    {{name}}
</div>

```

::: v-pre

在解析如上模板代码时，遇到 {{name}} 就会给属性 name 添加发布订阅。

:::

```

// 通过 Dep 解耦
class Dep {
    constructor() {
        this.subs = []
    }
    addSub(sub) {
        // sub 是 Watcher 实例
        this.subs.push(sub)
    }
    notify() {
        this.subs.forEach(sub => {
            sub.update()
        })
    }
}

```

```
    })
  }
}
// 全局属性, 通过该属性配置 Watcher
Dep.target = null

function update(value) {
  document.querySelector('div').innerText = value
}

class Watcher {
  constructor(obj, key, cb) {
    // 将 Dep.target 指向自己
    // 然后触发属性的 getter 添加监听
    // 最后将 Dep.target 置空
    Dep.target = this
    this.cb = cb
    this.obj = obj
    this.key = key
    this.value = obj[key]
    Dep.target = null
  }
  update() {
    // 获得新值
    this.value = this.obj[this.key]
    // 调用 update 方法更新 Dom
    this.cb(this.value)
  }
}

var data = { name: 'yck' }
observe(data)
// 模拟解析到 `{{name}}` 触发的操作
new Watcher(data, 'name', update)
// update Dom innerText
data.name = 'yyy'
```

接下来,对 `defineReactive` 函数进行改造

```
function defineReactive(obj, key, val) {
  // 递归子属性
  observe(val)
  let dp = new Dep()
  Object.defineProperty(obj, key, {
    enumerable: true,
    configurable: true,
    get: function reactiveGetter() {
      console.log('get value')
      // 将 Watcher 添加到订阅
      if (Dep.target) {
        dp.addSub(Dep.target)
      }
      return val
    },
    set: function reactiveSetter(newVal) {
      console.log('change value')
      val = newVal
      // 执行 watcher 的 update 方法
      dp.notify()
    }
  })
}
```

以上实现了一个简易的双向绑定,核心思路就是手动触发一次属性的 `getter` 来实现发布订阅的添加。

Proxy 与 Object.defineProperty 对比

`Object.defineProperty` 虽然已经能够实现双向绑定了,但是他还是有缺陷的。

1. 只能对属性进行数据劫持,所以需要深度遍历整个对象
2. 对于数组不能监听到数据的变化

虽然 Vue 中确实能检测到数组数据的变化，但是其实是使用了 hack 的办法，并且也是有缺陷的。

```
const arrayProto = Array.prototype
export const arrayMethods = Object.create(arrayProto)
// hack 以下几个函数
const methodsToPatch = [
  'push',
  'pop',
  'shift',
  'unshift',
  'splice',
  'sort',
  'reverse'
]
methodsToPatch.forEach(function (method) {
  // 获得原生函数
  const original = arrayProto[method]
  def(arrayMethods, method, function mutator (...args) {
    // 调用原生函数
    const result = original.apply(this, args)
    const ob = this.__ob__
    let inserted
    switch (method) {
      case 'push':
      case 'unshift':
        inserted = args
        break
      case 'splice':
        inserted = args.slice(2)
        break
    }
    if (inserted) ob.observeArray(inserted)
    // 触发更新
    ob.dep.notify()
  })
})
```



```

    return result
  })
})

```

反观 Proxy 就没以上的问题，原生支持监听数组变化，并且可以直接对整个对象进行拦截，所以 Vue 也将在下一个大版本中使用 Proxy 替换 Object.defineProperty

```

let onWatch = (obj, setBind, getLogger) => {
  let handler = {
    get(target, property, receiver) {
      getLogger(target, property)
      return Reflect.get(target, property, receiver);
    },
    set(target, property, value, receiver) {
      setBind(value);
      return Reflect.set(target, property, value);
    }
  };
  return new Proxy(obj, handler);
};

let obj = { a: 1 }
let value
let p = onWatch(obj, (v) => {
  value = v
}, (target, property) => {
  console.log(`Get '${property}' = ${target[property]}`);
})
p.a = 2 // bind `value` to `2`
p.a // -> Get 'a' = 2

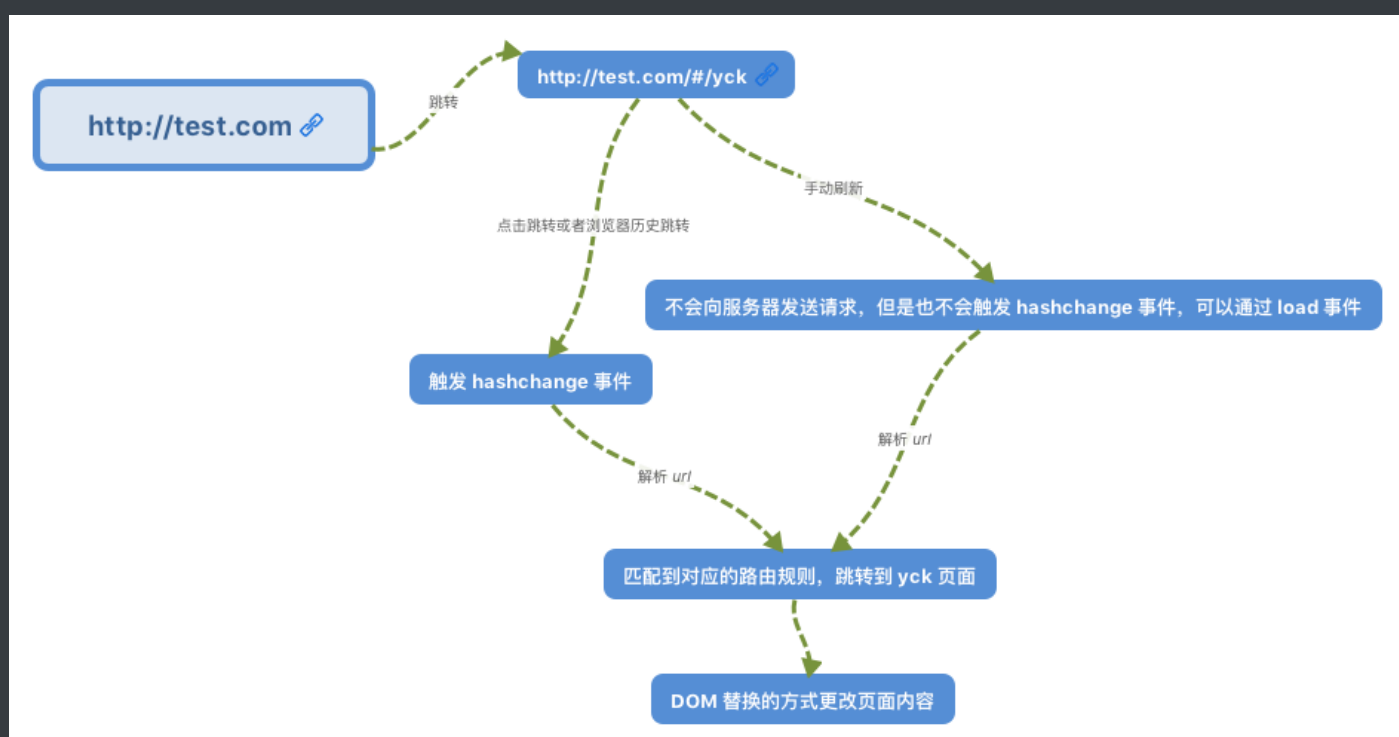
```

路由原理

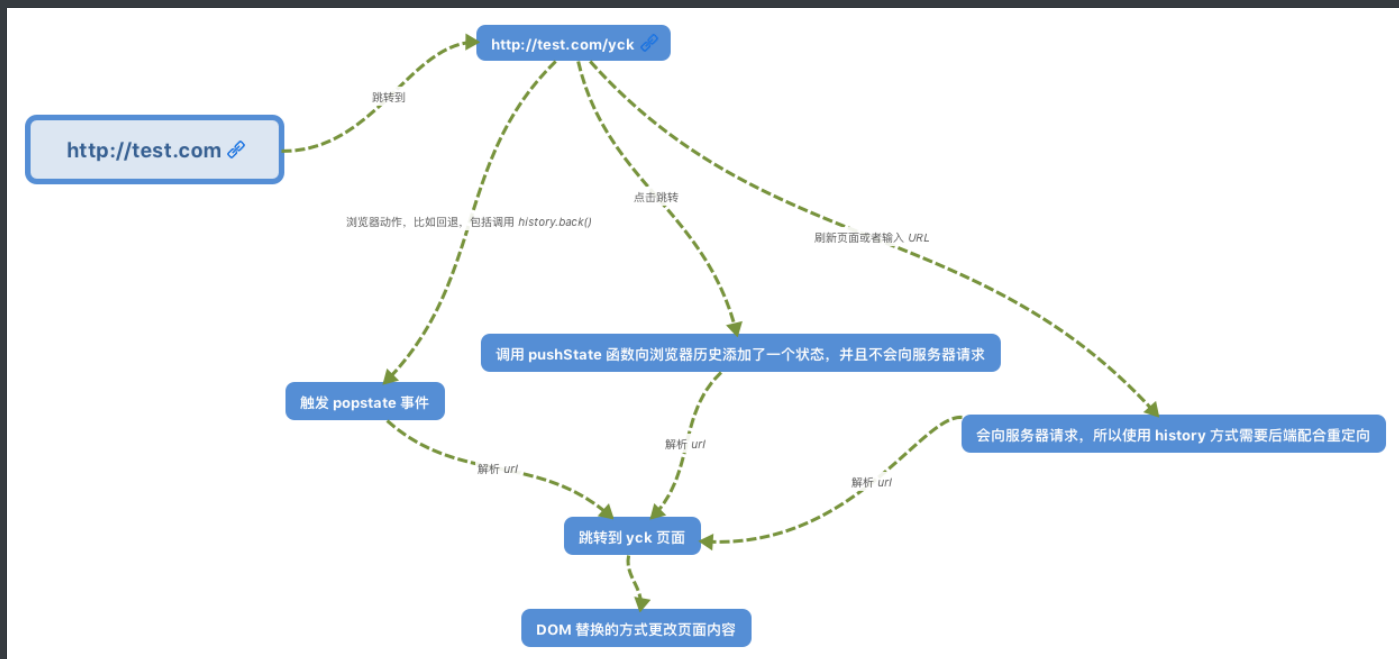
前端路由实现起来其实很简单，本质就是监听 URL 的变化，然后匹配路由规则，显示相应的页面，并且无须刷新。目前单页面使用的路由就只有两种实现方式

- hash 模式
- history 模式

`www.test.com/#/` 就是 Hash URL，当 # 后面的哈希值发生变化时，不会向服务器请求数据，可以通过 `hashchange` 事件来监听到 URL 的变化，从而进行跳转页面。



History 模式是 HTML5 新推出的功能，比之 Hash URL 更加美观



Virtual Dom

代码地址

为什么需要 Virtual Dom

众所周知，操作 DOM 是很耗费性能的一件事情，既然如此，我们可以考虑通过 JS 对象来模拟 DOM 对象，毕竟操作 JS 对象比操作 DOM 省时的多。

举个例子

```
// 假设这里模拟一个 ul，其中包含了 5 个 li  
[1, 2, 3, 4, 5]  
// 这里替换上面的 li  
[1, 2, 5, 4]
```

从上述例子中，我们一眼就可以看出先前的 ul 中的第三个 li 被移除了，四五替换了位置。

如果以上操作对应到 DOM 中，那么就是以下代码

```

// 删除第三个 li
ul.childNodes[2].remove()
// 将第四个 li 和第五个交换位置
let fromNode = ul.childNodes[4]
let toNode = ul.childNodes[3]
let cloneFromNode = fromNode.cloneNode(true)
let cloneToNode = toNode.cloneNode(true)
ul.replaceChild(cloneFromNode, toNode)
ul.replaceChild(cloneToNode, fromNode)

```

当然在实际操作中，我们还需要给每个节点一个标识，作为判断是同一个节点的依据。所以这也是 Vue 和 React 中官方推荐列表里的节点使用唯一的 key 来保证性能。

那么既然 DOM 对象可以通过 JS 对象来模拟，反之也可以通过 JS 对象来渲染出对应的 DOM

以下是一个 JS 对象模拟 DOM 对象的简单实现

```

export default class Element {
  /**
   * @param {String} tag 'div'
   * @param {Object} props { class: 'item' }
   * @param {Array} children [ Element1, 'text' ]
   * @param {String} key option
   */
  constructor(tag, props, children, key) {
    this.tag = tag
    this.props = props
    if (Array.isArray(children)) {
      this.children = children
    } else if (isString(children)) {
      this.key = children
      this.children = null
    }
    if (key) this.key = key
  }
}

```

```
}  
// 渲染  
render() {  
  let root = this._createElement(  
    this.tag,  
    this.props,  
    this.children,  
    this.key  
  )  
  document.body.appendChild(root)  
  return root  
}  
create() {  
  return this._createElement(this.tag, this.props, this.children,  
this.key)  
}  
// 创建节点  
_createElement(tag, props, child, key) {  
  // 通过 tag 创建节点  
  let el = document.createElement(tag)  
  // 设置节点属性  
  for (const key in props) {  
    if (props.hasOwnProperty(key)) {  
      const value = props[key]  
      el.setAttribute(key, value)  
    }  
  }  
  if (key) {  
    el.setAttribute('key', key)  
  }  
  // 递归添加子节点  
  if (child) {  
    child.forEach(element => {  
      let child  
      if (element instanceof Element) {  
        child = this._createElement(  

```

```
        element.tag,  
        element.props,  
        element.children,  
        element.key  
    )  
    } else {  
        child = document.createTextNode(element)  
    }  
    el.appendChild(child)  
  })  
}  
return el  
}  
}
```

Virtual Dom 算法简述

既然我们已经通过 JS 来模拟实现了 DOM，那么接下来的难点就在于如何判断旧的对象和新的对象之间的差异。

DOM 是多叉树的结构，如果需要完整的对比两颗树的差异，那么需要的时间复杂度会是 $O(n^3)$ ，这个复杂度肯定是不能接受的。于是 React 团队优化了算法，实现了 $O(n)$ 的复杂度来对比差异。

实现 $O(n)$ 复杂度的关键就是只对比同层的节点，而不是跨层对比，这也是考虑到在实际业务中很少会去跨层的移动 DOM 元素。

所以判断差异的算法就分为了两步

- 首先从上至下，从左往右遍历对象，也就是树的深度遍历，这一步中会给每个节点添加索引，便于最后渲染差异
- 一旦节点有子元素，就去判断子元素是否有不同

Virtual Dom 算法实现

树的递归

首先我们来实现树的递归算法，在实现该算法前，先来考虑下两个节点对比会有几种情况

1. 新的节点的 tagName 或者 key 和旧的不同，这种情况代表需要替换旧的节点，并且也不再需要遍历新旧节点的子元素了，因为整个旧节点都被删掉了
2. 新的节点的 tagName 和 key（可能都没有）和旧的相同，开始遍历子树
3. 没有新的节点，那么什么都不用做

```
import { StateEnums, isString, move } from './util'
import Element from './element'

export default function diff(oldDomTree, newDomTree) {
  // 用于记录差异
  let pathchs = {}
  // 一开始的索引为 0
  dfs(oldDomTree, newDomTree, 0, pathchs)
  return pathchs
}

function dfs(oldNode, newNode, index, patches) {
  // 用于保存子树的更改
  let curPatches = []
  // 需要判断三种情况
  // 1. 没有新的节点，那么什么都不用做
  // 2. 新的节点的 tagName 和 `key` 和旧的不同，就替换
  // 3. 新的节点的 tagName 和 key（可能都没有） 和旧的相同，开始遍历子树
  if (!newNode) {
  } else if (newNode.tag === oldNode.tag && newNode.key === oldNode.key) {
  }
  // 判断属性是否变更
  let props = diffProps(oldNode.props, newNode.props)
  if (props.length) curPatches.push({ type: StateEnums.ChangeProps,
  props })
}
```

```

    // 遍历子树
    diffChildren(oldNode.children, newNode.children, index, patches)
  } else {
    // 节点不同，需要替换
    curPatches.push({ type: StateEnums.Replace, node: newNode })
  }

  if (curPatches.length) {
    if (patches[index]) {
      patches[index] = patches[index].concat(curPatches)
    } else {
      patches[index] = curPatches
    }
  }
}
}

```

判断属性的更改

判断属性的更改也分三个步骤

1. 遍历旧的属性列表，查看每个属性是否还存在于新的属性列表中
2. 遍历新的属性列表，判断两个列表中都存在的属性的值是否有变化
3. 在第二步中同时查看是否有属性不存在与旧的属性列表中

```

function diffProps(oldProps, newProps) {
  // 判断 Props 分以下三步骤
  // 先遍历 oldProps 查看是否存在删除的属性
  // 然后遍历 newProps 查看是否有属性值被修改
  // 最后查看是否有属性新增
  let change = []
  for (const key in oldProps) {
    if (oldProps.hasOwnProperty(key) && !newProps[key]) {
      change.push({
        prop: key
      })
    }
  }
}

```



```

    }
  }
  for (const key in newProps) {
    if (newProps.hasOwnProperty(key)) {
      const prop = newProps[key]
      if (oldProps[key] && oldProps[key] !== newProps[key]) {
        change.push({
          prop: key,
          value: newProps[key]
        })
      } else if (!oldProps[key]) {
        change.push({
          prop: key,
          value: newProps[key]
        })
      }
    }
  }
  return change
}

```

判断列表差异算法实现

这个算法是整个 Virtual Dom 中最核心的算法，且让我一一为你道来。

这里的主要步骤其实和判断属性差异是类似的，也是分为三步

1. 遍历旧的节点列表，查看每个节点是否还存在于新的节点列表中
2. 遍历新的节点列表，判断是否有新的节点
3. 在第二步中同时判断节点是否有移动

PS: 该算法只对有 key 的节点做处理

```

function listDiff(oldList, newList, index, patches) {
  // 为了遍历方便，先取出两个 list 的所有 keys
  let oldKeys = getKeys(oldList)

```

```
let newKeys = getKeys(newList)
let changes = []

// 用于保存变更后的节点数据
// 使用该数组保存有以下好处
// 1.可以正确获得被删除节点索引
// 2.交换节点位置只需要操作一遍 DOM
// 3.用于 `diffChildren` 函数中的判断，只需要遍历
// 两个树中都存在的节点，而对于新增或者删除的节点来说，完全没必要
// 再去判断一遍
let list = []
oldList &&
  oldList.forEach(item => {
    let key = item.key
    if (isString(item)) {
      key = item
    }
    // 寻找新的 children 中是否含有当前节点
    // 没有的话需要删除
    let index = newKeys.indexOf(key)
    if (index === -1) {
      list.push(null)
    } else list.push(key)
  })
// 遍历变更后的数组
let length = list.length
// 因为删除数组元素是会更改索引的
// 所有从后往前删可以保证索引不变
for (let i = length - 1; i >= 0; i--) {
  // 判断当前元素是否为空，为空表示需要删除
  if (!list[i]) {
    list.splice(i, 1)
    changes.push({
      type: StateEnums.Remove,
      index: i
    })
  })
}
```

```

    }
  }
  // 遍历新的 list, 判断是否有节点新增或移动
  // 同时也对 `list` 做节点新增和移动节点的操作
  newList &&
  newList.forEach((item, i) => {
    let key = item.key
    if (isString(item)) {
      key = item
    }
    // 寻找旧的 children 中是否含有当前节点
    let index = list.indexOf(key)
    // 没找到代表新节点, 需要插入
    if (index === -1 || key == null) {
      changes.push({
        type: StateEnums.Insert,
        node: item,
        index: i
      })
      list.splice(i, 0, key)
    } else {
      // 找到了, 需要判断是否需要移动
      if (index !== i) {
        changes.push({
          type: StateEnums.Move,
          from: index,
          to: i
        })
        move(list, index, i)
      }
    }
  })
  return { changes, list }
}

function getKeys(list) {

```

```

let keys = []
let text
list &&
  list.forEach(item => {
    let key
    if (isString(item)) {
      key = [item]
    } else if (item instanceof Element) {
      key = item.key
    }
    keys.push(key)
  })
return keys
}

```

遍历子元素打标识

对于这个函数来说，主要功能就两个

1. 判断两个列表差异
2. 给节点打上标记

总体来说，该函数实现的功能很简单

```

function diffChildren(oldChild, newChild, index, patches) {
  let { changes, list } = listDiff(oldChild, newChild, index, patches)
  if (changes.length) {
    if (patches[index]) {
      patches[index] = patches[index].concat(changes)
    } else {
      patches[index] = changes
    }
  }
}
// 记录上一个遍历过的节点
let last = null
oldChild &&

```

```

oldChild.forEach((item, i) => {
  let child = item && item.children
  if (child) {
    index =
      last && last.children ? index + last.children.length + 1 :
index + 1
    let keyIndex = list.indexOf(item.key)
    let node = newChild[keyIndex]
    // 只遍历新旧中都存在的节点，其他新增或者删除的没必要遍历
    if (node) {
      dfs(item, node, index, patches)
    }
  } else index += 1
  last = item
})
}

```

渲染差异

通过之前的算法，我们已经可以得出两个树的差异了。既然知道了差异，就需要局部去更新 DOM 了，下面就让我们来看看 Virtual Dom 算法的最后一步骤

这个函数主要两个功能

1. 深度遍历树，将需要做变更操作的取出来
2. 局部更新 DOM

整体来说这部分代码还是很好理解的

```

let index = 0
export default function patch(node, patches) {
  let changes = patches[index]
  let childNodes = node && node.childNodes
  // 这里的深度遍历和 diff 中是一样的
  if (!childNodes) index += 1
  if (changes && changes.length && patches[index]) {

```

```

    changeDom(node, changes)
  }
  let last = null
  if (childNodes && childNodes.length) {
    childNodes.forEach((item, i) => {
      index =
        last && last.children ? index + last.children.length + 1 : index
+ 1
      patch(item, patches)
      last = item
    })
  }
}

```

```

function changeDom(node, changes, noChild) {
  changes &&
  changes.forEach(change => {
    let { type } = change
    switch (type) {
      case StateEnums.ChangeProps:
        let { props } = change
        props.forEach(item => {
          if (item.value) {
            node.setAttribute(item.prop, item.value)
          } else {
            node.removeAttribute(item.prop)
          }
        })
        break
      case StateEnums.Remove:
        node.childNodes[change.index].remove()
        break
      case StateEnums.Insert:
        let dom
        if (isString(change.node)) {
          dom = document.createTextNode(change.node)

```

```

    } else if (change.node instanceof Element) {
      dom = change.node.create()
    }
    node.insertBefore(dom, node.childNodes[change.index])
    break
  case StateEnums.Replace:
    node.parentNode.replaceChild(change.node.create(), node)
    break
  case StateEnums.Move:
    let fromNode = node.childNodes[change.from]
    let toNode = node.childNodes[change.to]
    let cloneFromNode = fromNode.cloneNode(true)
    let cloneToNode = toNode.cloneNode(true)
    node.replaceChild(cloneFromNode, toNode)
    node.replaceChild(cloneToNode, fromNode)
    break
  default:
    break
}
})
}

```

最后

Virtual Dom 算法的实现也就是以下三步

1. 通过 JS 来模拟创建 DOM 对象
2. 判断两个对象的差异
3. 渲染差异

```

let test4 = new Element('div', { class: 'my-div' }, ['test4'])
let test5 = new Element('ul', { class: 'my-div' }, ['test5'])

let test1 = new Element('div', { class: 'my-div' }, [test4])

```

```
let test2 = new Element('div', { id: '11' }, [test5, test4])

let root = test1.render()

let pathchs = diff(test1, test2)
console.log(pathchs)

setTimeout(() => {
  console.log('开始更新')
  patch(root, pathchs)
  console.log('结束更新')
}, 1000)
```

当然目前的实现还略显粗糙，但是对于理解 Virtual Dom 算法来说已经是完全足够的了。

Vue 章节

NextTick 原理分析

nextTick 可以让我们在下次 DOM 更新循环结束之后执行延迟回调，用于获得更新后的 DOM。

在 Vue 2.4 之前都是使用的 microtasks，但是 microtasks 的优先级过高，在某些情况下可能会出现比事件冒泡更快的情况，但如果都使用 macrotasks 又可能会出现渲染的性能问题。所以在新版本中，会默认使用 microtasks，但在特殊情况下会使用 macrotasks，比如 v-on。

对于实现 macrotasks，会先判断是否能使用 setImmediate，不能的话降级为 MessageChannel，以上都不行的话就使用 setTimeout

```
if (typeof setImmediate !== 'undefined' && isNative(setImmediate)) {
  macroTimerFunc = () => {
    setImmediate(flushCallbacks)
  }
}
```



```

    }
  } else if (
    typeof MessageChannel !== 'undefined' &&
    (isNative(MessageChannel) ||
      // PhantomJS
      MessageChannel.toString() === '[object MessageChannelConstructor]')
  ) {
    const channel = new MessageChannel()
    const port = channel.port2
    channel.port1.onmessage = flushCallbacks
    macroTimerFunc = () => {
      port.postMessage(1)
    }
  } else {
    /* istanbul ignore next */
    macroTimerFunc = () => {
      setTimeout(flushCallbacks, 0)
    }
  }
}

```

nextTick 同时也支持 Promise 的使用，会判断是否实现了 Promise

```

export function nextTick(cb?: Function, ctx?: Object) {
  let _resolve
  // 将回调函数整合进一个数组中
  callbacks.push(() => {
    if (cb) {
      try {
        cb.call(ctx)
      } catch (e) {
        handleError(e, ctx, 'nextTick')
      }
    } else if (_resolve) {
      _resolve(ctx)
    }
  })
}

```

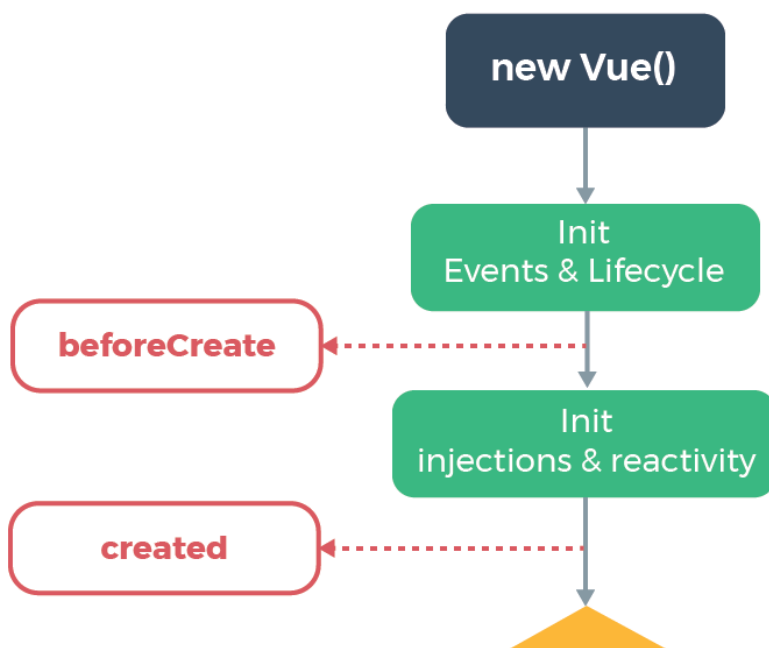
```

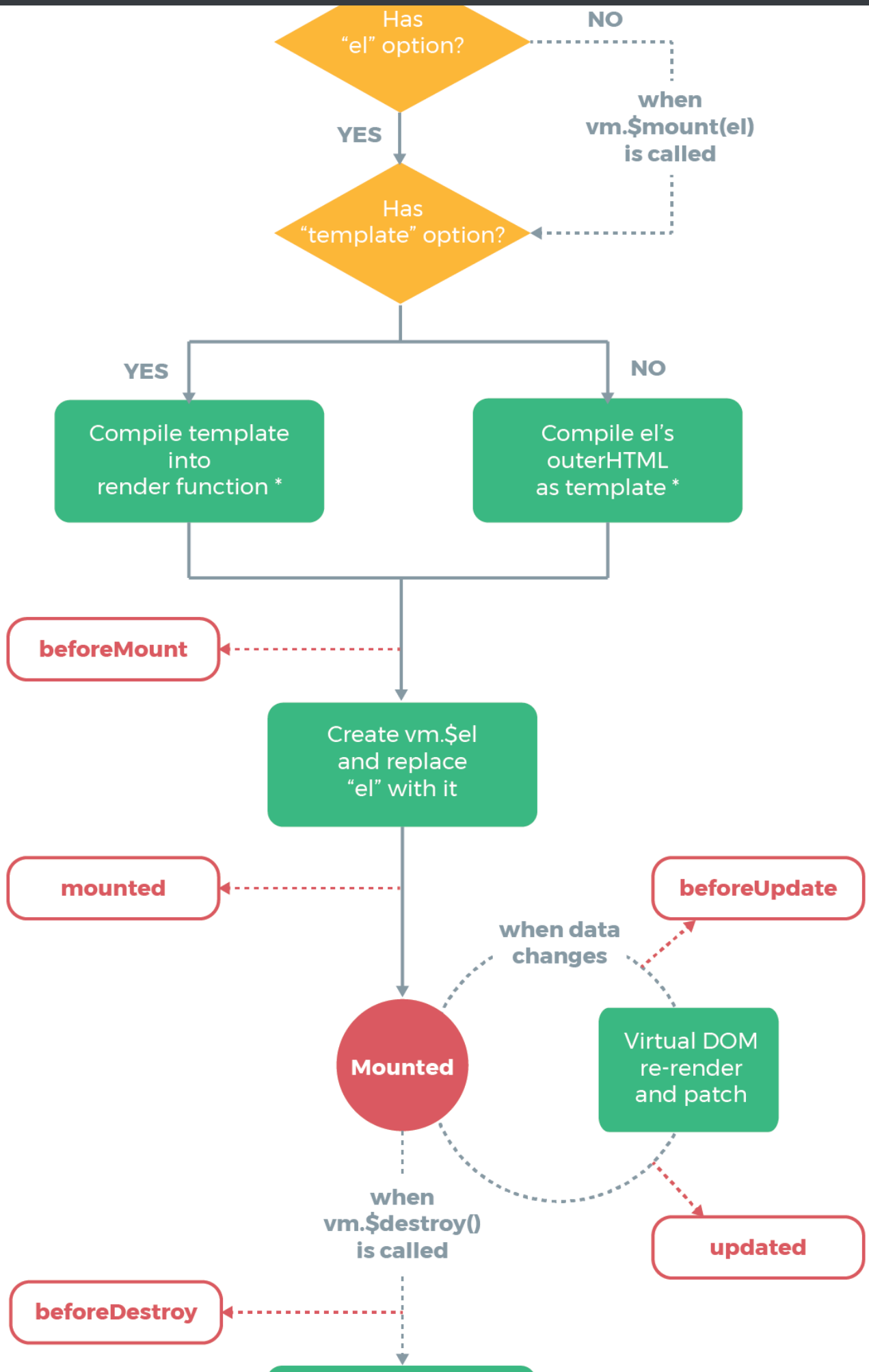
  })
  if (!pending) {
    pending = true
    if (useMacroTask) {
      macroTimerFunc()
    } else {
      microTimerFunc()
    }
  }
}
// 判断是否可以使用 Promise
// 可以的话给 _resolve 赋值
// 这样回调函数就能以 promise 的方式调用
if (!cb && typeof Promise !== 'undefined') {
  return new Promise(resolve => {
    _resolve = resolve
  })
}
}
}

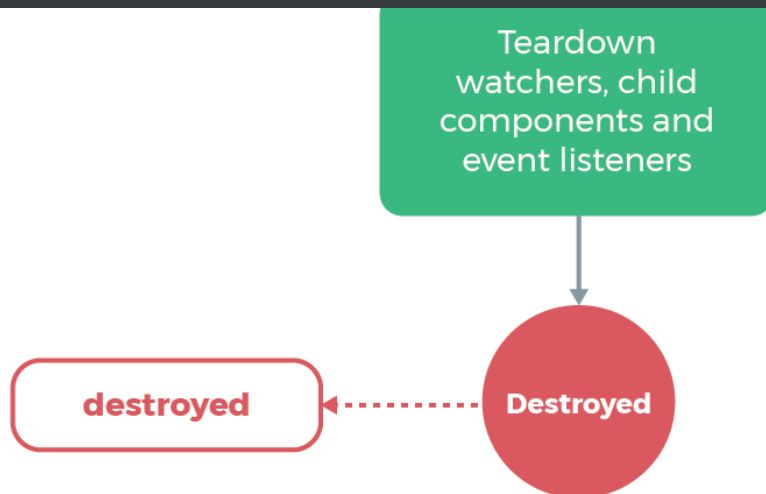
```

生命周期分析

生命周期函数就是组件在初始化或者数据更新时会触发的钩子函数。







* template compilation is performed ahead-of-time if using a build step, e.g. single-file components

在初始化时，会调用以下代码，生命周期就是通过 `callHook` 调用的

```
Vue.prototype._init = function(options) {  
  initLifecycle(vm)  
  initEvents(vm)  
  initRender(vm)  
  callHook(vm, 'beforeCreate') // 拿不到 props data  
  initInjections(vm)  
  initState(vm)  
  initProvide(vm)  
  callHook(vm, 'created')  
}
```

可以发现在以上代码中，`beforeCreate` 调用的时候，是获取不到 `props` 或者 `data` 中的数据，因为这些数据的初始化都在 `initState` 中。

接下来会执行挂载函数

```

export function mountComponent {
  callHook(vm, 'beforeMount')
  // ...
  if (vm.$vnode == null) {
    vm._isMounted = true
    callHook(vm, 'mounted')
  }
}

```

beforeMount 就是在挂载前执行的，然后开始创建 VDOM 并替换成真实 DOM，最后执行 mounted 钩子。这里会有个判断逻辑，如果是外部 new Vue({}) 的话，不会存在 \$vnode，所以直接执行 mounted 钩子了。如果有子组件的话，会递归挂载子组件，只有当所有子组件全部挂载完毕，才会执行根组件的挂载钩子。

接下来是数据更新时会调用的钩子函数

```

function flushSchedulerQueue() {
  // ...
  for (index = 0; index < queue.length; index++) {
    watcher = queue[index]
    if (watcher.before) {
      watcher.before() // 调用 beforeUpdate
    }
    id = watcher.id
    has[id] = null
    watcher.run()
    // in dev build, check and stop circular updates.
    if (process.env.NODE_ENV !== 'production' && has[id] !== null) {
      circular[id] = (circular[id] || 0) + 1
      if (circular[id] > MAX_UPDATE_COUNT) {
        warn(
          'You may have an infinite update loop ' +
            (watcher.user
              ? `in watcher with expression "${watcher.expression}"`
              : `in a component render function.`),

```

```

        watcher.vm
      )
      break
    }
  }
}
callUpdatedHooks(updatedQueue)
}

function callUpdatedHooks(queue) {
  let i = queue.length
  while (i--) {
    const watcher = queue[i]
    const vm = watcher.vm
    if (vm._watcher === watcher && vm._isMounted) {
      callHook(vm, 'updated')
    }
  }
}

```

上图还有两个生命周期没有说，分别为 `activated` 和 `deactivated`，这两个钩子函数是 `keep-alive` 组件独有的。用 `keep-alive` 包裹的组件在切换时不会进行销毁，而是缓存到内存中并执行 `deactivated` 钩子函数，命中缓存渲染后会执行 `activated` 钩子函数。

最后就是销毁组件的钩子函数了

```

Vue.prototype.$destroy = function() {
  // ...
  callHook(vm, 'beforeDestroy')
  vm._isBeingDestroyed = true
  // remove self from parent
  const parent = vm.$parent
  if (parent && !parent._isBeingDestroyed && !vm.$options.abstract) {
    remove(parent.$children, vm)
  }
}

```

```

// teardown watchers
if (vm._watcher) {
  vm._watcher.teardown()
}
let i = vm._watchers.length
while (i--) {
  vm._watchers[i].teardown()
}
// remove reference from data ob
// frozen object may not have observer.
if (vm._data.__ob__) {
  vm._data.__ob__.vmCount--
}
// call the last hook...
vm._isDestroyed = true
// invoke destroy hooks on current rendered tree
vm.__patch__(vm._vnode, null)
// fire destroyed hook
callHook(vm, 'destroyed')
// turn off all instance listeners.
vm.$off()
// remove __vue__ reference
if (vm.$el) {
  vm.$el.__vue__ = null
}
// release circular reference (#6759)
if (vm.$vnode) {
  vm.$vnode.parent = null
}
}

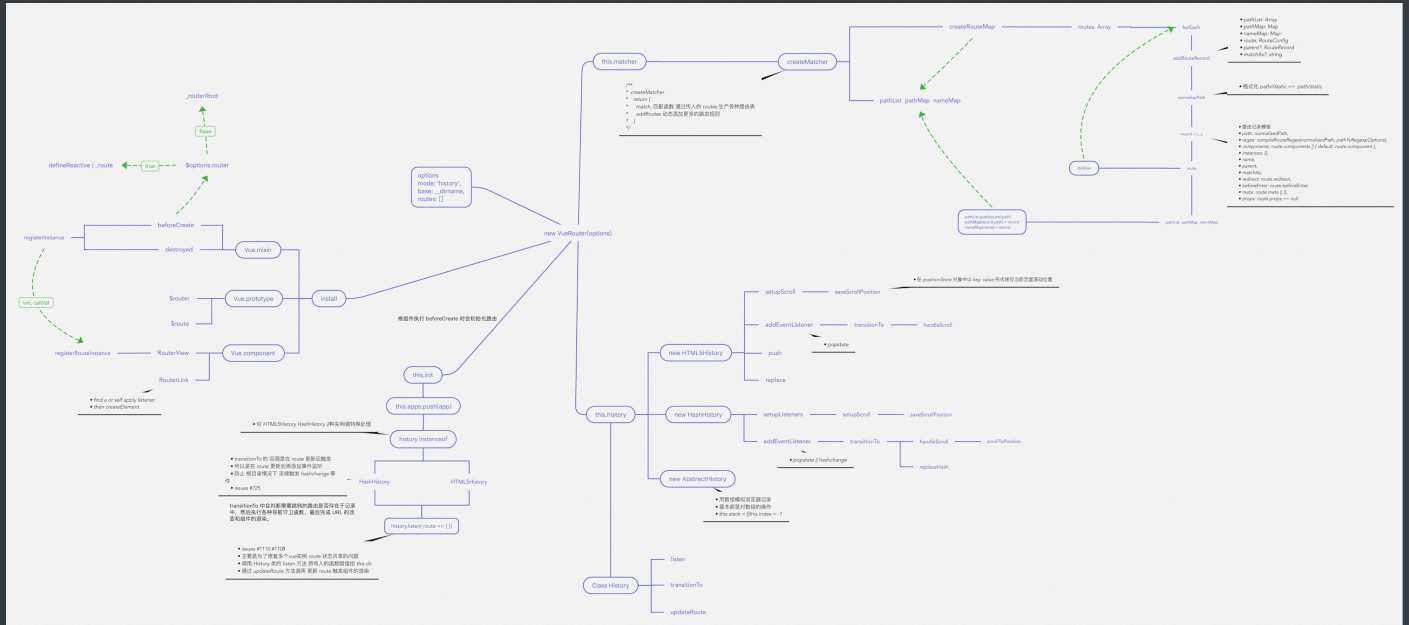
```

在执行销毁操作前会调用 `beforeDestroy` 钩子函数，然后进行一系列的销毁操作，如果有子组件的话，也会递归销毁子组件，所有子组件都销毁完毕后会执行根组件的 `destroyed` 钩子函数。

VueRouter 源码解析

重要函数思维导图

以下思维导图罗列了源码中重要的一些函数



路由注册

在开始之前，推荐大家 clone 一份源码对照着看。因为篇幅较长，函数间的跳转也很多。

使用路由之前，需要调用 `Vue.use(VueRouter)`，这是因为让插件可以使用 `Vue`

```
export function initUse (Vue: GlobalAPI) {
  Vue.use = function (plugin: Function | Object) {
    // 判断重复安装插件
    const installedPlugins = (this._installedPlugins || (this._installedPlugins = []))
    if (installedPlugins.indexOf(plugin) > -1) {
      return this
    }
    const args = toArray(arguments, 1)
    // 插入 Vue
    args.unshift(this)
    // 一般插件都会有一个 install 函数
  }
}
```



```

    // 通过该函数让插件可以使用 Vue
    if (typeof plugin.install === 'function') {
      plugin.install.apply(plugin, args)
    } else if (typeof plugin === 'function') {
      plugin.apply(null, args)
    }
    installedPlugins.push(plugin)
    return this
  }
}

```

接下来看下 `install` 函数的部分实现

```

export function install (Vue) {
  // 确保 install 调用一次
  if (install.installed && _Vue === Vue) return
  install.installed = true
  // 把 Vue 赋值给全局变量
  _Vue = Vue
  const registerInstance = (vm, callVal) => {
    let i = vm.$options._parentVnode
    if (isDef(i) && isDef(i = i.data) && isDef(i =
i.registerRouteInstance)) {
      i(vm, callVal)
    }
  }
  // 给每个组件的钩子函数混入实现
  // 可以发现在 `beforeCreate` 钩子执行时
  // 会初始化路由
  Vue.mixin({
    beforeCreate () {
      // 判断组件是否存在 router 对象，该对象只在根组件上有
      if (isDef(this.$options.router)) {
        // 根路由设置为自己
        this._routerRoot = this
      }
    }
  })
}

```

```

    this._router = this.$options.router
    // 初始化路由
    this._router.init(this)
    // 很重要, 为 _route 属性实现双向绑定
    // 触发组件渲染
    Vue.util.defineReactive(this, '_route',
this._router.history.current)
  } else {
    // 用于 router-view 层级判断
    this._routerRoot = (this.$parent && this.$parent._routerRoot) ||
this
  }
  registerInstance(this, this)
},
destroyed () {
  registerInstance(this)
}
})
// 全局注册组件 router-link 和 router-view
Vue.component('RouterView', View)
Vue.component('RouterLink', Link)
}

```

对于路由注册来说，核心就是调用 `Vue.use(VueRouter)`，使得 `VueRouter` 可以使用 `Vue`。然后通过 `Vue` 来调用 `VueRouter` 的 `install` 函数。在该函数中，核心就是给组件混入钩子函数和全局注册两个路由组件。

VueRouter 实例化

在安装插件后，对 `VueRouter` 进行实例化。

```

const Home = { template: '<div>home</div>' }
const Foo = { template: '<div>foo</div>' }
const Bar = { template: '<div>bar</div>' }

```

```
// 3. Create the router
const router = new VueRouter({
  mode: 'hash',
  base: __dirname,
  routes: [
    { path: '/', component: Home }, // all paths are defined without the
    hash.
    { path: '/foo', component: Foo },
    { path: '/bar', component: Bar }
  ]
})
```

来看一下 VueRouter 的构造函数

```
constructor(options: RouterOptions = {}) {
  // ...
  // 路由匹配对象
  this.matcher = createMatcher(options.routes || [], this)

  // 根据 mode 采取不同的路由方式
  let mode = options.mode || 'hash'
  this.fallback =
    mode === 'history' && !supportsPushState && options.fallback !==
false
  if (this.fallback) {
    mode = 'hash'
  }
  if (!inBrowser) {
    mode = 'abstract'
  }
  this.mode = mode

  switch (mode) {
    case 'history':
      this.history = new HTML5History(this, options.base)
```

```

        break
      case 'hash':
        this.history = new HashHistory(this, options.base,
this.fallback)
        break
      case 'abstract':
        this.history = new AbstractHistory(this, options.base)
        break
      default:
        if (process.env.NODE_ENV !== 'production') {
          assert(false, `invalid mode: ${mode}`)
        }
    }
  }
}

```

在实例化 VueRouter 的过程中，核心是创建一个路由匹配对象，并且根据 mode 来采取不同的路由方式。

创建路由匹配对象

```

export function createMatcher (
  routes: Array<RouteConfig>,
  router: VueRouter
): Matcher {
  // 创建路由映射表
  const { pathList, pathMap, nameMap } = createRouteMap(routes)

  function addRoutes (routes) {
    createRouteMap(routes, pathList, pathMap, nameMap)
  }

  // 路由匹配
  function match (
    raw: RawLocation,
    currentRoute?: Route,
    redirectedFrom?: Location

```

```

): Route {
  //...
}

return {
  match,
  addRoutes
}
}

```

`createMatcher` 函数的作用就是创建路由映射表，然后通过闭包的方式让 `addRoutes` 和 `match` 函数能够使用路由映射表的几个对象，最后返回一个 `Matcher` 对象。

接下来看 `createMatcher` 函数时如何创建映射表的

```

export function createRouteMap (
  routes: Array<RouteConfig>,
  oldPathList?: Array<string>,
  oldPathMap?: Dictionary<RouteRecord>,
  oldNameMap?: Dictionary<RouteRecord>
): {
  pathList: Array<string>;
  pathMap: Dictionary<RouteRecord>;
  nameMap: Dictionary<RouteRecord>;
} {
  // 创建映射表
  const pathList: Array<string> = oldPathList || []
  const pathMap: Dictionary<RouteRecord> = oldPathMap ||
Object.create(null)
  const nameMap: Dictionary<RouteRecord> = oldNameMap ||
Object.create(null)
  // 遍历路由配置，为每个配置添加路由记录
  routes.forEach(route => {
    addRouteRecord(pathList, pathMap, nameMap, route)
  })
}

```

```

// 确保通配符在最后
for (let i = 0, l = pathList.length; i < l; i++) {
  if (pathList[i] === '*') {
    pathList.push(pathList.splice(i, 1)[0])
    l--
    i--
  }
}
return {
  pathList,
  pathMap,
  nameMap
}
}

// 添加路由记录
function addRouteRecord (
  pathList: Array<string>,
  pathMap: Dictionary<RouteRecord>,
  nameMap: Dictionary<RouteRecord>,
  route: RouteConfig,
  parent?: RouteRecord,
  matchAs?: string
) {
  // 获得路由配置下的属性
  const { path, name } = route
  const pathToRegexpOptions: PathToRegexpOptions =
route.pathToRegexpOptions || {}
  // 格式化 url, 替换 /
  const normalizedPath = normalizePath(
    path,
    parent,
    pathToRegexpOptions.strict
  )
  // 生成记录对象
  const record: RouteRecord = {
    path: normalizedPath,

```

```

    regex: compileRouteRegex(normalizedPath, pathToRegexOptions),
    components: route.components || { default: route.component },
    instances: {},
    name,
    parent,
    matchAs,
    redirect: route.redirect,
    beforeEnter: route.beforeEnter,
    meta: route.meta || {},
    props: route.props == null
      ? {}
      : route.components
        ? route.props
        : { default: route.props }
  }

  if (route.children) {
    // 递归路由配置的 children 属性, 添加路由记录
    route.children.forEach(child => {
      const childMatchAs = matchAs
        ? cleanPath(`${matchAs}/${child.path}`)
        : undefined
      addRouteRecord(pathList, pathMap, nameMap, child, record,
childMatchAs)
    })
  }
  // 如果路由有别名的话
  // 给别名也添加路由记录
  if (route.alias !== undefined) {
    const aliases = Array.isArray(route.alias)
      ? route.alias
      : [route.alias]

    aliases.forEach(alias => {
      const aliasRoute = {
        path: alias,

```

```

        children: route.children
    }
    addRouteRecord(
        pathList,
        pathMap,
        nameMap,
        aliasRoute,
        parent,
        record.path || '/' // matchAs
    )
  })
}
// 更新映射表
if (!pathMap[record.path]) {
  pathList.push(record.path)
  pathMap[record.path] = record
}
// 命名路由添加记录
if (name) {
  if (!nameMap[name]) {
    nameMap[name] = record
  } else if (process.env.NODE_ENV !== 'production' && !matchAs) {
    warn(
      false,
      `Duplicate named routes definition: ` +
      `{ name: "${name}", path: "${record.path}" }`
    )
  }
}
}
}

```

以上就是创建路由匹配对象的全过程，通过用户配置的路由规则来创建对应的路由映射表。

路由初始化

当根组件调用 `beforeCreate` 钩子函数时，会执行以下代码

```
beforeCreate () {  
  // 只有根组件有 router 属性，所以根组件初始化时会初始化路由  
  if (isDef(this.$options.router)) {  
    this._routerRoot = this  
    this._router = this.$options.router  
    this._router.init(this)  
    Vue.util.defineReactive(this, '_route',  
this._router.history.current)  
  } else {  
    this._routerRoot = (this.$parent && this.$parent._routerRoot) ||  
this  
  }  
  registerInstance(this, this)  
}
```

接下来看下路由初始化会做些什么

```
init(app: any /* Vue component instance */) {  
  // 保存组件实例  
  this.apps.push(app)  
  // 如果根组件已经有了就返回  
  if (this.app) {  
    return  
  }  
  this.app = app  
  // 赋值路由模式  
  const history = this.history  
  // 判断路由模式，以哈希模式为例  
  if (history instanceof HTML5History) {  
    history.transitionTo(history.getCurrentLocation())  
  } else if (history instanceof HashHistory) {
```

```

// 添加 hashchange 监听
const setupHashListener = () => {
  history.setupListeners()
}
// 路由跳转
history.transitionTo(
  history.getCurrentLocation(),
  setupHashListener,
  setupHashListener
)
}
// 该回调会在 transitionTo 中调用
// 对组件的 _route 属性进行赋值，触发组件渲染
history.listen(route => {
  this.apps.forEach(app => {
    app._route = route
  })
})
}

```

在路由初始化时，核心就是进行路由的跳转，改变 URL 然后渲染对应的组件。接下来来看一下路由是如何进行跳转的。

路由跳转

```

transitionTo (location: RawLocation, onComplete?: Function, onAbort?:
Function) {
  // 获取匹配的路由信息
  const route = this.router.match(location, this.current)
  // 确认切换路由
  this.confirmTransition(route, () => {
    // 以下为切换路由成功或失败的回调
    // 更新路由信息，对组件的 _route 属性进行赋值，触发组件渲染
    // 调用 afterHooks 中的钩子函数
    this.updateRoute(route)
  })
}

```

```

    // 添加 hashchange 监听
    onComplete && onComplete(route)
    // 更新 URL
    this.ensureURL()
    // 只执行一次 ready 回调
    if (!this.ready) {
      this.ready = true
      this.readyCbs.forEach(cb => { cb(route) })
    }
  }, err => {
    // 错误处理
    if (onAbort) {
      onAbort(err)
    }
    if (err && !this.ready) {
      this.ready = true
      this.readyErrorCbs.forEach(cb => { cb(err) })
    }
  })
}

```

在路由跳转中，需要先获取匹配的路由信息，所以先来看下如何获取匹配的路由信息

```

function match (
  raw: RawLocation,
  currentRoute?: Route,
  redirectedFrom?: Location
): Route {
  // 序列化 url
  // 比如对于该 url 来说 /abc?foo=bar&baz=qux#hello
  // 会序列化路径为 /abc
  // 哈希为 #hello
  // 参数为 foo: 'bar', baz: 'qux'
  const location = normalizeLocation(raw, currentRoute, false, router)
  const { name } = location

```

```
// 如果是命名路由，就判断记录中是否有该命名路由配置
if (name) {
  const record = nameMap[name]
  // 没找到表示没有匹配的路由
  if (!record) return _createRoute(null, location)
  const paramNames = record.regex.keys
    .filter(key => !key.optional)
    .map(key => key.name)
  // 参数处理
  if (typeof location.params !== 'object') {
    location.params = {}
  }
  if (currentRoute && typeof currentRoute.params === 'object') {
    for (const key in currentRoute.params) {
      if (!(key in location.params) && paramNames.indexOf(key) > -1) {
        location.params[key] = currentRoute.params[key]
      }
    }
  }
  if (record) {
    location.path = fillParams(record.path, location.params, `named
route "${name}"`)
    return _createRoute(record, location, redirectedFrom)
  }
} else if (location.path) {
  // 非命名路由处理
  location.params = {}
  for (let i = 0; i < pathList.length; i++) {
    // 查找记录
    const path = pathList[i]
    const record = pathMap[path]
    // 如果匹配路由，则创建路由
    if (matchRoute(record.regex, location.path, location.params)) {
      return _createRoute(record, location, redirectedFrom)
    }
  }
}
```

```
    }  
    // 没有匹配的路由  
    return _createRoute(null, location)  
  }  
}
```

接下来看看如何创建路由

```
// 根据条件创建不同的路由  
function _createRoute(  
  record: ?RouteRecord,  
  location: Location,  
  redirectedFrom?: Location  
) : Route {  
  if (record && record.redirect) {  
    return redirect(record, redirectedFrom || location)  
  }  
  if (record && record.matchAs) {  
    return alias(record, location, record.matchAs)  
  }  
  return createRoute(record, location, redirectedFrom, router)  
}  
  
export function createRoute (  
  record: ?RouteRecord,  
  location: Location,  
  redirectedFrom?: ?Location,  
  router?: VueRouter  
) : Route {  
  const stringifyQuery = router && router.options.stringifyQuery  
  // 克隆参数  
  let query: any = location.query || {}  
  try {  
    query = clone(query)  
  } catch (e) {}  
  // 创建路由对象
```

```

const route: Route = {
  name: location.name || (record && record.name),
  meta: (record && record.meta) || {},
  path: location.path || '/',
  hash: location.hash || '',
  query,
  params: location.params || {},
  fullPath: getFullPath(location, stringifyQuery),
  matched: record ? formatMatch(record) : []
}
if (redirectedFrom) {
  route.redirectedFrom = getFullPath(redirectedFrom, stringifyQuery)
}
// 让路由对象不可修改
return Object.freeze(route)
}
// 获得包含当前路由的所有嵌套路径片段的路由记录
// 包含从根路由到当前路由的匹配记录，从上至下
function formatMatch(record: ?RouteRecord): Array<RouteRecord> {
  const res = []
  while (record) {
    res.unshift(record)
    record = record.parent
  }
  return res
}

```

至此匹配路由已经完成，我们回到 `transitionTo` 函数中，接下来执行 `confirmTransition`

```

transitionTo (location: RawLocation, onComplete?: Function, onAbort?:
Function) {
  // 确认切换路由
  this.confirmTransition(route, () => {}
}

```

```

confirmTransition(route: Route, onComplete: Function, onAbort?:
Function) {
    const current = this.current
    // 中断跳转路由函数
    const abort = err => {
        if (isError(err)) {
            if (this.errorCbs.length) {
                this.errorCbs.forEach(cb => {
                    cb(err)
                })
            } else {
                warn(false, 'uncaught error during route navigation:')
                console.error(err)
            }
        }
        onAbort && onAbort(err)
    }
    // 如果是相同的路由就不跳转
    if (
        isSameRoute(route, current) &&
        route.matched.length === current.matched.length
    ) {
        this.ensureURL()
        return abort()
    }
    // 通过对比路由解析出可复用的组件，需要渲染的组件，失活的组件
    const { updated, deactivated, activated } = resolveQueue(
        this.current.matched,
        route.matched
    )

    function resolveQueue(
        current: Array<RouteRecord>,
        next: Array<RouteRecord>
    ): {
        updated: Array<RouteRecord>,

```

```

    activated: Array<RouteRecord>,
    deactivated: Array<RouteRecord>
  } {
    let i
    const max = Math.max(current.length, next.length)
    for (i = 0; i < max; i++) {
      // 当前路由路径和跳转路由路径不同时跳出遍历
      if (current[i] !== next[i]) {
        break
      }
    }
    return {
      // 可复用的组件对应路由
      updated: next.slice(0, i),
      // 需要渲染的组件对应路由
      activated: next.slice(i),
      // 失活的组件对应路由
      deactivated: current.slice(i)
    }
  }
// 导航守卫数组
const queue: Array<?NavigationGuard> = [].concat(
  // 失活的组件钩子
  extractLeaveGuards(deactivated),
  // 全局 beforeEach 钩子
  this.router.beforeHooks,
  // 在当前路由改变，但是该组件被复用时调用
  extractUpdateHooks(updated),
  // 需要渲染组件 enter 守卫钩子
  activated.map(m => m.beforeEnter),
  // 解析异步路由组件
  resolveAsyncComponents(activated)
)
// 保存路由
this.pending = route
// 迭代器，用于执行 queue 中的导航守卫钩子

```



```
const iterator = (hook: NavigationGuard, next) => {
  // 路由不相等就不跳转路由
  if (this.pending !== route) {
    return abort()
  }
  try {
    // 执行钩子
    hook(route, current, (to: any) => {
      // 只有执行了钩子函数中的 next, 才会继续执行下一个钩子函数
      // 否则会暂停跳转
      // 以下逻辑是在判断 next() 中的传参
      if (to === false || isError(to)) {
        // next(false)
        this.ensureURL(true)
        abort(to)
      } else if (
        typeof to === 'string' ||
        (typeof to === 'object' &&
          (typeof to.path === 'string' || typeof to.name ===
'string')))
      ) {
        // next('/') 或者 next({ path: '/' }) -> 重定向
        abort()
        if (typeof to === 'object' && to.replace) {
          this.replace(to)
        } else {
          this.push(to)
        }
      } else {
        // 这里执行 next
        // 也就是执行下面函数 runQueue 中的 step(index + 1)
        next(to)
      }
    })
  } catch (e) {
    abort(e)
  }
}
```

```

    }
  }
  // 经典的同步执行异步函数
  runQueue(queue, iterator, () => {
    const postEnterCbs = []
    const isValid = () => this.current === route
    // 当所有异步组件加载完成后，会执行这里的回调，也就是 runQueue 中的 cb()
    // 接下来执行 需要渲染组件的导航守卫钩子
    const enterGuards = extractEnterGuards(activated, postEnterCbs,
    isValid)
    const queue = enterGuards.concat(this.router.resolveHooks)
    runQueue(queue, iterator, () => {
      // 跳转完成
      if (this.pending !== route) {
        return abort()
      }
      this.pending = null
      onComplete(route)
      if (this.router.app) {
        this.router.app.$nextTick(() => {
          postEnterCbs.forEach(cb => {
            cb()
          })
        })
      }
    })
  })
}

export function runQueue (queue: Array<?NavigationGuard>, fn: Function,
cb: Function) {
  const step = index => {
    // 队列中的函数都执行完毕，就执行回调函数
    if (index >= queue.length) {
      cb()
    } else {
      if (queue[index]) {

```

```

    // 执行迭代器，用户在钩子函数中执行 next() 回调
    // 回调中判断传参，没有问题就执行 next()，也就是 fn 函数中的第二个参数
    fn(queue[index], () => {
      step(index + 1)
    })
  } else {
    step(index + 1)
  }
}
}
// 取出队列中第一个钩子函数
step(0)
}

```

接下来介绍导航守卫

```

const queue: Array<?NavigationGuard> = [].concat(
  // 失活的组件钩子
  extractLeaveGuards(deactivated),
  // 全局 beforeEach 钩子
  this.router.beforeHooks,
  // 在当前路由改变，但是该组件被复用时调用
  extractUpdateHooks(updated),
  // 需要渲染组件 enter 守卫钩子
  activated.map(m => m.beforeEnter),
  // 解析异步路由组件
  resolveAsyncComponents(activated)
)

```

第一步是先执行失活组件的钩子函数

```

function extractLeaveGuards(deactivated: Array<RouteRecord>): Array<?
Function> {
  // 传入需要执行的钩子函数名
  return extractGuards(deactivated, 'beforeRouteLeave', bindGuard, true)
}

```

```

}
function extractGuards(
  records: Array<RouteRecord>,
  name: string,
  bind: Function,
  reverse?: boolean
): Array<?Function> {
  const guards = flatMapComponents(records, (def, instance, match, key)
=> {
    // 找出组件中对应的钩子函数
    const guard = extractGuard(def, name)
    if (guard) {
      // 给每个钩子函数添加上下文对象为组件自身
      return Array.isArray(guard)
        ? guard.map(guard => bind(guard, instance, match, key))
        : bind(guard, instance, match, key)
    }
  })
  // 数组降维，并且判断是否需要翻转数组
  // 因为某些钩子函数需要从子执行到父
  return flatten(reverse ? guards.reverse() : guards)
}
export function flatMapComponents (
  matched: Array<RouteRecord>,
  fn: Function
): Array<?Function> {
  // 数组降维
  return flatten(matched.map(m => {
    // 将组件中的对象传入回调函数中，获得钩子函数数组
    return Object.keys(m.components).map(key => fn(
      m.components[key],
      m.instances[key],
      m, key
    ))
  }))
}

```

第二步执行全局 beforeEach 钩子函数

```
beforeEach(fn: Function): Function {
  return registerHook(this.beforeHooks, fn)
}
function registerHook(list: Array<any>, fn: Function): Function {
  list.push(fn)
  return () => {
    const i = list.indexOf(fn)
    if (i > -1) list.splice(i, 1)
  }
}
```

在 VueRouter 类中有以上代码，每当给 VueRouter 实例添加 beforeEach 函数时就会将函数 push 进 beforeHooks 中。

第三步执行 beforeRouteUpdate 钩子函数，调用方式和第一步相同，只是传入的函数名不同，在该函数中可以访问到 this 对象。

第四步执行 beforeEnter 钩子函数，该函数是路由独享的钩子函数。

第五步是解析异步组件。

```
export function resolveAsyncComponents (matched: Array<RouteRecord>):
Function {
  return (to, from, next) => {
    let hasAsync = false
    let pending = 0
    let error = null
    // 该函数作用之前已经介绍过了
    flatMapComponents(matched, (def, _, match, key) => {
      // 判断是否是异步组件
      if (typeof def === 'function' && def.cid === undefined) {
        hasAsync = true
```

```

    pending++
    // 成功回调
    // once 函数确保异步组件只加载一次
    const resolve = once(resolvedDef => {
      if (isESModule(resolvedDef)) {
        resolvedDef = resolvedDef.default
      }
      // 判断是否是构造函数
      // 不是的话通过 Vue 来生成组件构造函数
      def.resolved = typeof resolvedDef === 'function'
        ? resolvedDef
        : _Vue.extend(resolvedDef)
      // 赋值组件
      // 如果组件全部解析完毕，继续下一步
      match.components[key] = resolvedDef
      pending--
      if (pending <= 0) {
        next()
      }
    })
    // 失败回调
    const reject = once(reason => {
      const msg = `Failed to resolve async component ${key}:
    ${reason}`
      process.env.NODE_ENV !== 'production' && warn(false, msg)
      if (!error) {
        error = isError(reason)
          ? reason
          : new Error(msg)
        next(error)
      }
    })
    let res
    try {
      // 执行异步组件函数
      res = def(resolve, reject)
    }
  }
}

```

```

    } catch (e) {
      reject(e)
    }
    if (res) {
      // 下载完成执行回调
      if (typeof res.then === 'function') {
        res.then(resolve, reject)
      } else {
        const comp = res.component
        if (comp && typeof comp.then === 'function') {
          comp.then(resolve, reject)
        }
      }
    }
  }
}
})
// 不是异步组件直接下一步
if (!hasAsync) next()
}
}

```

以上就是第一个 `runQueue` 中的逻辑，第五步完成后会执行第一个 `runQueue` 中回调函数

```

// 该回调用于保存 `beforeRouteEnter` 钩子中的回调函数
const postEnterCbs = []
const isValid = () => this.current === route
// beforeRouteEnter 导航守卫钩子
const enterGuards = extractEnterGuards(activated, postEnterCbs, isValid)
// beforeResolve 导航守卫钩子
const queue = enterGuards.concat(this.router.resolveHooks)
runQueue(queue, iterator, () => {
  if (this.pending !== route) {
    return abort()
  }
  this.pending = null
}

```

```
// 这里会执行 afterEach 导航守卫钩子
onComplete(route)
if (this.router.app) {
  this.router.app.$nextTick(() => {
    postEnterCbs.forEach(cb => {
      cb()
    })
  })
}
})
```

第六步是执行 `beforeRouteEnter` 导航守卫钩子，`beforeRouteEnter` 钩子不能访问 `this` 对象，因为钩子在导航确认前被调用，需要渲染的组件还没被创建。但是该钩子函数是唯一一个支持在回调中获取 `this` 对象的函数，回调会在路由确认执行。

```
beforeRouteEnter (to, from, next) {
  next(vm => {
    // 通过 `vm` 访问组件实例
  })
}
```

下面来看看是如何支持在回调中拿到 `this` 对象的

```
function extractEnterGuards(
  activated: Array<RouteRecord>,
  cbs: Array<Function>,
  isValid: () => boolean
): Array<?Function> {
  // 这里和之前调用导航守卫基本一致
  return extractGuards(
    activated,
    'beforeRouteEnter',
    (guard, _, match, key) => {
      return bindEnterGuard(guard, match, key, cbs, isValid)
    }
  )
}
```



```

    )
  }
  function bindEnterGuard(
    guard: NavigationGuard,
    match: RouteRecord,
    key: string,
    cbs: Array<Function>,
    isValid: () => boolean
  ): NavigationGuard {
    return function routeEnterGuard(to, from, next) {
      return guard(to, from, cb => {
        // 判断 cb 是否是函数
        // 是的话就 push 进 postEnterCbs
        next(cb)
        if (typeof cb === 'function') {
          cbs.push(() => {
            // 循环直到拿到组件实例
            poll(cb, match.instances, key, isValid)
          })
        }
      })
    }
  }
}

// 该函数是为了解决 issus #750
// 当 router-view 外面包裹了 mode 为 out-in 的 transition 组件
// 会在组件初次导航到时获得不到组件实例对象
function poll(
  cb: any, // somehow flow cannot infer this is a function
  instances: Object,
  key: string,
  isValid: () => boolean
) {
  if (
    instances[key] &&
    !instances[key]._isBeingDestroyed // do not reuse being destroyed
    instance

```

```

    ) {
      cb(instances[key])
    } else if (isValid()) {
      // setTimeout 16ms 作用和 nextTick 基本相同
      setTimeout(() => {
        poll(cb, instances, key, isValid)
      }, 16)
    }
  }
}

```

第七步是执行 `beforeResolve` 导航守卫钩子，如果注册了全局 `beforeResolve` 钩子就会在这里执行。

第八步就是导航确认，调用 `afterEach` 导航守卫钩子了。

以上都执行完成后，会触发组件的渲染

```

history.listen(route => {
  this.apps.forEach(app => {
    app._route = route
  })
})

```

以上回调会在 `updateRoute` 中调用

```

updateRoute(route: Route) {
  const prev = this.current
  this.current = route
  this.cb && this.cb(route)
  this.router.afterHooks.forEach(hook => {
    hook && hook(route, prev)
  })
}

```

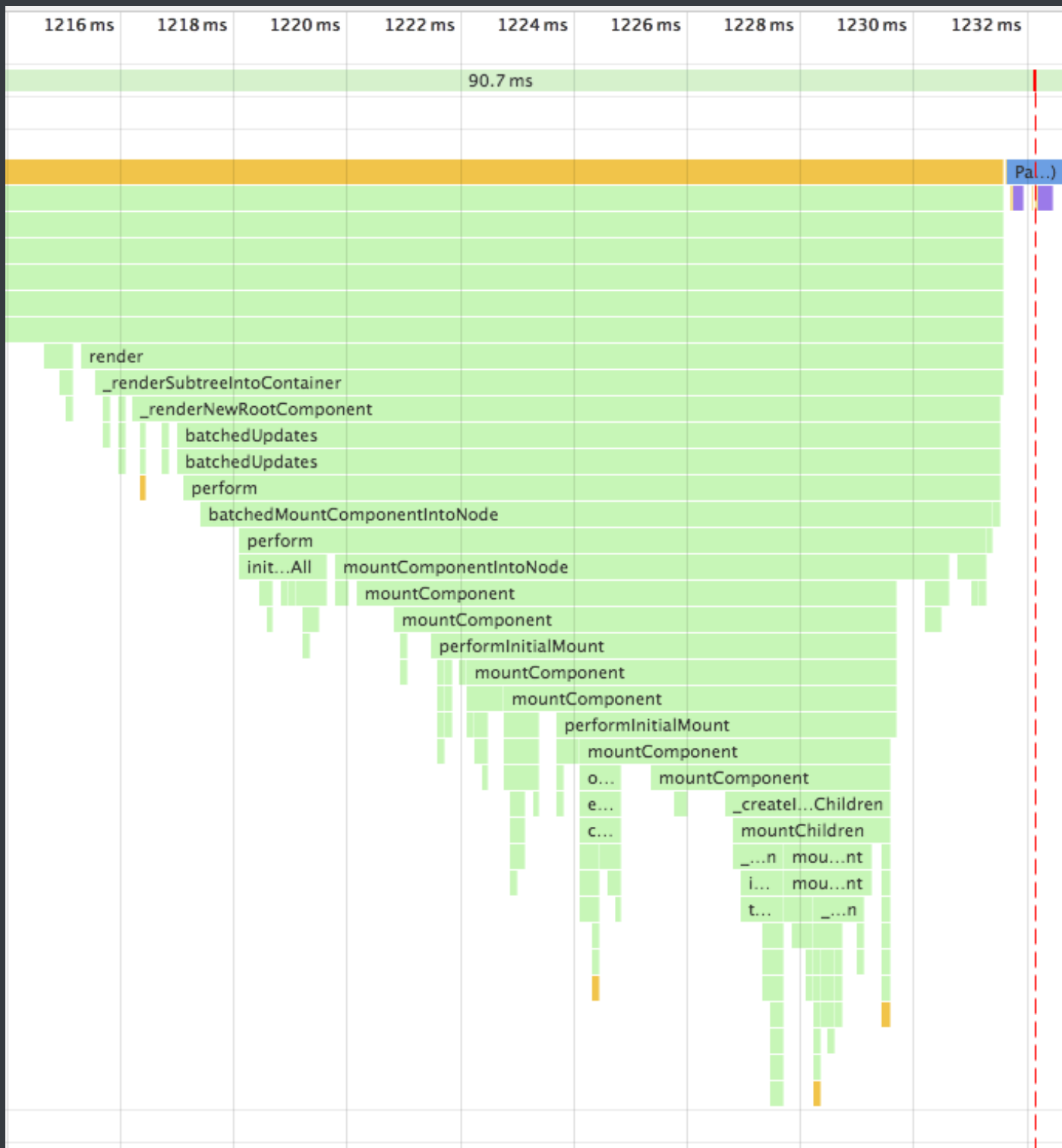
至此，路由跳转已经全部分析完毕。核心就是判断需要跳转的路由是否存在于记录中，然后执行各种导航守卫函数，最后完成 URL 的改变和组件的渲染。

React 章节

React 生命周期分析

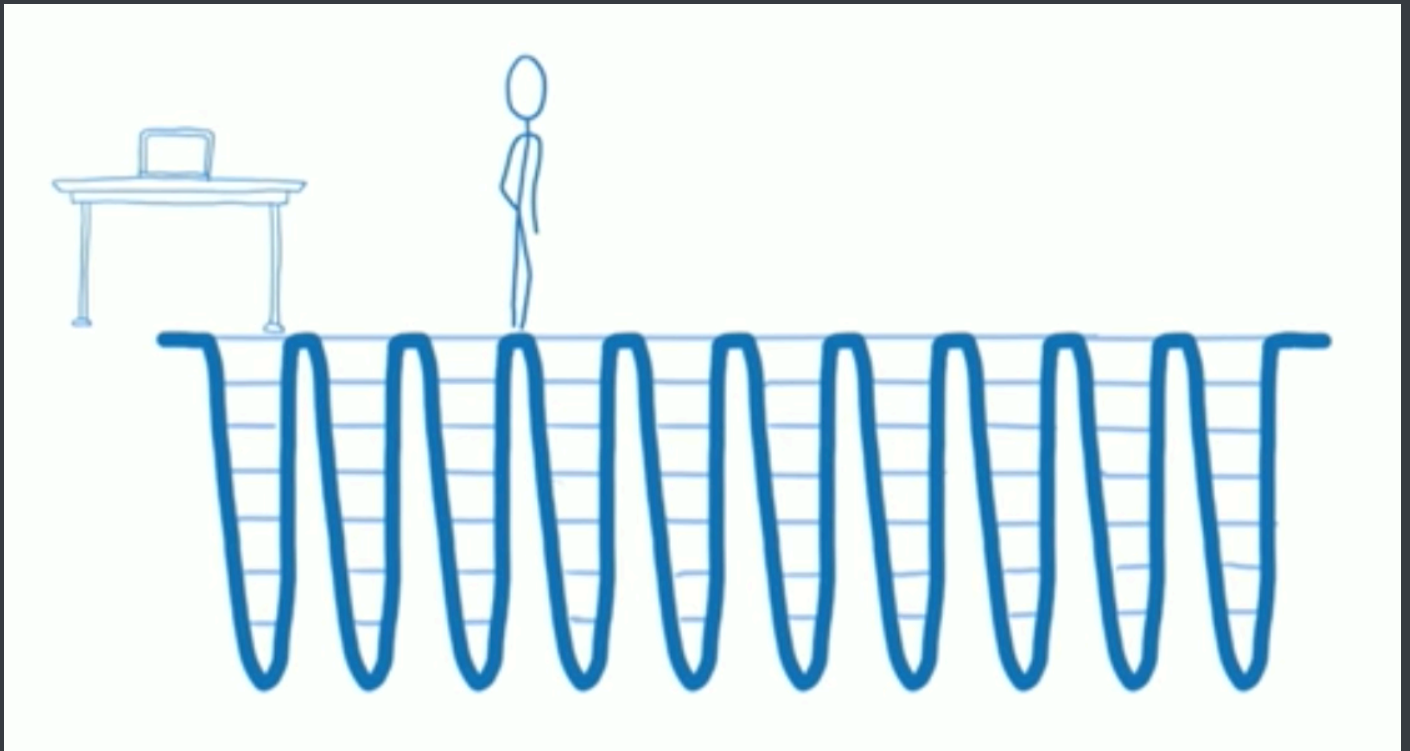
在 V16 版本中引入了 Fiber 机制。这个机制一定程度上的影响了部分生命周期的调用，并且也引入了新的 2 个 API 来解决问题。

在之前的版本中，如果你拥有一个很复杂的复合组件，然后改动了最上层组件的 `state`，那么调用栈可能会很长



调用栈过长，再加上中间进行了复杂的操作，就可能导致长时间阻塞主线程，带来不好的用户体验。Fiber 就是为了解决该问题而生。

Fiber 本质上是一个虚拟的堆栈帧，新的调度器会按照优先级自由调度这些帧，从而将之前的同步渲染改成了异步渲染，在不影响体验的情况下去分段计算更新。



对于如何区别优先级，React 有自己的一套逻辑。对于动画这种实时性很高的东西，也就是 16 ms 必须渲染一次保证不卡顿的情况下，React 会每 16 ms（以内）暂停一下更新，返回来继续渲染动画。

对于异步渲染，现在渲染有两个阶段：reconciliation 和 commit。前者过程是可以打断的，后者不能暂停，会一直更新界面直到完成。

Reconciliation 阶段

- `componentWillMount`
- `componentWillReceiveProps`
- `shouldComponentUpdate`
- `componentWillUpdate`

Commit 阶段

- `componentDidMount`
- `componentDidUpdate`
- `componentWillUnmount`

因为 reconciliation 阶段是可以被打断的，所以 reconciliation 阶段会执行的生命周期函数就可能会出现调用多次的情况，从而引起 Bug。所以对于 reconciliation 阶段调用的几个函数，除了 shouldComponentUpdate 以外，其他都应该避免去使用，并且 V16 中也引入了新的 API 来解决这个问题。

getDerivedStateFromProps 用于替换 componentWillMountReceiveProps，该函数会在初始化和 update 时被调用

```
class ExampleComponent extends React.Component {
  // Initialize state in constructor,
  // Or with a property initializer.
  state = {};

  static getDerivedStateFromProps(nextProps, prevState) {
    if (prevState.someMirroredValue !== nextProps.someValue) {
      return {
        derivedData: computeDerivedState(nextProps),
        someMirroredValue: nextProps.someValue
      };
    }

    // Return null to indicate no change to state.
    return null;
  }
}
```

getSnapshotBeforeUpdate 用于替换 componentWillUpdate，该函数会在 update 后 DOM 更新前被调用，用于读取最新的 DOM 数据。

V16 生命周期函数用法建议

```
class ExampleComponent extends React.Component {
  // 用于初始化 state
  constructor() {}
}
```

```
// 用于替换 `componentWillReceiveProps`，该函数会在初始化和 `update` 时被调用
// 因为该函数是静态函数，所以取不到 `this`
// 如果需要对比 `prevProps` 需要单独在 `state` 中维护
static getDerivedStateFromProps(nextProps, prevState) {}
// 判断是否需要更新组件，多用于组件性能优化
shouldComponentUpdate(nextProps, nextState) {}
// 组件挂载后调用
// 可以在该函数中进行请求或者订阅
componentDidMount() {}
// 用于获得最新的 DOM 数据
getSnapshotBeforeUpdate() {}
// 组件即将销毁
// 可以在此处移除订阅，定时器等
componentWillUnmount() {}
// 组件销毁后调用
componentDidUnmount() {}
// 组件更新后调用
componentDidUpdate() {}
// 渲染组件函数
render() {}
// 以下函数不建议使用
UNSAFE_componentWillMount() {}
UNSAFE_componentWillUpdate(nextProps, nextState) {}
UNSAFE_componentWillReceiveProps(nextProps) {}
}
```

setState

setState 在 React 中是经常使用的一个 API，但是它存在一些问题，可能会导致犯错，核心原因就是因为这个 API 是异步的。

首先 setState 的调用并不会马上引起 state 的改变，并且如果你一次调用了多个 setState，那么结果可能并不如你期待的一样。

```

handle() {
  // 初始化 `count` 为 0
  console.log(this.state.count) // -> 0
  this.setState({ count: this.state.count + 1 })
  this.setState({ count: this.state.count + 1 })
  this.setState({ count: this.state.count + 1 })
  console.log(this.state.count) // -> 0
}

```

第一，两次的打印都为 0，因为 `setState` 是个异步 API，只有同步代码运行完毕才会执行。`setState` 异步的原因我认为在于，`setState` 可能会导致 DOM 的重绘，如果调用一次就马上去进行重绘，那么调用多次就会造成不必要的性能损失。设计成异步的话，就可以将多次调用放入一个队列中，在恰当的时候统一进行更新过程。

第二，虽然调用了三次 `setState`，但是 `count` 的值还是为 1。因为多次调用会合并为一次，只有当更新结束后 `state` 才会改变，三次调用等同于如下代码

```

Object.assign(
  {},
  { count: this.state.count + 1 },
  { count: this.state.count + 1 },
  { count: this.state.count + 1 },
)

```

当然你也可以通过以下方式来实现调用三次 `setState` 使得 `count` 为 3

```

handle() {
  this.setState((prevState) => ({ count: prevState.count + 1 }))
  this.setState((prevState) => ({ count: prevState.count + 1 }))
  this.setState((prevState) => ({ count: prevState.count + 1 }))
}

```

如果你想在每次调用 `setState` 后获得正确的 `state`，可以通过如下代码实现


```
handle() {
  this.setState((prevState) => ({ count: prevState.count + 1 }), () => {
    console.log(this.state)
  })
}
```

Redux 源码分析

首先让我们来看下 combineReducers 函数

```
// 传入一个 object
export default function combineReducers(reducers) {
  // 获取该 Object 的 key 值
  const reducerKeys = Object.keys(reducers)
  // 过滤后的 reducers
  const finalReducers = {}
  // 获取每一个 key 对应的 value
  // 在开发环境下判断值是否为 undefined
  // 然后将值类型是函数的值放入 finalReducers
  for (let i = 0; i < reducerKeys.length; i++) {
    const key = reducerKeys[i]

    if (process.env.NODE_ENV !== 'production') {
      if (typeof reducers[key] === 'undefined') {
        warning(`No reducer provided for key "${key}"`)
      }
    }

    if (typeof reducers[key] === 'function') {
      finalReducers[key] = reducers[key]
    }
  }
  // 拿到过滤后的 reducers 的 key 值
  const finalReducerKeys = Object.keys(finalReducers)
```

```
// 在开发环境下判断，保存不期望 key 的缓存用以下面做警告
let unexpectedKeyCache
if (process.env.NODE_ENV !== 'production') {
  unexpectedKeyCache = {}
}

let shapeAssertionError
try {
  // 该函数解析在下面
  assertReducerShape(finalReducers)
} catch (e) {
  shapeAssertionError = e
}

// combineReducers 函数返回一个函数，也就是合并后的 reducer 函数
// 该函数返回总的 state
// 并且你也可以发现这里使用了闭包，函数里面使用到了外面的一些属性
return function combination(state = {}, action) {
  if (shapeAssertionError) {
    throw shapeAssertionError
  }
  // 该函数解析在下面
  if (process.env.NODE_ENV !== 'production') {
    const warningMessage = getUnexpectedStateShapeWarningMessage(
      state,
      finalReducers,
      action,
      unexpectedKeyCache
    )
    if (warningMessage) {
      warning(warningMessage)
    }
  }
  // state 是否改变
  let hasChanged = false
  // 改变后的 state
```

```

const nextState = {}
for (let i = 0; i < finalReducerKeys.length; i++) {
  // 拿到相应的 key
  const key = finalReducerKeys[i]
  // 获得 key 对应的 reducer 函数
  const reducer = finalReducers[key]
  // state 树下的 key 是与 finalReducers 下的 key 相同的
  // 所以你在 combineReducers 中传入的参数的 key 即代表了 各个 reducer 也
  // 代表了各个 state
  const previousStateForKey = state[key]
  // 然后执行 reducer 函数获得该 key 值对应的 state
  const nextStateForKey = reducer(previousStateForKey, action)
  // 判断 state 的值, undefined 的话就报错
  if (typeof nextStateForKey === 'undefined') {
    const errorMessage = getUndefinedStateErrorMessage(key, action)
    throw new Error(errorMessage)
  }
  // 然后将 value 塞进去
  nextState[key] = nextStateForKey
  // 如果 state 改变
  hasChanged = hasChanged || nextStateForKey !== previousStateForKey
}
// state 只要改变过, 就返回新的 state
return hasChanged ? nextState : state
}
}

```

combineReducers 函数总的来说很简单, 总结来说就是接收一个对象, 将参数过滤后返回一个函数。该函数里有一个过滤参数后的对象 finalReducers, 遍历该对象, 然后执行对象中的每一个 reducer 函数, 最后将新的 state 返回。

接下来让我们来看看 combineReducers 中用到的两个函数

```

// 这是执行的第一个用于抛错的函数
function assertReducerShape(reducers) {

```

```

// 将 combineReducers 中的参数遍历
Object.keys(reducers).forEach(key => {
  const reducer = reducers[key]
  // 给他传入一个 action
  const initialState = reducer(undefined, { type: ActionTypes.INIT })
  // 如果得到的 state 为 undefined 就抛错
  if (typeof initialState === 'undefined') {
    throw new Error(
      `Reducer "${key}" returned undefined during initialization. ` +
      `If the state passed to the reducer is undefined, you must ` +
      `explicitly return the initial state. The initial state may ` +
      `not be undefined. If you don't want to set a value for this
reducer, ` +
      `you can use null instead of undefined.`
    )
  }
  // 再过滤一次，考虑到万一你在 reducer 中给 ActionTypes.INIT 返回了值
  // 传入一个随机的 action 判断值是否为 undefined
  const type =
    '@@redux/PROBE_UNKNOWN_ACTION_' +
    Math.random()
      .toString(36)
      .substring(7)
      .split('')
      .join('.')
  if (typeof reducer(undefined, { type }) === 'undefined') {
    throw new Error(
      `Reducer "${key}" returned undefined when probed with a random
type. ` +
      `Don't try to handle ${
        ActionTypes.INIT
      } or other actions in "redux/*" ` +
      `namespace. They are considered private. Instead, you must
return the ` +

```

```

        `current state for any unknown actions, unless it is
undefined, ` +
        `in which case you must return the initial state, regardless
of the ` +
        `action type. The initial state may not be undefined, but can
be null.`
    )
  }
})
}

```

```

function getUnexpectedStateShapeWarningMessage(
  inputState,
  reducers,
  action,
  unexpectedKeyCache
) {
  // 这里的 reducers 已经是 finalReducers
  const reducerKeys = Object.keys(reducers)
  const argumentName =
    action && action.type === ActionTypes.INIT
      ? 'preloadedState argument passed to createStore'
      : 'previous state received by the reducer'

  // 如果 finalReducers 为空
  if (reducerKeys.length === 0) {
    return (
      'Store does not have a valid reducer. Make sure the argument
passed ' +
      'to combineReducers is an object whose values are reducers.'
    )
  }

  // 如果你传入的 state 不是对象
  if (!isPlainObject(inputState)) {
    return (
      `The ${argumentName} has unexpected type of "` +

```

```

    {}.toString.call(inputState).match(/\s([a-zA-Z]+)/)[1] +
    `". Expected argument to be an object with the following ` +
    `keys: "${reducerKeys.join(' ', '')}"`
  )
}

// 将传入的 state 于 finalReducers 下的 key 做比较, 过滤出多余的 key
const unexpectedKeys = Object.keys(inputState).filter(
  key => !reducers.hasOwnProperty(key) && !unexpectedKeyCache[key]
)

unexpectedKeys.forEach(key => {
  unexpectedKeyCache[key] = true
})

if (action && action.type === ActionTypes.REPLACE) return

// 如果 unexpectedKeys 有值的话
if (unexpectedKeys.length > 0) {
  return (
    `Unexpected ${unexpectedKeys.length > 1 ? 'keys' : 'key'} ` +
    `${unexpectedKeys.join(' ', '')}" found in ${argumentName}. ` +
    `Expected to find one of the known reducer keys instead: ` +
    `${reducerKeys.join(' ', '')}. Unexpected keys will be ignored.`
  )
}
}

```

接下来让我们先来看看 `compose` 函数

```

// 这个函数设计的很巧妙, 通过传入函数引用的方式让我们完成多个函数的嵌套使用, 术语叫做
// 高阶函数
// 通过使用 reduce 函数做到从右至左调用函数
// 对于上面项目中的例子
compose(
  applyMiddleware(thunkMiddleware),

```

```

    window.devToolsExtension ? window.devToolsExtension() : f => f
  )
  // 经过 compose 函数变成了 applyMiddleware(thunkMiddleware)
  (window.devToolsExtension())()
  // 所以在找不到 window.devToolsExtension 时你应该返回一个函数
  export default function compose(...funcs) {
    if (funcs.length === 0) {
      return arg => arg
    }

    if (funcs.length === 1) {
      return funcs[0]
    }

    return funcs.reduce((a, b) => (...args) => a(b(...args)))
  }

```

然后我们来解析 createStore 函数的部分代码

```

export default function createStore(reducer, preloadedState, enhancer) {
  // 一般 preloadedState 用的少，判断类型，如果第二个参数是函数且没有第三个参数，
  就调换位置
  if (typeof preloadedState === 'function' && typeof enhancer ===
  'undefined') {
    enhancer = preloadedState
    preloadedState = undefined
  }
  // 判断 enhancer 是否是函数
  if (typeof enhancer !== 'undefined') {
    if (typeof enhancer !== 'function') {
      throw new Error('Expected the enhancer to be a function.')
    }

    // 类型没错的话，先执行 enhancer，然后再执行 createStore 函数
    return enhancer(createStore)(reducer, preloadedState)
  }
}

```

```

// 判断 reducer 是否是函数
if (typeof reducer !== 'function') {
  throw new Error('Expected the reducer to be a function.')
}
// 当前 reducer
let currentReducer = reducer
// 当前状态
let currentState = preloadedState
// 当前监听函数数组
let currentListeners = []
// 这是一个很重要的设计，为的就是每次在遍历监听器的时候保证 currentListeners 数组不变
// 可以考虑下只存在 currentListeners 的情况，如果我在某个 subscribe 中再次执行 subscribe
// 或者 unsubscribe，这样会导致当前的 currentListeners 数组大小发生改变，从而可能导致
// 索引出错
let nextListeners = currentListeners
// reducer 是否正在执行
let isDispatching = false
// 如果 currentListeners 和 nextListeners 相同，就赋值回去
function ensureCanMutateNextListeners() {
  if (nextListeners === currentListeners) {
    nextListeners = currentListeners.slice()
  }
}
// .....
}

```

接下来先来介绍 `applyMiddleware` 函数

在这之前我需要先来介绍一下函数柯里化，柯里化是一种将使用多个参数的一个函数转换成一系列使用一个参数的函数的技术。


```
function add(a,b) { return a + b }
```

```
add(1, 2) => 3
```

// 对于以上函数如果使用柯里化可以这样改造

```
function add(a) {  
  return b => {  
    return a + b  
  }  
}
```

```
add(1)(2) => 3
```

// 你可以这样理解函数柯里化，通过闭包保存了外部的一个变量，然后返回一个接收参数的函数，在该函数中使用了保存的变量，然后再返回值。

// 这个函数应该是整个源码中最难理解的一块了

// 该函数返回一个柯里化的函数

// 所以调用这个函数应该这样写 applyMiddleware(...middlewares)(createStore)
(...args)

```
export default function applyMiddleware(...middlewares) {  
  return createStore => (...args) => {
```

// 这里执行 createStore 函数，把 applyMiddleware 函数最后次调用的参数传进来

```
    const store = createStore(...args)
```

```
    let dispatch = () => {
```

```
      throw new Error(
```

```
        `Dispatching while constructing your middleware is not allowed.
```

```
    ` +
```

```
        `Other middleware would not be applied to this dispatch.`
```

```
    )
```

```
  }
```

```
  let chain = []
```

// 每个中间件都应该有这两个函数

```
  const middlewareAPI = {
```

```
    getState: store.getState,
```

```
    dispatch: (...args) => dispatch(...args)
```

```
  }
```

// 把 middlewares 中的每个中间件都传入 middlewareAPI

```
  chain = middlewares.map(middleware => middleware(middlewareAPI))
```

```

// 和之前一样，从右至左调用每个中间件，然后传入 store.dispatch
dispatch = compose(...chain)(store.dispatch)
// 这里只看这部分代码有点抽象，我这里放入 redux-thunk 的代码来结合分析
// createThunkMiddleware返回了3层函数，第一层函数接收 middlewareAPI 参数
// 第二次函数接收 store.dispatch
// 第三层函数接收 dispatch 中的参数
{function createThunkMiddleware(extraArgument) {
  return ({ dispatch, getState }) => next => action => {
    // 判断 dispatch 中的参数是否为函数
    if (typeof action === 'function') {
      // 是函数的话再把这些参数传进去，直到 action 不为函数，执行 dispatch({type:
'XXX'})
      return action(dispatch, getState, extraArgument);
    }

    return next(action);
  };
}
const thunk = createThunkMiddleware();

export default thunk;}
// 最后把经过中间件加强后的 dispatch 于剩余 store 中的属性返回，这样你的
dispatch
  return {
    ...store,
    dispatch
  }
}
}
}

```

好了，我们现在将困难的部分都攻克了，来看一些简单的代码

```

// 这个没啥好说的，就是把当前的 state 返回，但是当正在执行 reducer 时不能执行该方法
function getState() {

```

```
    if (isDispatching) {
      throw new Error(
        'You may not call store.getState() while the reducer is
executing. ' +
        'The reducer has already received the state as an argument. '
+
        'Pass it down from the top reducer instead of reading it from
the store.'
      )
    }

    return currentState
  }
// 接收一个函数参数
function subscribe(listener) {
  if (typeof listener !== 'function') {
    throw new Error('Expected listener to be a function.')
  }
// 这部分最主要的设计 nextListeners 已经讲过，其他基本没什么好说的
  if (isDispatching) {
    throw new Error(
      'You may not call store.subscribe() while the reducer is
executing. ' +
      'If you would like to be notified after the store has been
updated, subscribe from a ' +
      'component and invoke store.getState() in the callback to
access the latest state. ' +
      'See http://redux.js.org/docs/api/Store.html#subscribe for
more details.'
    )
  }

  let isSubscribed = true

  ensureCanMutateNextListeners()
  nextListeners.push(listener)
```

```
// 返回一个取消订阅函数
return function unsubscribe() {
  if (!isSubscribed) {
    return
  }

  if (isDispatching) {
    throw new Error(
      'You may not unsubscribe from a store listener while the
reducer is executing. ' +
      'See http://redux.js.org/docs/api/Store.html#unsubscribe for
more details.'
    )
  }

  isSubscribed = false

  ensureCanMutateNextListeners()
  const index = nextListeners.indexOf(listener)
  nextListeners.splice(index, 1)
}

function dispatch(action) {
// 原生的 dispatch 会判断 action 是否为对象
  if (!isPlainObject(action)) {
    throw new Error(
      'Actions must be plain objects. ' +
      'Use custom middleware for async actions.'
    )
  }

  if (typeof action.type === 'undefined') {
    throw new Error(
      'Actions may not have an undefined "type" property. ' +

```

```

        'Have you misspelled a constant?'
    )
}
// 注意在 Reducers 中是不能执行 dispatch 函数的
// 因为你一旦在 reducer 函数中执行 dispatch, 会引发死循环
    if (isDispatching) {
        throw new Error('Reducers may not dispatch actions.')
    }
// 执行 combineReducers 组合后的函数
    try {
        isDispatching = true
        currentState = currentReducer(currentState, action)
    } finally {
        isDispatching = false
    }
// 然后遍历 currentListeners, 执行数组中保存的函数
    const listeners = (currentListeners = nextListeners)
    for (let i = 0; i < listeners.length; i++) {
        const listener = listeners[i]
        listener()
    }

    return action
}
// 然后在 createStore 末尾会发起一个 action dispatch({ type:
ActionTypes.INIT });
// 用以初始化 state

```

安全章节

XSS

跨网站指令码（英语：Cross-site scripting，通常简称为：XSS）是一种网站应用程序的安全漏洞攻击，是代码注入的一种。它允许恶意使用者将程式码注入到网页上，其他使用者在观看网页时就会受到影响。这类攻击通常包含了 HTML 以及使用者端脚本语言。

XSS 分为三种：反射型，存储型和 DOM-based

如何攻击

XSS 通过修改 HTML 节点或者执行 JS 代码来攻击网站。

例如通过 URL 获取某些参数

```
<!-- http://www.domain.com?name=<script>alert(1)</script> -->
<div>{{name}}</div>
```

上述 URL 输入可能会将 HTML 改为 `<div><script>alert(1)</script></div>`，这样页面中就凭空多了一段可执行脚本。这种攻击类型是反射型攻击，也可以说是 DOM-based 攻击。

也有另一种场景，比如写了一篇包含攻击代码 `<script>alert(1)</script>` 的文章，那么可能浏览文章的用户都会被攻击到。这种攻击类型是存储型攻击，也可以说是 DOM-based 攻击，并且这种攻击打击面更广。

如何防御

最普遍的做法是转义输入输出的内容，对于引号，尖括号，斜杠进行转义

```
function escape(str) {
  str = str.replace(/&/g, "&amp;");
  str = str.replace(/</g, "&lt;");
  str = str.replace(/>/g, "&gt;");
  str = str.replace(/"/g, "&quot;");
  str = str.replace(/'/g, "&#39;");
  str = str.replace(/`/g, "&#96;");
  str = str.replace(/\\/g, "&#x2F;");
  return str
}
```

通过转义可以将攻击代码 `<script>alert(1)</script>` 变成

```
// -> &lt;script&gt;alert(1)&lt;&#x2F;script&gt;
escape('<script>alert(1)</script>')
```

对于显示富文本来说，不能通过上面的办法来转义所有字符，因为这样会把需要的格式也过滤掉。这种情况通常采用白名单过滤的办法，当然也可以通过黑名单过滤，但是考虑到需要过滤的标签和标签属性实在太多，更加推荐使用白名单的方式。

```
var xss = require("xss");
var html = xss('<h1 id="title">XSS Demo</h1><script>alert("xss");</script>');
// -> <h1>XSS Demo</h1>&lt;script&gt;alert("xss");&lt;/script&gt;
console.log(html);
```

以上示例使用了 `js-xss` 来实现。可以看到在输出中保留了 `h1` 标签且过滤了 `script` 标签

CSP

内容安全策略 (**CSP**) 是一个额外的安全层，用于检测并削弱某些特定类型的攻击，包括跨站脚本 (**XSS**) 和数据注入攻击等。无论是数据盗取、网站内容污染还是散发恶意软件，这些攻击都是主要的手段。

我们可以通过 CSP 来尽量减少 XSS 攻击。CSP 本质上也是建立白名单，规定了浏览器只能执行特定来源的代码。

通常可以通过 HTTP Header 中的 Content-Security-Policy 来开启 CSP

- 只允许加载本站资源

```
Content-Security-Policy: default-src 'self'
```

- 只允许加载 HTTPS 协议图片

```
Content-Security-Policy: img-src https://*
```

- 允许加载任何来源框架

```
Content-Security-Policy: child-src 'none'
```

更多属性可以查看 [这里](#)

CSRF

跨站请求伪造（英语：Cross-site request forgery），也被称为 **one-click attack** 或者 **session riding**，通常缩写为 **CSRF** 或者 **XSRF**，是一种挟制用户在当前已登录的Web应用程序上执行非本意的操作的攻击方法。^[1]跟跨網站指令碼（XSS）相比，**XSS** 利用的是用户对指定网站的信任，CSRF 利用的是网站对用户网页浏览器的信任。

简单点说，CSRF 就是利用用户的登录态发起恶意请求。

如何攻击

假设网站中有一个通过 Get 请求提交用户评论的接口，那么攻击者就可以在钓鱼网站中加入一个图片，图片的地址就是评论接口

```

```


如果接口是 Post 提交的，就相对麻烦点，需要用表单来提交接口

```
<form action="http://www.domain.com/xxx" id="CSRF" method="post">
  <input name="comment" value="attack" type="hidden">
</form>
```

如何防御

防范 CSRF 可以遵循以下几种规则：

1. Get 请求不对数据进行修改
2. 不让第三方网站访问到用户 Cookie
3. 阻止第三方网站请求接口
4. 请求时附带验证信息，比如验证码或者 token

SameSite

可以对 Cookie 设置 SameSite 属性。该属性设置 Cookie 不随着跨域请求发送，该属性可以很大程度减少 CSRF 的攻击，但是该属性目前并不是所有浏览器都兼容。

验证 Referer

对于需要防范 CSRF 的请求，我们可以通过验证 Referer 来判断该请求是否为第三方网站发起的。

Token

服务器下发一个随机 Token（算法不能复杂），每次发起请求时将 Token 携带上，服务器验证 Token 是否有效。

密码安全

密码安全虽然大多是后端的事情，但是作为一名优秀的前端程序员也需要熟悉这方面的知识。

加盐

对于密码存储来说，必然是不能明文存储在数据库中的，否则一旦数据库泄露，会对用户造成很大的损失。并且不建议只对密码单纯通过加密算法加密，因为存在彩虹表的关系。

通常需要对密码加盐，然后进行几次不同加密算法的加密。

```
// 加盐也就是给原密码添加字符串，增加原密码长度  
sha256(sha1(md5(salt + password + salt)))
```

但是加盐并不能阻止别人盗取账号，只能确保即使数据库泄露，也不会暴露用户的真实密码。一旦攻击者得到了用户的账号，可以通过暴力破解的方式破解密码。对于这种情况，通常使用验证码增加延时或者限制尝试次数的方式。并且一旦用户输入了错误的密码，也不能直接提示用户输错密码，而应该提示账号或密码错误。

网络章节

UDP

面向报文

UDP 是一个面向报文（报文可以理解为一段段的数据）的协议。意思就是 UDP 只是报文的搬运工，不会对报文进行任何拆分和拼接操作。

具体来说

- 在发送端，应用层将数据传递给传输层的 UDP 协议，UDP 只会给数据增加一个 UDP 头标识下是 UDP 协议，然后就传递给网络层了
- 在接收端，网络层将数据传递给传输层，UDP 只去除 IP 报文头就传递给应用层，不会任何拼接操作

不可靠性

1. UDP 是无连接的，也就是说通信不需要建立和断开连接。
2. UDP 也是不可靠的。协议收到什么数据就传递什么数据，并且也不会备份数据，对方能不能收到是不关心的
3. UDP 没有拥塞控制，一直会以恒定的速度发送数据。即使网络条件不好，也不会对发送速率进行调整。这样实现的弊端就是在网络条件不好的情况下可能会导致丢包，但是优点也很明显，在某些实时性要求高的场景（比如电话会议）就需要使用 UDP 而不是 TCP。

高效

因为 UDP 没有 TCP 那么复杂，需要保证数据不丢失且有序到达。所以 UDP 的头部开销小，只有八字节，相比 TCP 的至少二十字节要少得多，在传输数据报文时是很高效的。

UDP Header																															
0								1								2								3							
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
Source port																Destination port															
Length																Checksum															

头部包含了以下几个数据

- 两个十六位的端口号，分别为源端口（可选字段）和目标端口
- 整个数据报文的长度
- 整个数据报文的检验和（IPv4 可选 字段），该字段用于发现头部信息和数据中的错误

传输方式

UDP 不止支持一对一的传输方式，同样支持一对多，多对多，多对一的方式，也就是说 UDP 提供了单播，多播，广播的功能。

TCP

头部

TCP 头部比 UDP 头部复杂的多

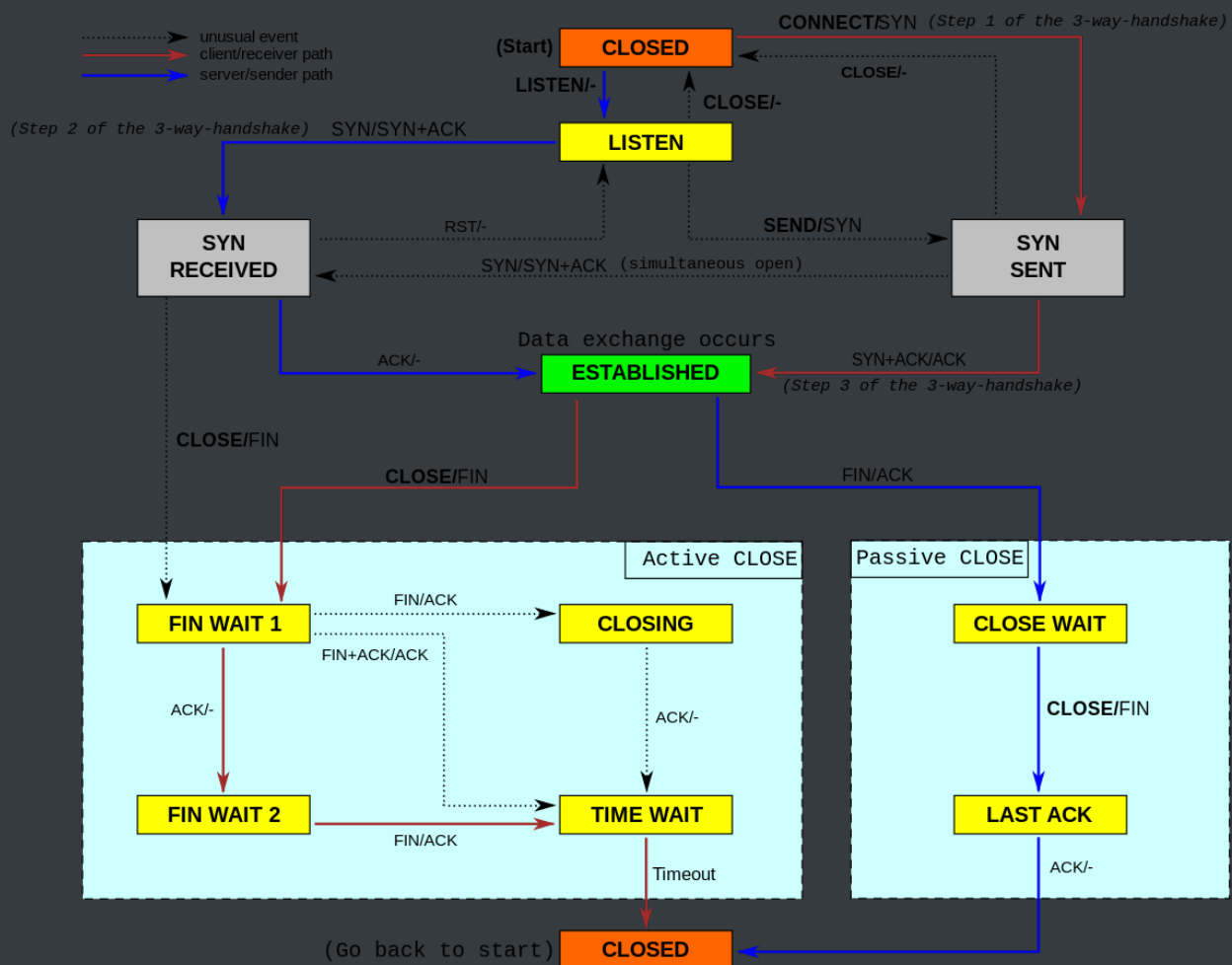
TCP Header																																		
Offsets	Octet	0								1								2								3								
Octet	Bit	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	
0	0	Source port																Destination port																
4	32	Sequence number																																
8	64	Acknowledgment number (if ACK set)																																
12	96	Data offset	Reserved			N	C	E	U	A	P	R	S	F	Window Size																			
		0	0	0	S	W	C	R	C	S	S	Y	I																					
						R	E	G	K	H	T	N	N																					
16	128	Checksum																Urgent pointer (if URG set)																
20	160	Options (if data offset > 5. Padded at the end with "0" bytes if necessary.)																																
...	...	...																																

对于 TCP 头部来说，以下几个字段是很重要的

- Sequence number，这个序号保证了 TCP 传输的报文都是有序的，对端可以通过序号顺序的拼接报文
- Acknowledgement Number，这个序号表示数据接收端期望接收的下一个字节的编号是多少，同时也表示上一个序号的数据已经收到
- Window Size，窗口大小，表示还能接收多少字节的数据，用于流量控制
- 标识符
 - URG=1：该字段为一表示本数据报的数据部分包含紧急信息，是一个高优先级数据报文，此时紧急指针有效。紧急数据一定位于当前数据包数据部分的最前面，紧急指针标明了紧急数据的尾部。
 - ACK=1：该字段为一表示确认号字段有效。此外，TCP 还规定在连接建立后传送的所有报文段都必须把 ACK 置为一。
 - PSH=1：该字段为一表示接收端应该立即将数据 push 给应用层，而不是等到缓冲区满后再提交。
 - RST=1：该字段为一表示当前 TCP 连接出现严重问题，可能需要重新建立 TCP 连接，也可以用于拒绝非法的报文段和拒绝连接请求。
 - SYN=1：当 SYN=1，ACK=0 时，表示当前报文段是一个连接请求报文。当 SYN=1，ACK=1 时，表示当前报文段是一个同意建立连接的应答报文。
 - FIN=1：该字段为一表示此报文段是一个释放连接的请求报文。

状态机

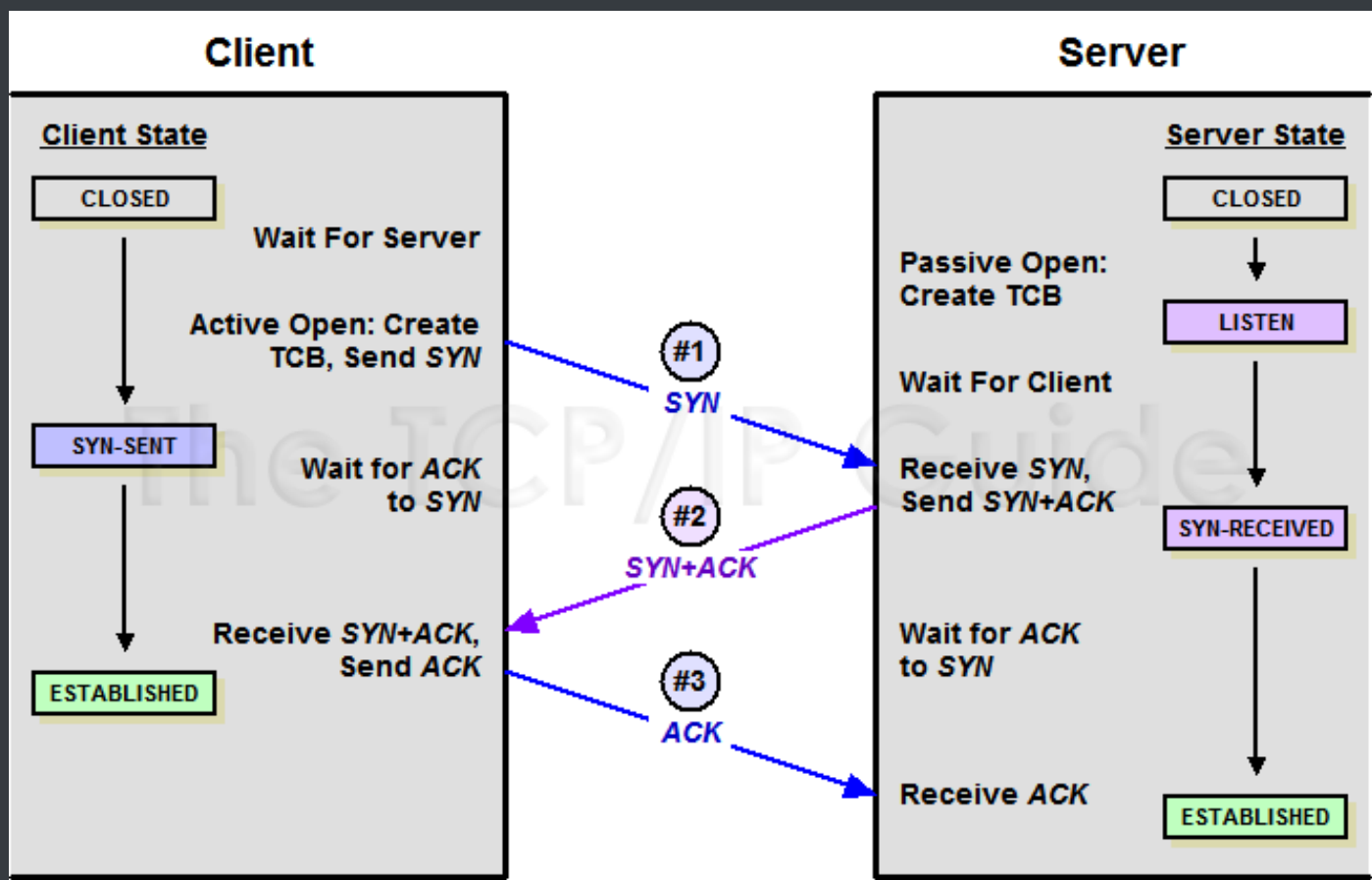
HTTP 是无连接的，所以作为下层的 TCP 协议也是无连接的，虽然看似 TCP 将两端连接了起来，但是其实只是两端共同维护了一个状态



TCP 的状态机是很复杂的，并且与建立断开连接时的握手息息相关，接下来就来详细描述下两种握手。

在这之前需要了解一个重要的性能指标 RTT。该指标表示发送端发送数据到接收到对端数据所需的往返时间。

建立连接三次握手



在 TCP 协议中，主动发起请求的一端为客户端，被动连接的一端称为服务端。不管是客户端还是服务端，TCP 连接建立完后都能发送和接收数据，所以 TCP 也是一个全双工的协议。

起初，两端都为 CLOSED 状态。在通信开始前，双方都会创建 TCB。服务器创建完 TCB 后遍进入 LISTEN 状态，此时开始等待客户端发送数据。

第一次握手

客户端向服务端发送连接请求报文段。该报文段中包含自身的数据通讯初始序号。请求发送后，客户端便进入 SYN-SENT 状态， x 表示客户端的数据通信初始序号。

第二次握手

服务端收到连接请求报文段后，如果同意连接，则会发送一个应答，该应答中也会包含自身的数据通讯初始序号，发送完成后便进入 SYN-RECEIVED 状态。

第三次握手

当客户端收到连接同意的应答后，还要向服务端发送一个确认报文。客户端发完这个报文段后便进入 ESTABLISHED 状态，服务端收到这个应答后也进入 ESTABLISHED 状态，此时连接建立成功。

PS：第三次握手可以包含数据，通过 TCP 快速打开（TFO）技术。其实只要涉及到握手的协议，都可以使用类似 TFO 的方式，客户端和服务端存储相同 cookie，下次握手时发出 cookie 达到减少 RTT 的目的。

你是否有疑惑明明两次握手就可以建立起连接，为什么还需要第三次应答？

因为这是为了防止失效的连接请求报文段被服务端接收，从而产生错误。

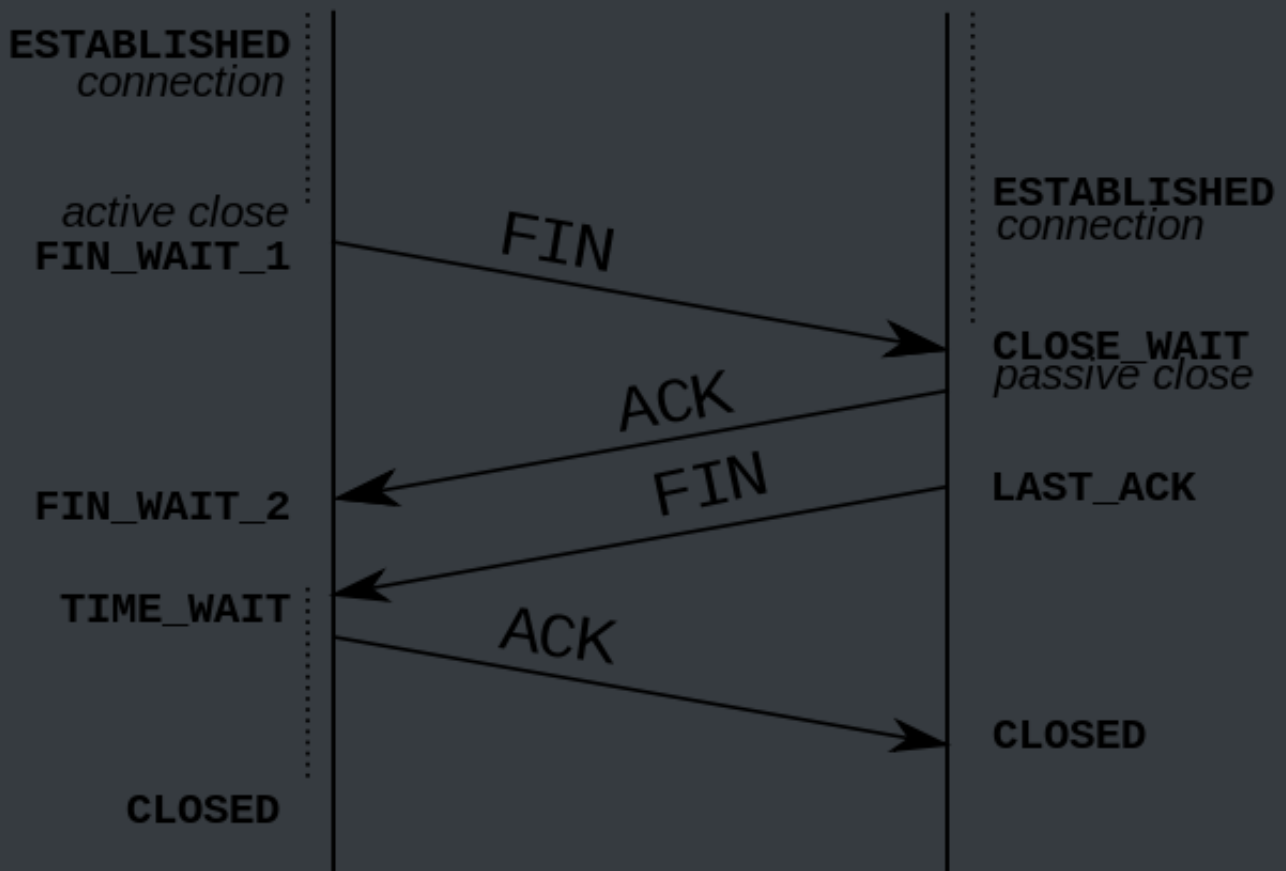
可以想象如下场景。客户端发送了一个连接请求 A，但是因为网络原因造成了超时，这时 TCP 会启动超时重传的机制再次发送一个连接请求 B。此时请求顺利到达服务端，服务端应答完就建立了请求。如果连接请求 A 在两端关闭后终于抵达了服务端，那么这时服务端会认为客户端又需要建立 TCP 连接，从而应答了该请求并进入 ESTABLISHED 状态。此时客户端其实是 CLOSED 状态，那么就会导致服务端一直等待，造成资源的浪费。

PS：在建立连接中，任意一端掉线，TCP 都会重发 SYN 包，一般会重试五次，在建立连接中可能会遇到 SYN FLOOD 攻击。遇到这种情况你可以选择调低重试次数或者干脆在不能处理的情况下拒绝请求。

断开链接四次握手

Initiator

Receiver



TCP 是全双工的，在断开连接时两端都需要发送 FIN 和 ACK。

第一次握手

若客户端 A 认为数据发送完成，则它需要向服务端 B 发送连接释放请求。

第二次握手

B 收到连接释放请求后，会告诉应用层要释放 TCP 链接。然后会发送 ACK 包，并进入 CLOSE_WAIT 状态，表示 A 到 B 的连接已经释放，不接收 A 发的数据了。但是因为 TCP 连接时双向的，所以 B 仍旧可以发送数据给 A。

第三次握手

B 如果此时还有没发完的数据会继续发送，完毕后会向 A 发送连接释放请求，然后 B 便进入 LAST-ACK 状态。

PS：通过延迟确认的技术（通常有时间限制，否则对方会误认为需要重传），可以将第二次和第三次握手合并，延迟 ACK 包的发送。

第四次握手

A 收到释放请求后，向 B 发送确认应答，此时 A 进入 TIME-WAIT 状态。该状态会持续 2MSL（最大段生存期，指报文段在网络中生存的时间，超时会被抛弃）时间，若该时间段内没有 B 的重发请求的话，就进入 CLOSED 状态。当 B 收到确认应答后，也便进入 CLOSED 状态。

为什么 A 要进入 TIME-WAIT 状态，等待 2MSL 时间后才进入 CLOSED 状态？

为了保证 B 能收到 A 的确认应答。若 A 发完确认应答后直接进入 CLOSED 状态，如果确认应答因为网络问题一直没有到达，那么会造成 B 不能正常关闭。

ARQ 协议

ARQ 协议也就是超时重传机制。通过确认和超时机制保证了数据的正确送达，ARQ 协议包含停止等待 ARQ 和连续 ARQ

停止等待 ARQ

正常传输过程

只要 A 向 B 发送一段报文，都要停止发送并启动一个定时器，等待对端回应，在定时器时间内接收到对端应答就取消定时器并发送下一段报文。

报文丢失或出错

在报文传输的过程中可能会出现丢包。这时候超过定时器设定的时间就会再次发送丢包的数据直到对端响应，所以需要每次都备份发送的数据。

即使报文正常的传输到对端，也可能出现在传输过程中报文出错的问题。这时候对端会抛弃该报文并等待 A 端重传。

PS：一般定时器设定的时间都会大于一个 RTT 的平均时间。

ACK 超时或丢失

对端传输的应答也可能出现丢失或超时的情况。那么超过定时器时间 A 端照样会重传报文。这时候 B 端收到相同序号的报文会丢弃该报文并重传应答，直到 A 端发送下一个序号的报文。

在超时的情况下也可能出现应答很迟到达，这时 A 端会判断该序号是否已经接收过，如果接收过只需要丢弃应答即可。

这个协议的缺点就是传输效率低，在良好的网络环境下每次发送报文都得等待对端的 ACK。

连续 ARQ

在连续 ARQ 中，发送端拥有一个发送窗口，可以在没有收到应答的情况下持续发送窗口内的数据，这样相比停止等待 ARQ 协议来说减少了等待时间，提高了效率。

累计确认

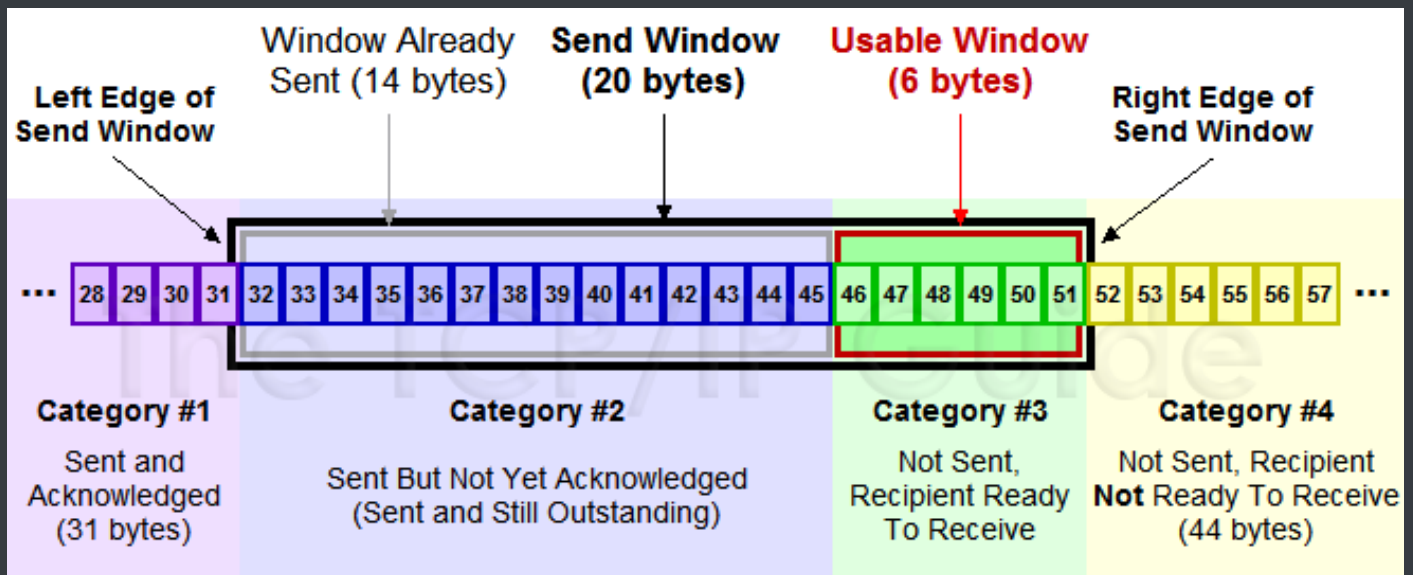
连续 ARQ 中，接收端会持续不断收到报文。如果和停止等待 ARQ 中接收一个报文就发送一个应答一样，就太浪费资源了。通过累计确认，可以在收到多个报文以后统一回复一个应答报文。报文中的 ACK 可以用来告诉发送端这个序号之前的数据已经全部接收到了，下次请发送这个序号 + 1 的数据。

但是累计确认也有一个弊端。在连续接收报文时，可能会遇到接收到序号 5 的报文后，并未接到序号 6 的报文，然而序号 7 以后的报文已经接收。遇到这种情况时，ACK 只能回复 6，这样会造成发送端重复发送数据，这种情况下可以通过 Sack 来解决，这个会在下文说到。

滑动窗口

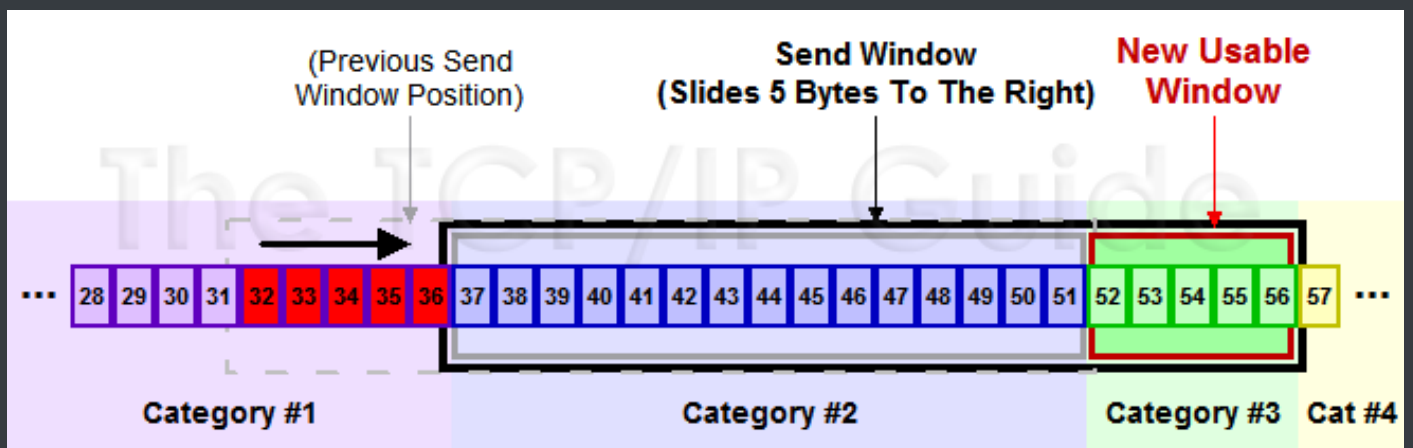
在上面小节中讲到了发送窗口。在 TCP 中，两端都维护着窗口：分别为发送端窗口和接收端窗口。

发送端窗口包含已发送但未收到应答的数据和可以发送但是未发送的数据。



发送端窗口是由接收窗口剩余大小决定的。接收方会把当前接收窗口的剩余大小写入应答报文，发送端收到应答后根据该值和当前网络拥塞情况设置发送窗口的大小，所以发送窗口的大小是不断变化的。

当发送端接收到应答报文后，会随之将窗口进行滑动



滑动窗口实现了流量控制。接收方通过报文告知发送方还可以发送多少数据，从而保证接收方能够来得及接收数据。

Zero 窗口

在发送报文的过程中，可能会遇到对端出现零窗口的情况。在该情况下，发送端会停止发送数据，并启动 persistent timer。该定时器会定时发送请求给对端，让对端告知窗口大小。在重试次数超过一定次数后，可能会中断 TCP 链接。

拥塞处理

拥塞处理和流量控制不同，后者是作用于接收方，保证接收方来得及接受数据。而前者是作用于网络，防止过多的数据拥塞网络，避免出现网络负载过大的情况。

拥塞处理包括了四个算法，分别为：慢开始，拥塞避免，快速重传，快速恢复。

慢开始算法

慢开始算法，顾名思义，就是在传输开始时将发送窗口慢慢指数级扩大，从而避免一开始就传输大量数据导致网络拥塞。

慢开始算法步骤具体如下

1. 连接初始设置拥塞窗口（Congestion Window）为 1 MSS（一个分段的最大数据量）
2. 每过一个 RTT 就将窗口大小乘二
3. 指数级增长肯定不能没有限制的，所以有一个阈值限制，当窗口大小大于阈值时就会启动拥塞避免算法。

拥塞避免算法

拥塞避免算法相比简单点，每过一个 RTT 窗口大小只加一，这样能够避免指数级增长导致网络拥塞，慢慢将大小调整到最佳值。

在传输过程中可能定时器超时的情况，这时候 TCP 会认为网络拥塞了，会马上进行以下步骤：

- 将阈值设为当前拥塞窗口的一半
- 将拥塞窗口设为 1 MSS
- 启动拥塞避免算法

快速重传

快速重传一般和快恢复一起出现。一旦接收端收到的报文出现失序的情况，接收端只会回复最后一个顺序正确的报文序号（没有 Sack 的情况下）。如果收到三个重复的 ACK，无需等待定时器超时再重发而是启动快速重传。具体算法分为两种：

TCP Tahoe 实现如下

- 将阈值设为当前拥塞窗口的一半
- 将拥塞窗口设为 1 MSS
- 重新开始慢开始算法

TCP Reno 实现如下

- 拥塞窗口减半
- 将阈值设为当前拥塞窗口
- 进入快恢复阶段（重发对端需要的包，一旦收到一个新的 ACK 答复就退出该阶段）
- 使用拥塞避免算法

TCP New Ren 改进后的快恢复

TCP New Reno 算法改进了之前 **TCP Reno** 算法的缺陷。在之前，快恢复中只要收到一个新的 ACK 包，就会退出快恢复。

在 **TCP New Reno** 中，TCP 发送方先记下三个重复 ACK 的分段的最大序号。

假如我有一个分段数据是 1 ~ 10 这十个序号的报文，其中丢失了序号为 3 和 7 的报文，那么该分段的最大序号就是 10。发送端只会收到 ACK 序号为 3 的应答。这时候重发序号为 3 的报文，接收方顺利接收并会发送 ACK 序号为 7 的应答。这时候 TCP 知道对端是有多个包未收到，会继续发送序号为 7 的报文，接收方顺利接收并会发送 ACK 序号为 11 的应答，这时发送端认为这个分段接收端已经顺利接收，接下来会退出快恢复阶段。

HTTP

HTTP 协议是个无状态协议，不会保存状态。

Post 和 Get 的区别

先引入副作用和幂等的概念。

副作用指对服务器上的资源做改变，搜索是无副作用的，注册是副作用的。

幂等指发送 M 和 N 次请求（两者不相同且都大于 1），服务器上资源的状态一致，比如注册 10 个和 11 个帐号是不幂等的，对文章进行更改 10 次和 11 次是幂等的。

在规范的应用场景上说，Get 多用于无副作用，幂等的场景，例如搜索关键字。Post 多用于副作用，不幂等的场景，例如注册。

在技术上说：

- Get 请求能缓存，Post 不能
- Post 相对 Get 安全一点点，因为Get 请求都包含在 URL 里，且会被浏览器保存历史记录，Post 不会，但是在抓包的情况下都是一样的。
- Post 可以通过 request body来传输比 Get 更多的数据，Get 没有这个技术
- URL有长度限制，会影响 Get 请求，但是这个长度限制是浏览器规定的，不是 RFC 规定的
- Post 支持更多的编码类型且不对数据类型限制

常见状态码

2XX 成功

- 200 OK，表示从客户端发来的请求在服务器端被正确处理
- 204 No content，表示请求成功，但响应报文不含实体的主体部分
- 205 Reset Content，表示请求成功，但响应报文不含实体的主体部分，但是与 204 响应不同在于要求请求方重置内容
- 206 Partial Content，进行范围请求

3XX 重定向

- 301 moved permanently，永久性重定向，表示资源已被分配了新的 URL
- 302 found，临时性重定向，表示资源临时被分配了新的 URL
- 303 see other，表示资源存在着另一个 URL，应使用 GET 方法获取资源
- 304 not modified，表示服务器允许访问资源，但因发生请求未满足条件的情况
- 307 temporary redirect，临时重定向，和302含义类似，但是期望客户端保持请求方法不变向新的地址发出请求

4XX 客户端错误

- 400 bad request, 请求报文存在语法错误
- 401 unauthorized, 表示发送的请求需要有通过 HTTP 认证的认证信息
- 403 forbidden, 表示对请求资源的访问被服务器拒绝
- 404 not found, 表示在服务器上没有找到请求的资源

5XX 服务器错误

- 500 internal sever error, 表示服务器端在执行请求时发生了错误
- 501 Not Implemented, 表示服务器不支持当前请求所需要的某个功能
- 503 service unavailable, 表明服务器暂时处于超负载或正在停机维护, 无法处理请求

HTTP 首部

通用字段	作用
Cache-Control	控制缓存的行为
Connection	浏览器想要优先使用的连接类型, 比如 keep-alive
Date	创建报文时间
Pragma	报文指令
Via	代理服务器相关信息
Transfer-Encoding	传输编码方式
Upgrade	要求客户端升级协议
Warning	在内容中可能存在错误

请求字段	作用
Accept	能正确接收的媒体类型
Accept-Charset	能正确接收的字符集
Accept-Encoding	能正确接收的编码格式列表
Accept-Language	能正确接收的语言列表
Expect	期待服务端的指定行为
From	请求方邮箱地址
Host	服务器的域名
If-Match	两端资源标记比较
If-Modified-Since	本地资源未修改返回 304（比较时间）
If-None-Match	本地资源未修改返回 304（比较标记）
User-Agent	客户端信息
Max-Forwards	限制可被代理及网关转发的次数
Proxy-Authorization	向代理服务器发送验证信息
Range	请求某个内容的一部分
Referer	表示浏览器所访问的前一个页面
TE	传输编码方式

响应字段	作用
Accept-Ranges	是否支持某些种类的范围
Age	资源在代理缓存中存在的时间
ETag	资源标识
Location	客户端重定向到某个 URL
Proxy-Authenticate	向代理服务器发送验证信息
Server	服务器名字
WWW-Authenticate	获取资源需要的验证信息

实体字段	作用
Allow	资源的正确请求方式
Content-Encoding	内容的编码格式
Content-Language	内容使用的语言
Content-Length	request body 长度
Content-Location	返回数据的备用地址
Content-MD5	Base64加密格式的内容 MD5检验值
Content-Range	内容的位置范围
Content-Type	内容的媒体类型
Expires	内容的过期时间
Last_modified	内容的最后修改时间

PS：缓存相关已在别的模块中写完，你可以 [阅读该小节](#)

HTTPS

HTTPS 还是通过了 HTTP 来传输信息，但是信息通过 TLS 协议进行了加密。

TLS

TLS 协议位于传输层之上，应用层之下。首次进行 TLS 协议传输需要两个 RTT，接下来可以通过 Session Resumption 减少到一个 RTT。

在 TLS 中使用了两种加密技术，分别为：对称加密和非对称加密。

对称加密：

对称加密就是两边拥有相同的密钥，两边都知道如何将密文加密解密。

非对称加密：

有公钥私钥之分，公钥所有人都是可以知道，可以将数据用公钥加密，但是将数据解密必须使用私钥解密，私钥只有分发公钥的一方才知道。

TLS 握手过程如下图：

1. 客户端发送一个随机值，需要的协议和加密方式
2. 服务端收到客户端的随机值，自己也产生一个随机值，并根据客户端需求的协议和加密方式来使用对应的方式，发送自己的证书（如果需要验证客户端证书需要说明）
3. 客户端收到服务端的证书并验证是否有效，验证通过会再生成一个随机值，通过服务端证书的公钥去加密这个随机值并发送给服务端，如果服务端需要验证客户端证书的话会附带证书
4. 服务端收到加密过的随机值并使用私钥解密获得第三个随机值，这时候两端都拥有了三个随机值，可以通过这三个随机值按照之前约定的加密方式生成密钥，接下来的通信就可以通过该密钥来加密解密

通过以上步骤可知，在 TLS 握手阶段，两端使用非对称加密的方式来通信，但是因为非对称加密损耗的性能比对称加密大，所以在正式传输数据时，两端使用对称加密的方式通信。

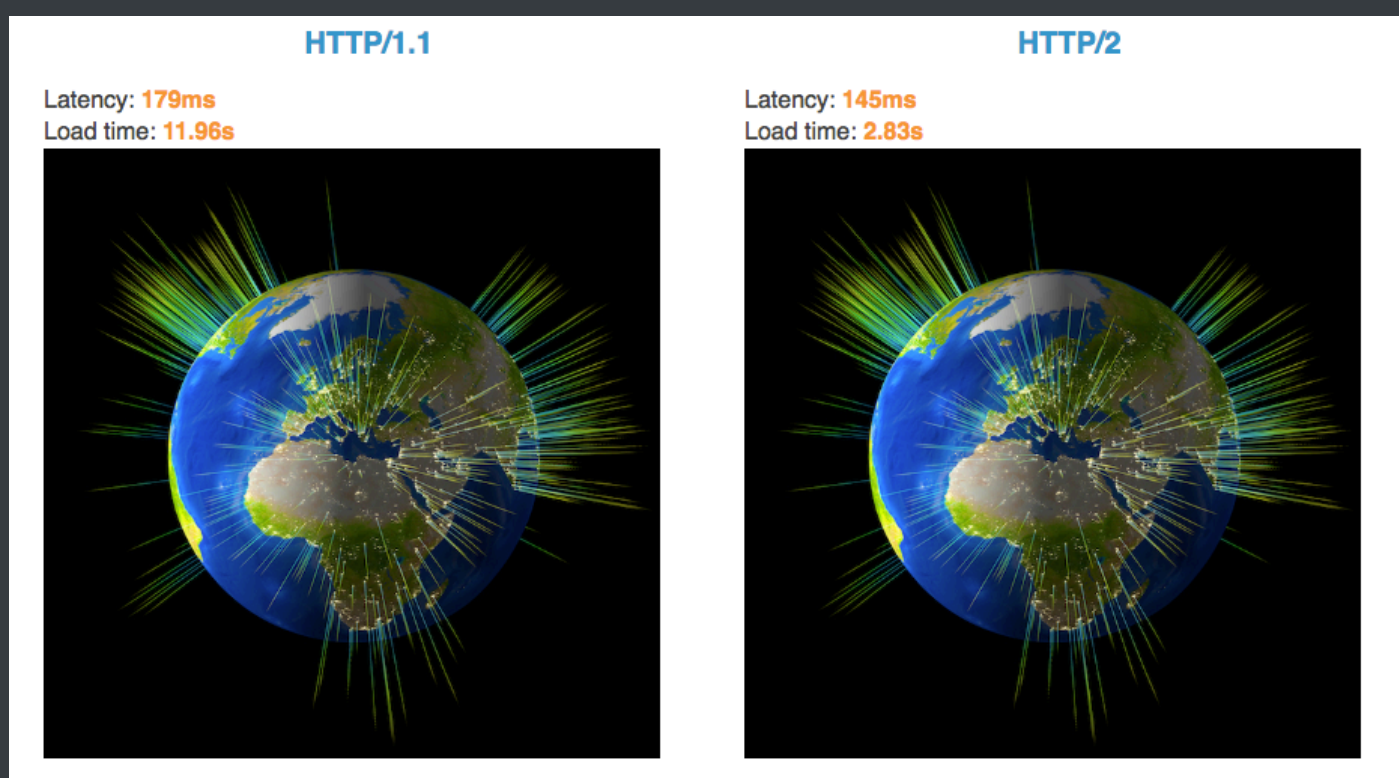
PS：以上说明的都是 TLS 1.2 协议的握手情况，在 1.3 协议中，首次建立连接只需要一个 RTT，后面恢复连接不需要 RTT 了。

HTTP 2.0

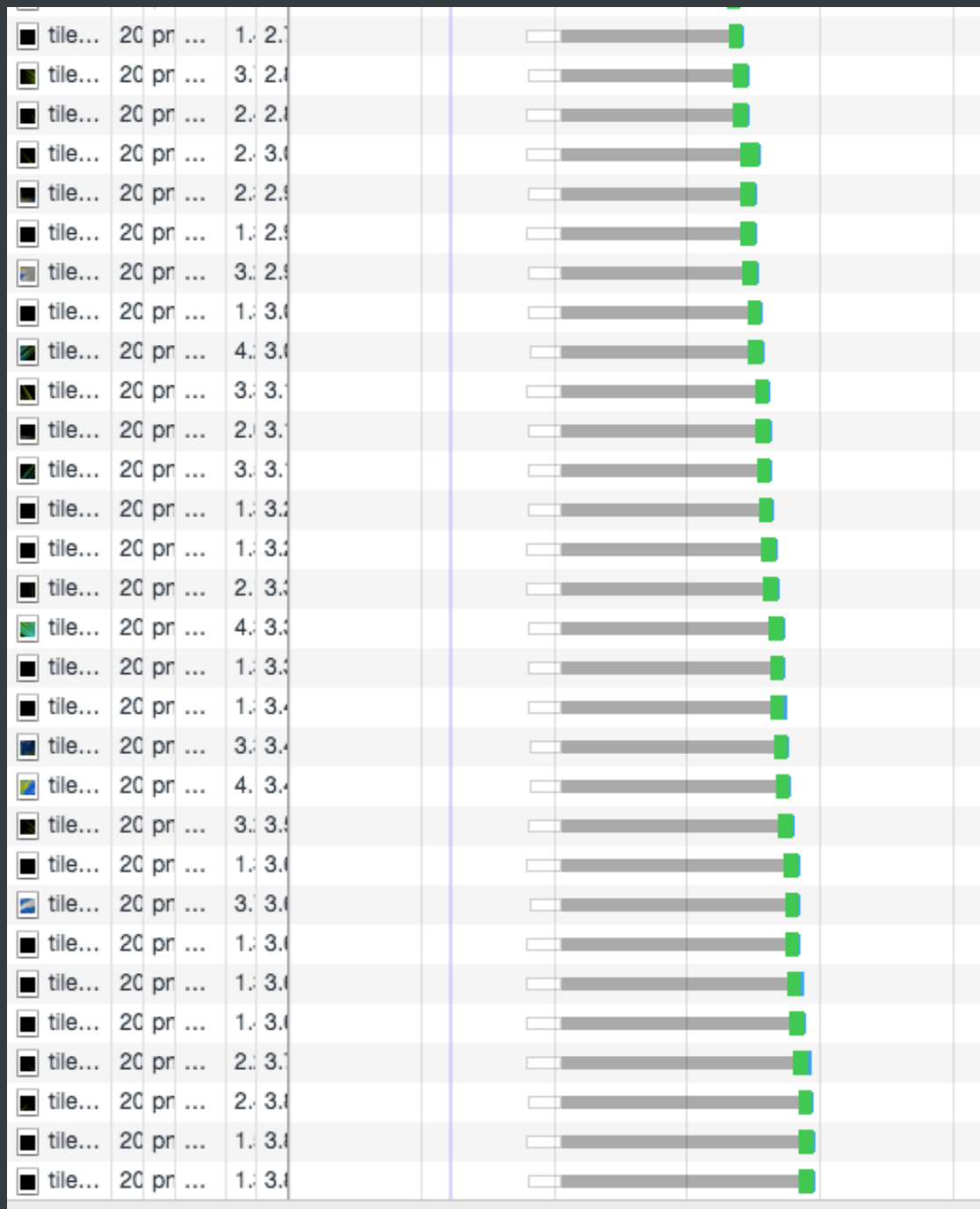
HTTP 2.0 相比于 HTTP 1.X, 可以说是大幅度提高了 web 的性能。

在 HTTP 1.X 中, 为了性能考虑, 我们会引入雪碧图、将小图内联、使用多个域名等等的方式。这一切都是因为浏览器限制了同一个域名下的请求数量, 当页面中需要请求很多资源的时候, 队头阻塞 (Head of line blocking) 会导致在达到最大请求数量时, 剩余的资源需要等待其他资源请求完成后才能发起请求。

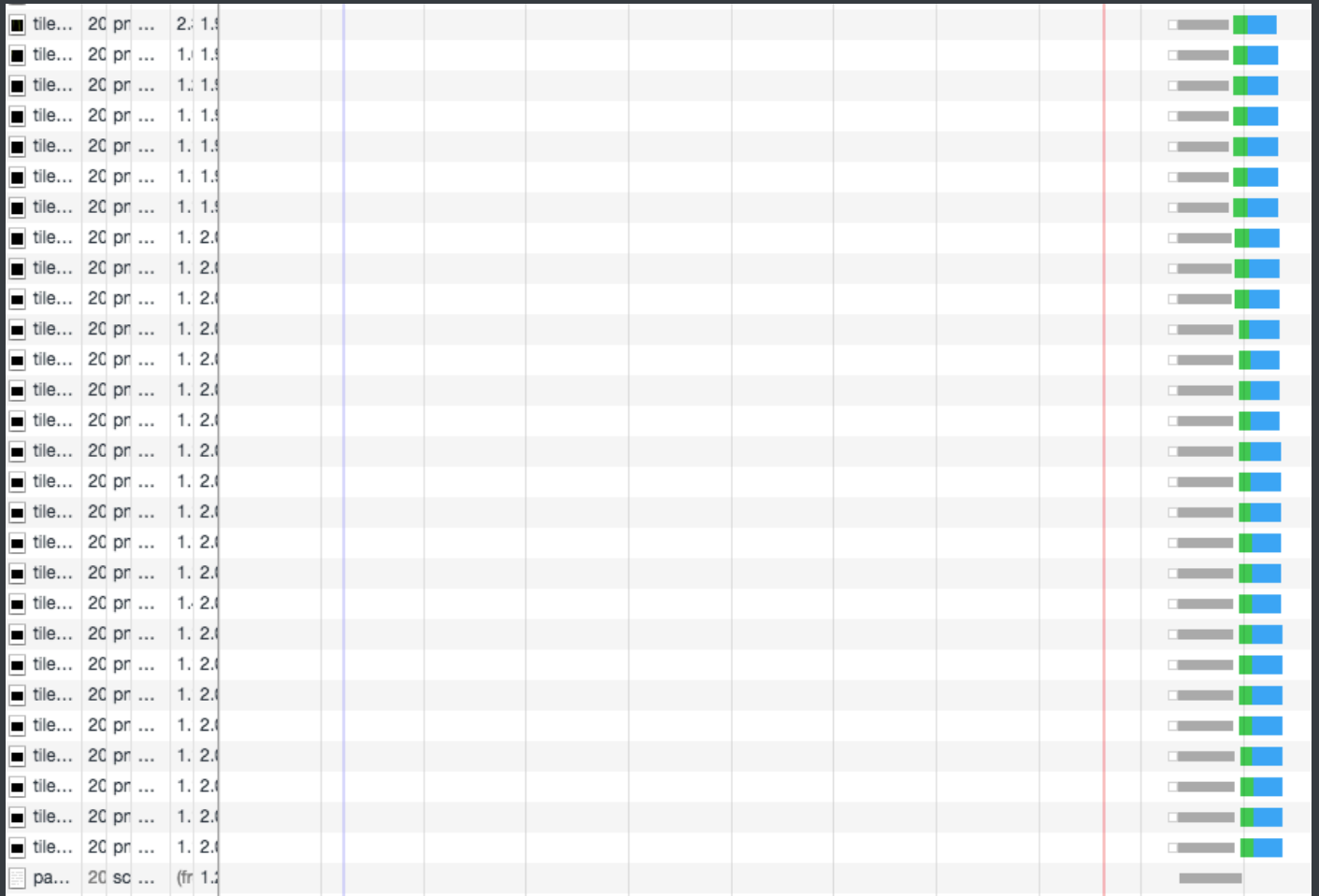
你可以通过 [该链接](#) 感受下 HTTP 2.0 比 HTTP 1.X 到底快了多少。



在 HTTP 1.X 中, 因为队头阻塞的原因, 你会发现请求是这样的

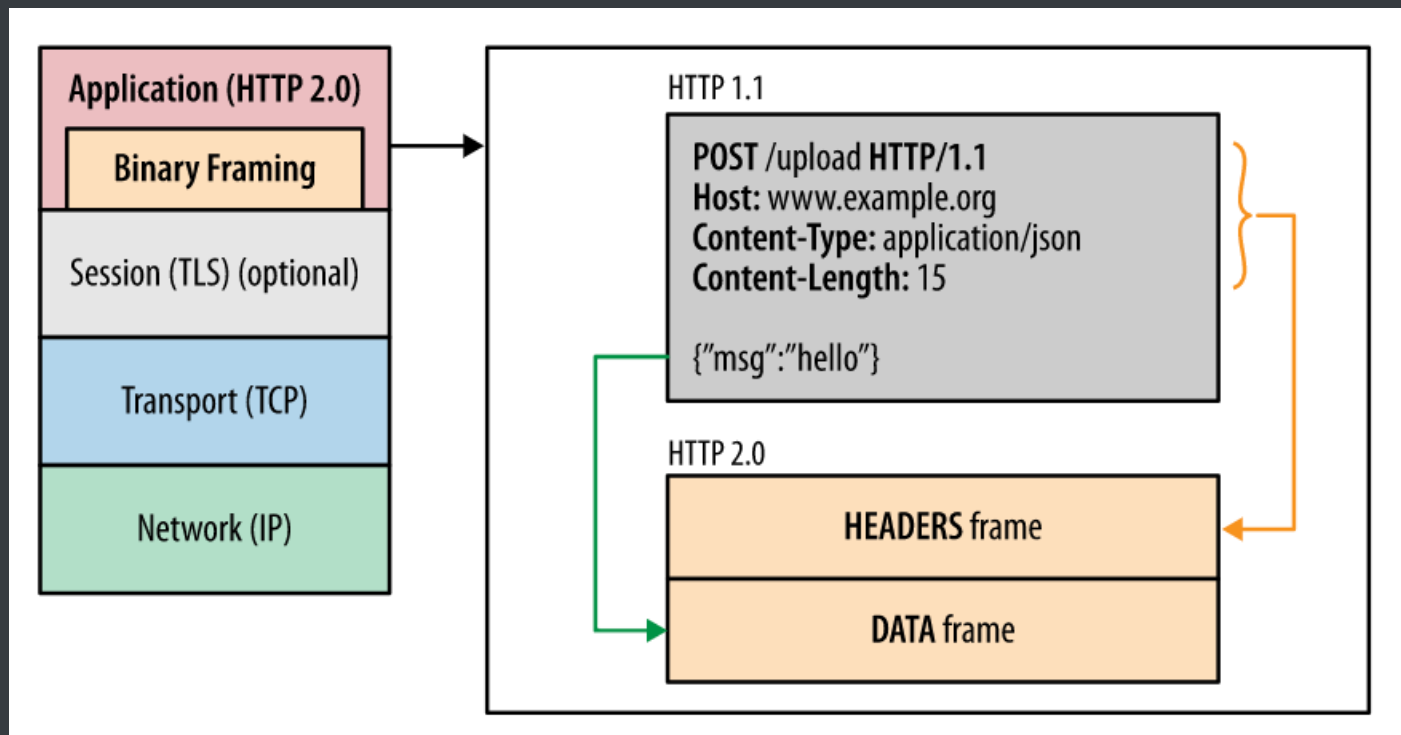


在 HTTP 2.0 中，因为引入了多路复用，你会发现请求是这样的



二进制传输

HTTP 2.0 中所有加强性能的核心点在于此。在之前的 HTTP 版本中，我们是通过文本的方式传输数据。在 HTTP 2.0 中引入了新的编码机制，所有传输的数据都会被分割，并采用二进制格式编码。

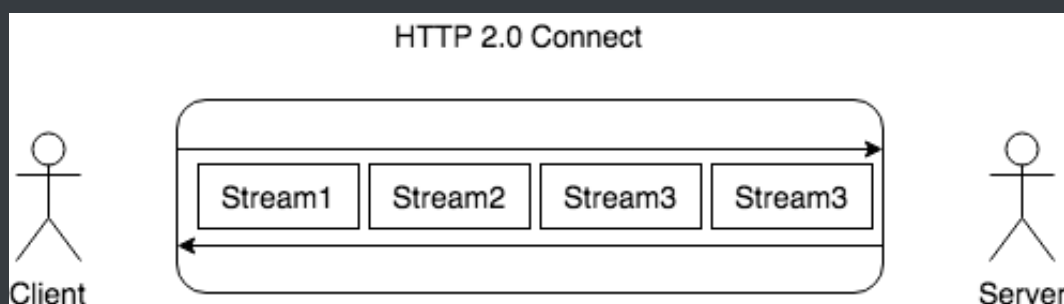


多路复用

在 HTTP 2.0 中，有两个非常重要的概念，分别是帧（frame）和流（stream）。

帧代表着最小的数据单位，每个帧会标识出该帧属于哪个流，流也就是多个帧组成的数据流。

多路复用，就是在一个 TCP 连接中可以存在多条流。换句话说，也就是可以发送多个请求，对端可以通过帧中的标识知道属于哪个请求。通过这个技术，可以避免 HTTP 旧版本中的队头阻塞问题，极大的提高传输性能。



Header 压缩

在 HTTP 1.X 中，我们使用文本的形式传输 header，在 header 携带 cookie 的情况下，可能每次都需要重复传输几百到几千的字节。

在 HTTP 2.0 中，使用了 HPACK 压缩格式对传输的 header 进行编码，减少了 header 的大小。并在两端维护了索引表，用于记录出现过的 header，后面在传输过程中就可以传输已经记录过的 header 的键名，对端收到数据后就可以通过键名找到对应的值。

服务端 Push

在 HTTP 2.0 中，服务端可以在客户端某个请求后，主动推送其他资源。

可以想象以下情况，某些资源客户端是一定会请求的，这时就可以采取服务端 push 的技术，提前给客户端推送必要的资源，这样就可以相对减少一点延迟时间。当然在浏览器兼容的情况下你也可以使用 prefetch。

QUIC

这是一个谷歌出品的基于 UDP 实现的同为传输层的协议，目标很远大，希望替代 TCP 协议。

- 该协议支持多路复用，虽然 HTTP 2.0 也支持多路复用，但是下层仍是 TCP，因为 TCP 的重传机制，只要一个包丢失就得判断丢失包并且重传，导致发生队头阻塞的问题，但是 UDP 没有这个机制
- 实现了自己的加密协议，通过类似 TCP 的 TFO 机制可以实现 0-RTT，当然 TLS 1.3 已经实现了 0-RTT 了
- 支持重传和纠错机制（向前恢复），在只丢失一个包的情况下不需要重传，使用纠错机制恢复丢失的包
 - 纠错机制：通过异或的方式，算出发出去的数据的异或值并单独发出一个包，服务端在发现有一个包丢失的情况下，通过其他数据包和异或值包算出丢失包
 - 在丢失两个包或以上的情况就使用重传机制，因为算不出来了

DNS

DNS 的作用就是通过域名查询到具体的 IP。

因为 IP 存在数字和英文的组合（IPv6），很不利于人类记忆，所以就出现了域名。你可以把域名看成是某个 IP 的别名，DNS 就是去查询这个别名的真正名称是什么。

在 TCP 握手之前就已经进行了 DNS 查询，这个查询是操作系统自己做的。当你在浏览器中想访问 `www.google.com` 时，会进行一下操作：

1. 操作系统会首先在本地缓存中查询
2. 没有的话会去系统配置的 DNS 服务器中查询
3. 如果这时候还没得话，会直接去 DNS 根服务器查询，这一步查询会找出负责 `com` 这个一级域名的服务器
4. 然后去该服务器查询 `google` 这个二级域名
5. 接下来三级域名的查询其实是我们配置的，你可以给 `www` 这个域名配置一个 IP，然后还可以给别的三级域名配置一个 IP

以上介绍的是 DNS 迭代查询，还有种是递归查询，区别就是前者是由客户端去做请求，后者是由系统配置的 DNS 服务器做请求，得到结果后将数据返回给客户端。

PS：DNS 是基于 UDP 做的查询。

从输入 URL 到页面加载完成的过程

这是一个很经典的面试题，在这题中可以将本文讲得内容都串联起来。

1. 首先做 DNS 查询，如果这一步做了智能 DNS 解析的话，会提供访问速度最快的 IP 地址回来
2. 接下来是 TCP 握手，应用层会下发数据给传输层，这里 TCP 协议会指明两端的端口号，然后下发给网络层。网络层中的 IP 协议会确定 IP 地址，并且指示了数据传输中如何跳转路由器。然后包会再被封装到数据链路层的数据帧结构中，最后就是物理层面的传输了
3. TCP 握手结束后会进行 TLS 握手，然后就开始正式的传输数据
4. 数据在进入服务端之前，可能还会先经过负责负载均衡的服务器，它的作用就是将请求合理的分发到多台服务器上，这时假设服务端会响应一个 HTML 文件
5. 首先浏览器会判断状态码是什么，如果是 200 那就继续解析，如果 400 或 500 的话就会报错，如果 300 的话会进行重定向，这里会有个重定向计数器，避免过多次的重定向，超过次数也会报错
6. 浏览器开始解析文件，如果是 gzip 格式的话会先解压一下，然后通过文件的编码格式知道如何去解码文件
7. 文件解码成功后会正式开始渲染流程，先会根据 HTML 构建 DOM 树，有 CSS 的话会去构建 CSSOM 树。如果遇到 script 标签的话，会判断是否存在 `async` 或者 `defer`，前者会并行进行下载并执行 JS，后者会先下载文件，然后等待 HTML 解析完成后顺序

执行，如果以上都没有，就会阻塞住渲染流程直到 JS 执行完毕。遇到文件下载的会去下载文件，这里如果使用 HTTP 2.0 协议的话会极大的提高多图的下载效率。

8. 初始的 HTML 被完全加载和解析后会触发 DOMContentLoaded 事件
9. CSSOM 树和 DOM 树构建完成后会开始生成 Render 树，这一步就是确定页面元素的布局、样式等等诸多方面的东西
10. 在生成 Render 树的过程中，浏览器就开始调用 GPU 绘制，合成图层，将内容显示在屏幕上了

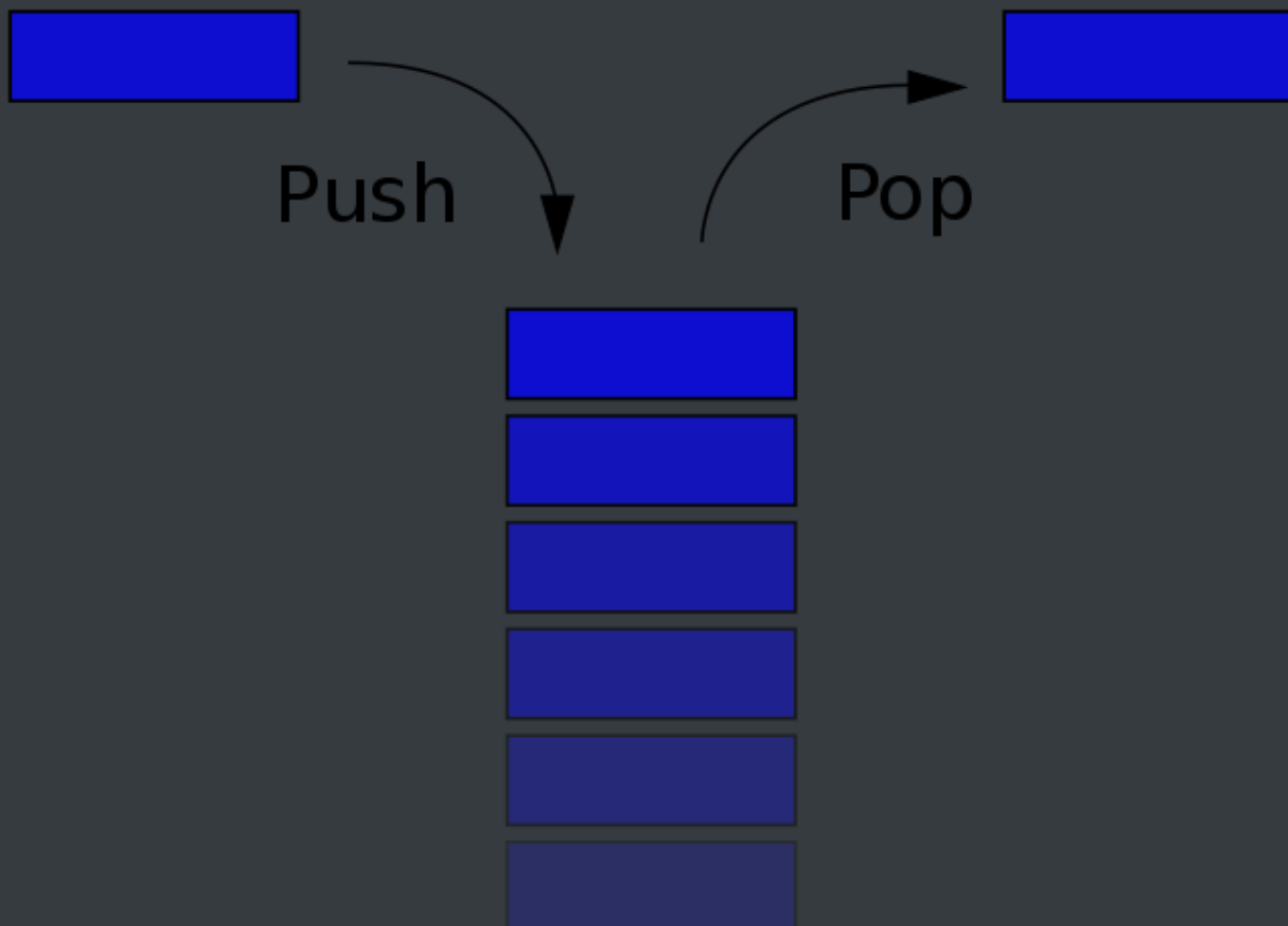
数据结构章节

栈

概念

栈是一个线性结构，在计算机中是一个相当常见的数据结构。

栈的特点是只能在某一端添加或删除数据，遵循先进后出的原则



实现

每种数据结构都可以用很多种方式来实现，其实可以把栈看成是数组的一个子集，所以这里使用数组来实现

```
class Stack {  
  constructor() {  
    this.stack = []  
  }  
  push(item) {  
    this.stack.push(item)  
  }  
  pop() {  
    this.stack.pop()  
  }  
  peek() {  
    return this.stack[this.getCount() - 1]  
  }  
}
```

```
}  
getCount() {  
    return this.stack.length  
}  
isEmpty() {  
    return this.getCount() === 0  
}  
}
```

应用

选取了 LeetCode 上序号为 20 的题目

题意是匹配括号，可以通过栈的特性来完成这道题目

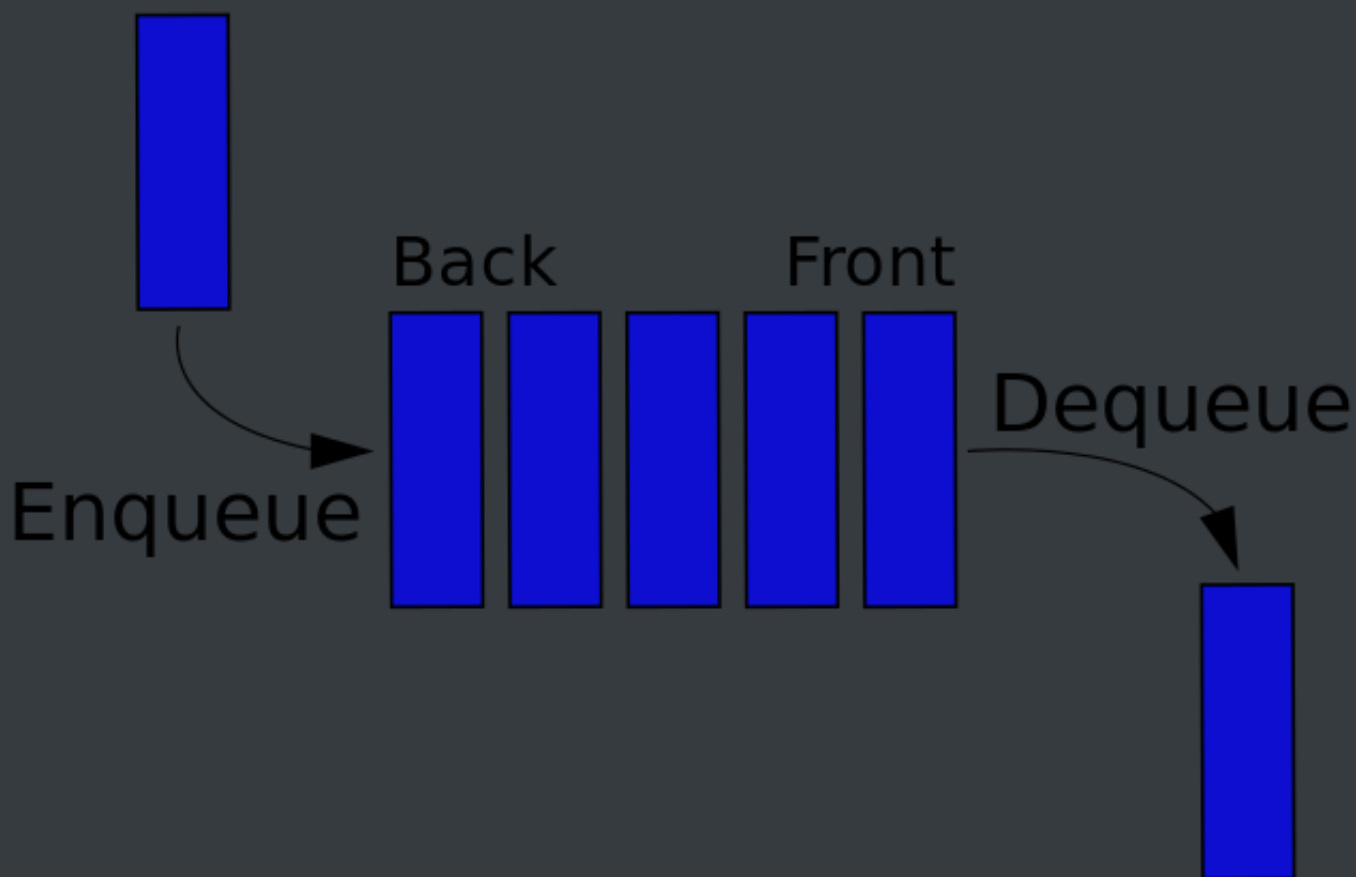
```
var isValid = function (s) {  
    let map = {  
        '(': -1,  
        ')': 1,  
        '[': -2,  
        ']': 2,  
        '{': -3,  
        '}': 3  
    }  
    let stack = []  
    for (let i = 0; i < s.length; i++) {  
        if (map[s[i]] < 0) {  
            stack.push(s[i])  
        } else {  
            let last = stack.pop()  
            if (map[last] + map[s[i]] !== 0) return false  
        }  
    }  
    if (stack.length > 0) return false  
    return true  
}
```

```
};
```

队列

概念

队列一个线性结构，特点是在某一端添加数据，在另一端删除数据，遵循先进先出的原则。



实现

这里会讲解两种实现队列的方式，分别是单链队列和循环队列。

单链队列

```
class Queue {  
    constructor() {  
        this.queue = []  
    }  
    enqueue(item) {
```

```

        this.queue.push(item)
    }
    deQueue() {
        return this.queue.shift()
    }
    getHeader() {
        return this.queue[0]
    }
    getLength() {
        return this.queue.length
    }
    isEmpty() {
        return this.getLength() === 0
    }
}

```

因为单链队列在出队操作的时候需要 $O(n)$ 的时间复杂度，所以引入了循环队列。循环队列的出队操作平均是 $O(1)$ 的时间复杂度。

循环队列

```

class SqQueue {
    constructor(length) {
        this.queue = new Array(length + 1)
        // 队头
        this.first = 0
        // 队尾
        this.last = 0
        // 当前队列大小
        this.size = 0
    }
    enqueue(item) {
        // 判断队尾 + 1 是否为队头
        // 如果是就代表需要扩容数组
        // % this.queue.length 是为了防止数组越界
    }
}

```

```

    if (this.first === (this.last + 1) % this.queue.length) {
        this.resize(this.getLength() * 2 + 1)
    }
    this.queue[this.last] = item
    this.size++
    this.last = (this.last + 1) % this.queue.length
}
deQueue() {
    if (this.isEmpty()) {
        throw Error('Queue is empty')
    }
    let r = this.queue[this.first]
    this.queue[this.first] = null
    this.first = (this.first + 1) % this.queue.length
    this.size--
    // 判断当前队列大小是否过小
    // 为了保证不浪费空间，在队列空间等于总长度四分之一时
    // 且不为 2 时缩小总长度为当前的一半
    if (this.size === this.getLength() / 4 && this.getLength() / 2 !==
0) {
        this.resize(this.getLength() / 2)
    }
    return r
}
getHeader() {
    if (this.isEmpty()) {
        throw Error('Queue is empty')
    }
    return this.queue[this.first]
}
getLength() {
    return this.queue.length - 1
}
isEmpty() {
    return this.first === this.last
}

```

```

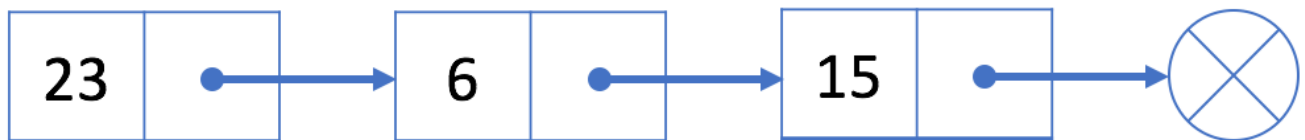
resize(length) {
  let q = new Array(length)
  for (let i = 0; i < length; i++) {
    q[i] = this.queue[(i + this.first) % this.queue.length]
  }
  this.queue = q
  this.first = 0
  this.last = this.size
}
}

```

链表

概念

链表是一个线性结构，同时也是一个天然的递归结构。链表结构可以充分利用计算机内存空间，实现灵活的内存动态管理。但是链表失去了数组随机读取的优点，同时链表由于增加了结点的指针域，空间开销比较大。



实现

单向链表

```

class Node {
  constructor(v, next) {
    this.value = v
    this.next = next
  }
}

class LinkedList {

```

```
constructor() {
    // 链表长度
    this.size = 0
    // 虚拟头部
    this.dummyNode = new Node(null, null)
}
find(header, index, currentIndex) {
    if (index === currentIndex) return header
    return this.find(header.next, index, currentIndex + 1)
}
addNode(v, index) {
    this.checkIndex(index)
    // 当往链表末尾插入时, prev.next 为空
    // 其他情况时, 因为要插入节点, 所以插入的节点
    // 的 next 应该是 prev.next
    // 然后设置 prev.next 为插入的节点
    let prev = this.find(this.dummyNode, index, 0)
    prev.next = new Node(v, prev.next)
    this.size++
    return prev.next
}
insertNode(v, index) {
    return this.addNode(v, index)
}
addToFirst(v) {
    return this.addNode(v, 0)
}
addToLast(v) {
    return this.addNode(v, this.size)
}
removeNode(index, isLast) {
    this.checkIndex(index)
    index = isLast ? index - 1 : index
    let prev = this.find(this.dummyNode, index, 0)
    let node = prev.next
    prev.next = node.next
}
```



```

        node.next = null
        this.size--
        return node
    }
    removeFirstNode() {
        return this.removeNode(0)
    }
    removeLastNode() {
        return this.removeNode(this.size, true)
    }
    checkIndex(index) {
        if (index < 0 || index > this.size) throw Error('Index error')
    }
    getNode(index) {
        this.checkIndex(index)
        if (this.isEmpty()) return
        return this.find(this.dummyNode, index, 0).next
    }
    isEmpty() {
        return this.size === 0
    }
    getSize() {
        return this.size
    }
}

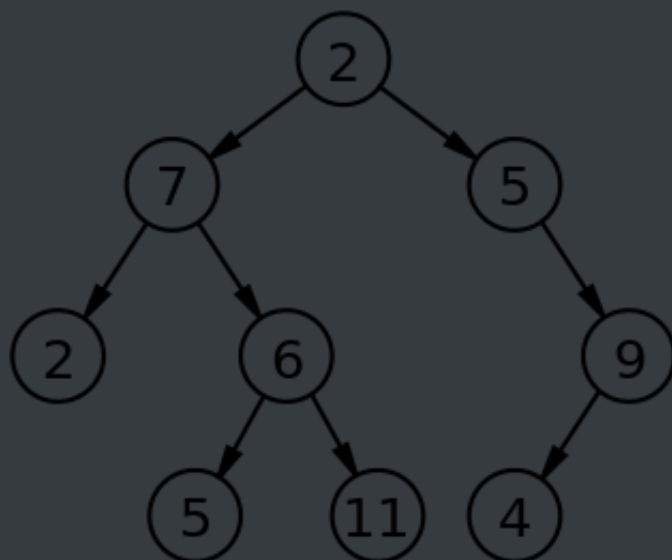
```

树

二叉树

树拥有很多种结构，二叉树是树中最常用的结构，同时也是一个天然的递归结构。

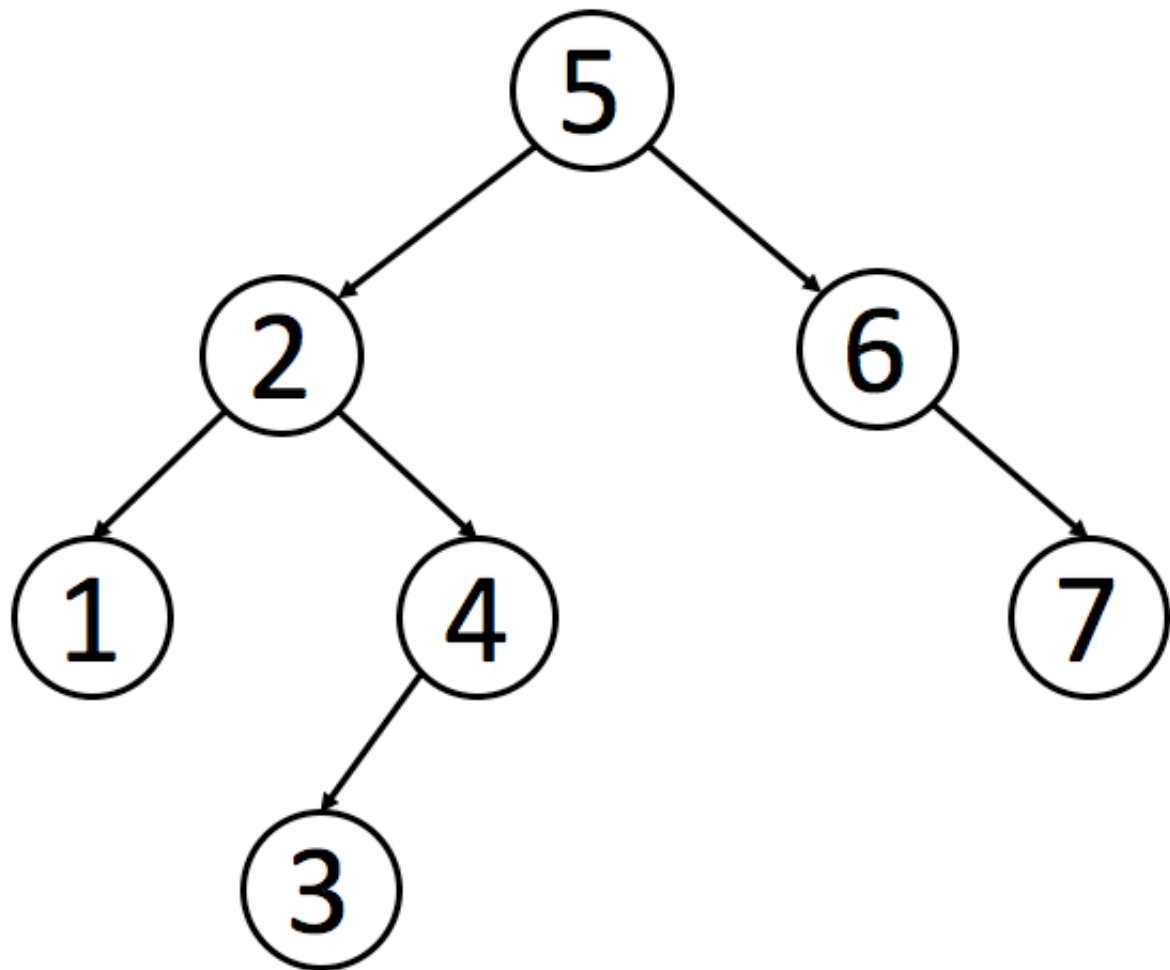
二叉树拥有一个根节点，每个节点至多拥有两个子节点，分别为：左节点和右节点。树的最底部节点称之为叶节点，当一颗树的叶数量数量为满时，该树可以称之为满二叉树。



二分搜索树

二分搜索树也是二叉树，拥有二叉树的特性。但是区别在于二分搜索树每个节点的值都比他的左子树的值大，比右子树的值小。

这种存储方式很适合于数据搜索。如下图所示，当需要查找 6 的时候，因为需要查找的值比根节点的值大，所以只需要在根节点的右子树上寻找，大大提高了搜索效率。



实现

```
class Node {
    constructor(value) {
        this.value = value
        this.left = null
        this.right = null
    }
}

class BST {
    constructor() {
        this.root = null
        this.size = 0
    }
    getSize() {
        return this.size
    }
}
```

```

    }
    isEmpty() {
        return this.size === 0
    }
    addNode(v) {
        this.root = this._addChild(this.root, v)
    }
    // 添加节点时，需要比较添加的节点值和当前
    // 节点值的大小
    _addChild(node, v) {
        if (!node) {
            this.size++
            return new Node(v)
        }
        if (node.value > v) {
            node.left = this._addChild(node.left, v)
        } else if (node.value < v) {
            node.right = this._addChild(node.right, v)
        }
        return node
    }
}

```

以上是最基本的二分搜索树实现，接下来实现树的遍历。

对于树的遍历来说，有三种遍历方法，分别是先序遍历、中序遍历、后序遍历。三种遍历的区别在于何时访问节点。在遍历树的过程中，每个节点都会遍历三次，分别是遍历到自己，遍历左子树和遍历右子树。如果只需要实现先序遍历，那么只需要第一次遍历到节点时进行操作即可。

以下都是递归实现，如果你想学习非递归实现，可以 [点击这里阅读](#)

```

// 先序遍历可用于打印树的结构
// 先序遍历先访问根节点，然后访问左节点，最后访问右节点。
preTraversal() {

```

```
    this._pre(this.root)
}
_pre(node) {
    if (node) {
        console.log(node.value)
        this._pre(node.left)
        this._pre(node.right)
    }
}
// 中序遍历可用于排序
// 对于 BST 来说，中序遍历可以实现一次遍历就
// 得到有序的值
// 中序遍历表示先访问左节点，然后访问根节点，最后访问右节点。
midTraversal() {
    this._mid(this.root)
}
_mid(node) {
    if (node) {
        this._mid(node.left)
        console.log(node.value)
        this._mid(node.right)
    }
}
// 后序遍历可用于先操作子节点
// 再操作父节点的场景
// 后序遍历表示先访问左节点，然后访问右节点，最后访问根节点。
backTraversal() {
    this._back(this.root)
}
_back(node) {
    if (node) {
        this._back(node.left)
        this._back(node.right)
        console.log(node.value)
    }
}
```

以上的这几种遍历都可以称之为深度遍历，对应的还有种遍历叫做广度遍历，也就是一层层地遍历树。对于广度遍历来说，我们需要利用之前讲过的队列结构来完成。

```
breadthTraversal() {
  if (!this.root) return null
  let q = new Queue()
  // 将根节点入队
  q.enqueue(this.root)
  // 循环判断队列是否为空，为空
  // 代表树遍历完毕
  while (!q.isEmpty()) {
    // 将队首出队，判断是否有左右子树
    // 有的话，就先左后右入队
    let n = q.dequeue()
    console.log(n.value)
    if (n.left) q.enqueue(n.left)
    if (n.right) q.enqueue(n.right)
  }
}
```

接下来先介绍如何在树中寻找最小值或最大数。因为二分搜索树的特性，所以最小值一定在根节点的最左边，最大值相反

```
getMin() {
  return this._getMin(this.root).value
}
_getMin(node) {
  if (!node.left) return node
  return this._getMin(node.left)
}
getMax() {
  return this._getMax(this.root).value
}
_getMax(node) {
```

```
    if (!node.right) return node
    return this._getMin(node.right)
}
```

向上取整和向下取整，这两个操作是相反的，所以代码也是类似的，这里只介绍如何向下取整。既然是向下取整，那么根据二分搜索树的特性，值一定在根节点的左侧。只需要一直遍历左子树直到当前节点的值不再大于等于需要的值，然后判断节点是否还拥有右子树。如果有的话，继续上面的递归判断。

```
floor(v) {
    let node = this._floor(this.root, v)
    return node ? node.value : null
}
_floor(node, v) {
    if (!node) return null
    if (node.value === v) return v
    // 如果当前节点值还比需要的值大，就继续递归
    if (node.value > v) {
        return this._floor(node.left, v)
    }
    // 判断当前节点是否拥有右子树
    let right = this._floor(node.right, v)
    if (right) return right
    return node
}
```

排名，这是用于获取给定值的排名或者排名第几的节点的值，这两个操作也是相反的，所以这个只介绍如何获取排名第几的节点的值。对于这个操作而言，我们需要略微的改造点代码，让每个节点拥有一个 `size` 属性。该属性表示该节点下有多少子节点（包含自身）。

```
class Node {
    constructor(value) {
        this.value = value
        this.left = null
        this.right = null
    }
}
```

```

        // 修改代码
        this.size = 1
    }
}
// 新增代码
_getSize(node) {
    return node ? node.size : 0
}
_addChild(node, v) {
    if (!node) {
        return new Node(v)
    }
    if (node.value > v) {
        // 修改代码
        node.size++
        node.left = this._addChild(node.left, v)
    } else if (node.value < v) {
        // 修改代码
        node.size++
        node.right = this._addChild(node.right, v)
    }
    return node
}
select(k) {
    let node = this._select(this.root, k)
    return node ? node.value : null
}
_select(node, k) {
    if (!node) return null
    // 先获取左子树下有几个节点
    let size = node.left ? node.left.size : 0
    // 判断 size 是否大于 k
    // 如果大于 k, 代表所需要的节点在左节点
    if (size > k) return this._select(node.left, k)
    // 如果小于 k, 代表所需要的节点在右节点
    // 注意这里需要重新计算 k, 减去根节点除了右子树的节点数量

```



```
    if (size < k) return this._select(node.right, k - size - 1)
    return node
}
```

接下来讲解的是二分搜索树中最难实现的部分：删除节点。因为对于删除节点来说，会存在以下几种情况

- 需要删除的节点没有子树
- 需要删除的节点只有一条子树
- 需要删除的节点有左右两条树

对于前两种情况很好解决，但是第三种情况就有难度了，所以先来实现相对简单的操作：删除最小节点，对于删除最小节点来说，是不存在第三种情况的，删除最大节点操作是和删除最小节点相反的，所以这里也就不再赘述。

```
delectMin() {
    this.root = this._delectMin(this.root)
    console.log(this.root)
}
_delectMin(node) {
    // 一直递归左子树
    // 如果左子树为空，就判断节点是否拥有右子树
    // 有右子树的话就把需要删除的节点替换为右子树
    if ((node !== null) & !node.left) return node.right
    node.left = this._delectMin(node.left)
    // 最后需要重新维护下节点的 `size`
    node.size = this._getSize(node.left) + this._getSize(node.right) + 1
    return node
}
```

最后讲解的就是如何删除任意节点了。对于这个操作，T.Hibbard 在 1962 年提出了解决这个难题的办法，也就是如何解决第三种情况。

当遇到这种情况时，需要取出当前节点的后继节点（也就是当前节点右子树的最小节点）来替换需要删除的节点。然后将需要删除节点的左子树赋值给后继节点，右子树删除后继节点后赋值给他。

你如果对于这个解决办法有疑问的话，可以这样考虑。因为二分搜索树的特性，父节点一定比所有左子节点大，比所有右子节点小。那么当需要删除父节点时，势必需要拿出一个比父节点大的节点来替换父节点。这个节点肯定不存在于左子树，必然存在于右子树。然后又需要保持父节点都是比右子节点小的，那么就可以取出右子树中最小的那个节点来替换父节点。

```
delect(v) {
  this.root = this._delect(this.root, v)
}
_delect(node, v) {
  if (!node) return null
  // 寻找的节点比当前节点小，去左子树找
  if (node.value < v) {
    node.right = this._delect(node.right, v)
  } else if (node.value > v) {
    // 寻找的节点比当前节点大，去右子树找
    node.left = this._delect(node.left, v)
  } else {
    // 进入这个条件说明已经找到节点
    // 先判断节点是否拥有左右子树中的一个
    // 是的话，将子树返回出去，这里和 `_delectMin` 的操作一样
    if (!node.left) return node.right
    if (!node.right) return node.left
    // 进入这里，代表节点拥有左右子树
    // 先取出当前节点的后继节点，也就是取当前节点右子树的最小值
    let min = this._getMin(node.right)
    // 取出最小值后，删除最小值
    // 然后把删除节点后的子树赋值给最小值节点
    min.right = this._delectMin(node.right)
    // 左子树不动
    min.left = node.left
  }
}
```

```
        node = min
    }
    // 维护 size
    node.size = this._getSize(node.left) + this._getSize(node.right) + 1
    return node
}
```

AVL 树

概念

二分搜索树实际在业务中是受到限制的，因为并不是严格的 $O(\log N)$ ，在极端情况下会退化成链表，比如加入一组升序的数字就会造成这种情况。

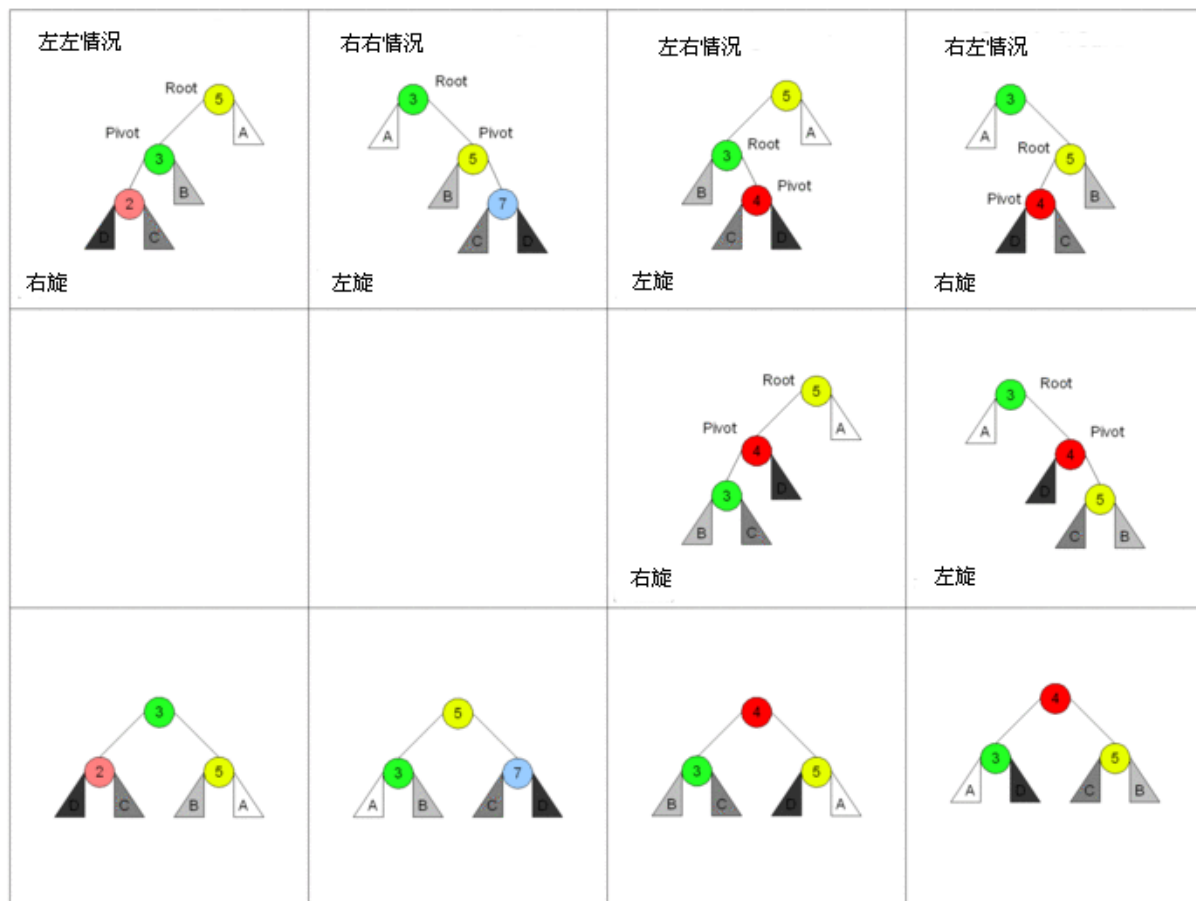
AVL 树改进了二分搜索树，在 AVL 树中任意节点的左右子树的高度差都不大于 1，这样保证了时间复杂度是严格的 $O(\log N)$ 。基于此，对 AVL 树增加或删除节点时可能需要旋转树来达到高度的平衡。

实现

因为 AVL 树是改进了二分搜索树，所以部分代码是于二分搜索树重复的，对于重复内容不作再次解析。

对于 AVL 树来说，添加节点会有四种情况

Root 是失去平衡的樹的根節點，Pivot 是旋轉后重新平衡的樹的根節點。



对于左左情况来说，新增加的节点位于节点 2 的左侧，这时树已经不平衡，需要旋转。因为搜索树的特性，节点比左节点大，比右节点小，所以旋转以后也要实现这个特性。

旋转之前：new < 2 < C < 3 < B < 5 < A，右旋之后节点 3 为根节点，这时候需要将节点 3 的右节点加到节点 5 的左边，最后还需要更新节点的高度。

对于右右情况来说，相反于左左情况，所以不再赘述。

对于左右情况来说，新增加的节点位于节点 4 的右侧。对于这种情况，需要通过两次旋转来达到目的。

首先对节点的左节点左旋，这时树满足左左的情况，再对节点进行一次右旋就可以达到目的。

```
class Node {
    constructor(value) {
        this.value = value
```

```

    this.left = null
    this.right = null
    this.height = 1
  }
}

class AVL {
  constructor() {
    this.root = null
  }
  addNode(v) {
    this.root = this._addChild(this.root, v)
  }
  _addChild(node, v) {
    if (!node) {
      return new Node(v)
    }
    if (node.value > v) {
      node.left = this._addChild(node.left, v)
    } else if (node.value < v) {
      node.right = this._addChild(node.right, v)
    } else {
      node.value = v
    }
    node.height =
      1 + Math.max(this._getHeight(node.left),
this._getHeight(node.right))
    let factor = this._getBalanceFactor(node)
    // 当需要右旋时，根节点的左树一定比右树高度高
    if (factor > 1 && this._getBalanceFactor(node.left) >= 0) {
      return this._rightRotate(node)
    }
    // 当需要左旋时，根节点的左树一定比右树高度矮
    if (factor < -1 && this._getBalanceFactor(node.right) <= 0) {
      return this._leftRotate(node)
    }
  }
}

```

```

// 左右情况
// 节点的左树比右树高，且节点的左树的右树比节点的左树的左树高
if (factor > 1 && this._getBalanceFactor(node.left) < 0) {
    node.left = this._leftRotate(node.left)
    return this._rightRotate(node)
}

// 右左情况
// 节点的左树比右树矮，且节点的右树的右树比节点的右树的左树矮
if (factor < -1 && this._getBalanceFactor(node.right) > 0) {
    node.right = this._rightRotate(node.right)
    return this._leftRotate(node)
}

return node
}

_getHeight(node) {
    if (!node) return 0
    return node.height
}

_getBalanceFactor(node) {
    return this._getHeight(node.left) - this._getHeight(node.right)
}

// 节点右旋
//
//      5              2
//     /  \          /  \
//    2    6  ==>  1    5
//   /  \        /    /  \
//  1    3      new  3    6
//   /
//  new
_rightRotate(node) {
    // 旋转后新根节点
    let newRoot = node.left
    // 需要移动的节点
    let moveNode = newRoot.right
    // 节点 2 的右节点改为节点 5

```

```

    newRoot.right = node
    // 节点 5 左节点改为节点 3
    node.left = moveNode
    // 更新树的高度
    node.height =
        1 + Math.max(this._getHeight(node.left),
this._getHeight(node.right))
    newRoot.height =
        1 +
        Math.max(this._getHeight(newRoot.left),
this._getHeight(newRoot.right))

```

```

    return newRoot
}

```

// 节点左旋

```

//          4              6
//        /  \          /  \
//       2    6    ==>  4    7
//            /  \      /  \   \
//           5    7    2    5    new
//                      \
//                     new

```

```

_leftRotate(node) {
    // 旋转后新根节点
    let newRoot = node.right
    // 需要移动的节点
    let moveNode = newRoot.left
    // 节点 6 的左节点改为节点 4
    newRoot.left = node
    // 节点 4 右节点改为节点 5
    node.right = moveNode
    // 更新树的高度
    node.height =
        1 + Math.max(this._getHeight(node.left),
this._getHeight(node.right))
    newRoot.height =

```

```
1 +  
    Math.max(this._getHeight(newRoot.left),  
this._getHeight(newRoot.right))  
  
    return newRoot  
  }  
}
```

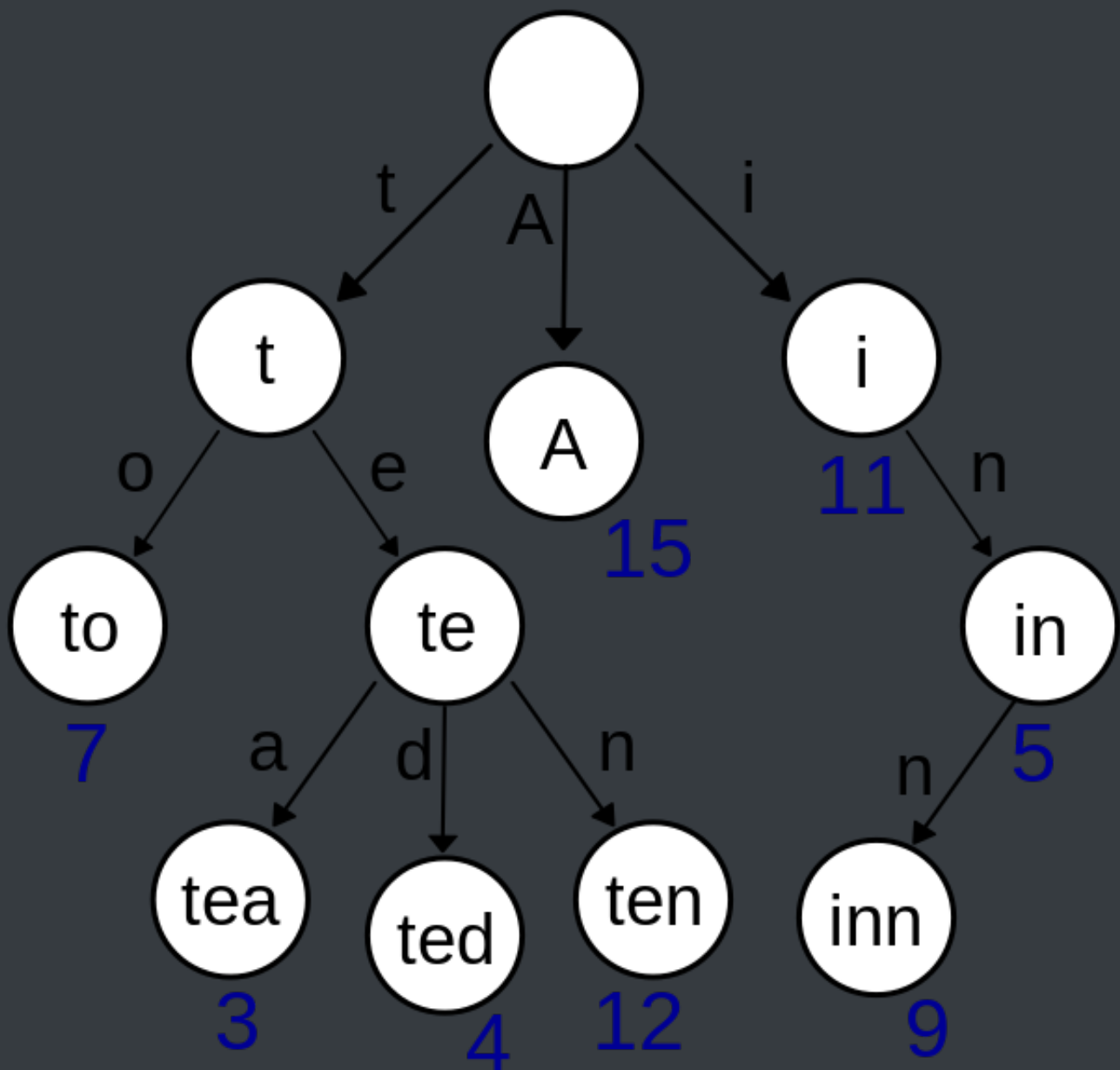
Trie

概念

在计算机科学，**trie**，又称**前缀树**或**字典树**，是一种有序树，用于保存关联数组，其中的键通常是字符串。

简单点来说，这个结构的作用大多是为了方便搜索字符串，该树有以下几个特点

- 根节点代表空字符串，每个节点都有 N（假如搜索英文字符，就有 26 条）条链接，每条链接代表一个字符
- 节点不存储字符，只有路径才存储，这点和其他的树结构不同
- 从根节点开始到任意一个节点，将沿途经过的字符连接起来就是该节点对应的字符串



实现

总的来说 Trie 的实现相比别的树结构来说简单的很多，实现就以搜索英文字符为例。

```
class TrieNode {  
    constructor() {  
        // 代表每个字符经过节点的次数  
        this.path = 0  
        // 代表到该节点的字符串有几个  
        this.end = 0  
    }  
}
```

```
        // 链接
        this.next = new Array(26).fill(null)
    }
}

class Trie {
    constructor() {
        // 根节点, 代表空字符
        this.root = new TrieNode()
    }

    // 插入字符串
    insert(str) {
        if (!str) return
        let node = this.root
        for (let i = 0; i < str.length; i++) {
            // 获得字符先对应的索引
            let index = str[i].charCodeAt() - 'a'.charCodeAt()
            // 如果索引对应没有值, 就创建
            if (!node.next[index]) {
                node.next[index] = new TrieNode()
            }
            node.path += 1
            node = node.next[index]
        }
        node.end += 1
    }

    // 搜索字符串出现的次数
    search(str) {
        if (!str) return
        let node = this.root
        for (let i = 0; i < str.length; i++) {
            let index = str[i].charCodeAt() - 'a'.charCodeAt()
            // 如果索引对应没有值, 代表没有需要搜索的字符串
            if (!node.next[index]) {
                return 0
            }
            node = node.next[index]
        }
    }
}
```

```

    }
    return node.end
}
// 删除字符串
delete(str) {
    if (!this.search(str)) return
    let node = this.root
    for (let i = 0; i < str.length; i++) {
        let index = str[i].charCodeAt() - 'a'.charCodeAt()
        // 如果索引对应的节点的 Path 为 0，代表经过该节点的字符串
        // 已经一个，直接删除即可
        if (--node.next[index].path == 0) {
            node.next[index] = null
            return
        }
        node = node.next[index]
    }
    node.end -= 1
}
}

```

并查集

概念

并查集是一种特殊的树结构，用于处理一些不交集的合并及查询问题。该结构中每个节点都有一个父节点，如果只有当前一个节点，那么该节点的父节点指向自己。

这个结构中有两个重要的操作，分别是：

- Find：确定元素属于哪一个子集。它可以被用来确定两个元素是否属于同一子集。
- Union：将两个子集合并成同一个集合。



MakeSet creates 8 singletons.



After some operations of Union, some sets are grouped together.

实现

```
class DisjointSet {
    // 初始化样本
    constructor(count) {
        // 初始化时，每个节点的父节点都是自己
        this.parent = new Array(count)
        // 用于记录树的深度，优化搜索复杂度
        this.rank = new Array(count)
        for (let i = 0; i < count; i++) {
            this.parent[i] = i
            this.rank[i] = 1
        }
    }
    find(p) {
        // 寻找当前节点的父节点是否为自己，不是的话表示还没找到
        // 开始进行路径压缩优化
        // 假设当前节点父节点为 A
        // 将当前节点挂载到 A 节点的父节点上，达到压缩深度的目的
        while (p !== this.parent[p]) {
            this.parent[p] = this.parent[this.parent[p]]
            p = this.parent[p]
        }
        return p
    }
    isConnected(p, q) {
```

```

    return this.find(p) === this.find(q)
  }
  // 合并
  union(p, q) {
    // 找到两个数字的父节点
    let i = this.find(p)
    let j = this.find(q)
    if (i === j) return
    // 判断两棵树的深度，深度小的加到深度大的树下面
    // 如果两棵树深度相等，那就无所谓怎么加
    if (this.rank[i] < this.rank[j]) {
      this.parent[i] = j
    } else if (this.rank[i] > this.rank[j]) {
      this.parent[j] = i
    } else {
      this.parent[i] = j
      this.rank[j] += 1
    }
  }
}

```

堆

概念

堆通常是一个可以被看做一棵树的数组对象。

堆的实现通过构造**二叉堆**，实为二叉树的一种。这种数据结构具有以下性质。

- 任意节点小于（或大于）它的所有子节点
- 堆总是一棵完全树。即除了最底层，其他层的节点都被元素填满，且最底层从左到右填入。

将根节点最大的堆叫做**最大堆**或**大根堆**，根节点最小的堆叫做**最小堆**或**小根堆**。

优先队列也完全可以用堆来实现，操作是一模一样的。

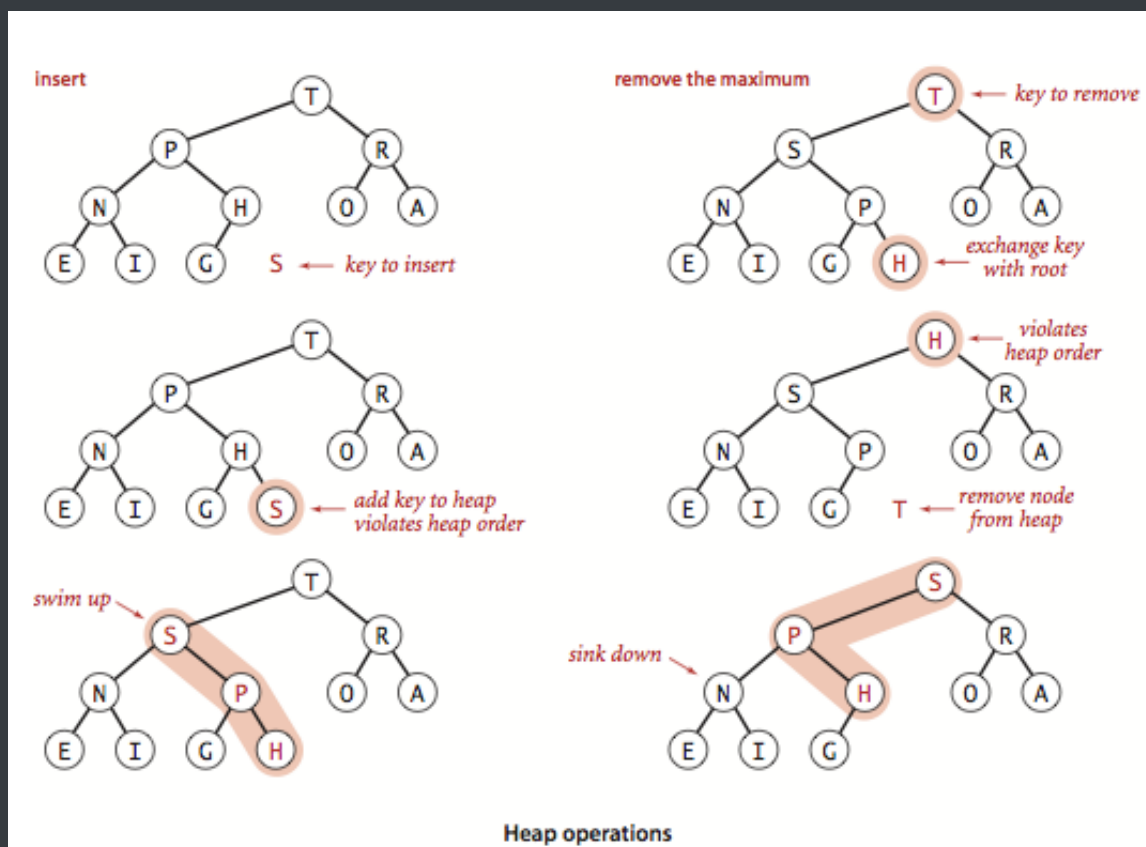
实现大根堆

堆的每个节点的左边子节点索引是 $i * 2 + 1$ ，右边是 $i * 2 + 2$ ，父节点是 $(i - 1) / 2$ 。

堆有两个核心的操作，分别是 `shiftUp` 和 `shiftDown`。前者用于添加元素，后者用于删除根节点。

`shiftUp` 的核心思路是一路将节点与父节点对比大小，如果比父节点大，就和父节点交换位置。

`shiftDown` 的核心思路是先将根节点和末尾交换位置，然后移除末尾元素。接下来循环判断父节点和两个子节点的大小，如果子节点大，就把最大的子节点和父节点交换。



```
class MaxHeap {
  constructor() {
    this.heap = []
  }
  size() {
```

```

    return this.heap.length
}
empty() {
    return this.size() == 0
}
add(item) {
    this.heap.push(item)
    this._shiftUp(this.size() - 1)
}
removeMax() {
    this._shiftDown(0)
}
getParentIndex(k) {
    return parseInt((k - 1) / 2)
}
getLeftIndex(k) {
    return k * 2 + 1
}
_shiftUp(k) {
    // 如果当前节点比父节点大，就交换
    while (this.heap[k] > this.heap[this.getParentIndex(k)]) {
        this._swap(k, this.getParentIndex(k))
        // 将索引变成父节点
        k = this.getParentIndex(k)
    }
}
_shiftDown(k) {
    // 交换首位并删除末尾
    this._swap(k, this.size() - 1)
    this.heap.splice(this.size() - 1, 1)
    // 判断节点是否有左孩子，因为二叉堆的特性，有右必有左
    while (this.getLeftIndex(k) < this.size()) {
        let j = this.getLeftIndex(k)
        // 判断是否有右孩子，并且右孩子是否大于左孩子
        if (j + 1 < this.size() && this.heap[j + 1] > this.heap[j]) j++
        // 判断父节点是否已经比子节点都大

```

```
        if (this.heap[k] >= this.heap[j]) break
        this._swap(k, j)
        k = j
    }
}
_swap(left, right) {
    let rightValue = this.heap[right]
    this.heap[right] = this.heap[left]
    this.heap[left] = rightValue
}
}
```

算法章节

时间复杂度

通常使用最差的时间复杂度来衡量一个算法的好坏。

常数时间 $O(1)$ 代表这个操作和数据量没关系，是一个固定时间的操作，比如说四则运算。

对于一个算法来说，可能会计算出如下操作次数 $aN + 1$ ， N 代表数据量。那么该算法的时间复杂度就是 $O(N)$ 。因为我们在计算时间复杂度的时候，数据量通常是非常大的，这时候低阶项和常数项可以忽略不计。

当然可能会出现两个算法都是 $O(N)$ 的时间复杂度，那么对比两个算法的好坏就要通过对比低阶项和常数项了。

位运算

位运算在算法中很有用，速度可以比四则运算快很多。

在学习位运算之前应该知道十进制如何转二进制，二进制如何转十进制。这里说明下简单的计算方式

- 十进制 33 可以看成是 $32 + 1$ ，并且 33 应该是六位二进制的（因为 33 近似 32，而 32 是 2 的五次方，所以是六位），那么十进制 33 就是 100001，只要是 2 的次方，那么就是 1 否则都为 0
- 那么二进制 100001 同理，首位是 2^5 ，末位是 2^0 ，相加得出 33

左移 <<

```
10 << 1 // -> 20
```

左移就是将二进制全部往左移动，10 在二进制中表示为 1010，左移一位后变成 10100，转换为十进制也就是 20，所以基本可以把左移看成以下公式 $a * (2 \wedge b)$

算数右移 >>

```
10 >> 1 // -> 5
```

算数右移就是将二进制全部往右移动并去除多余的右边，10 在二进制中表示为 1010，右移一位后变成 101，转换为十进制也就是 5，所以基本可以把右移看成以下公式 $\text{int } v = a / (2 \wedge b)$

右移很好用，比如可以用在二分算法中取中间值

```
13 >> 1 // -> 6
```

按位操作

按位与

每一位都为 1，结果才为 1

```
8 & 7 // -> 0  
// 1000 & 0111 -> 0000 -> 0
```

按位或

其中一位为 1，结果就是 1

```
8 | 7 // -> 15
// 1000 | 0111 -> 1111 -> 15
```

按位异或

每一位都不同，结果才为 1

```
8 ^ 7 // -> 15
8 ^ 8 // -> 0
// 1000 ^ 0111 -> 1111 -> 15
// 1000 ^ 1000 -> 0000 -> 0
```

从以上代码中可以发现按位异或就是不进位加法

面试题：两个数不使用四则运算得出和

这道题中可以按位异或，因为按位异或就是不进位加法， $8 \wedge 8 = 0$ 如果进位了，就是 16 了，所以我们只需要将两个数进行异或操作，然后进位。那么也就是说两个二进制都是 1 的位置，左边应该有一个进位 1，所以可以得出以下公式 $a + b = (a \wedge b) + ((a \& b) \ll 1)$ ，然后通过迭代的方式模拟加法

```
function sum(a, b) {
  if (a == 0) return b
  if (b == 0) return a
  let newA = a ^ b
  let newB = (a & b) << 1
  return sum(newA, newB)
}
```

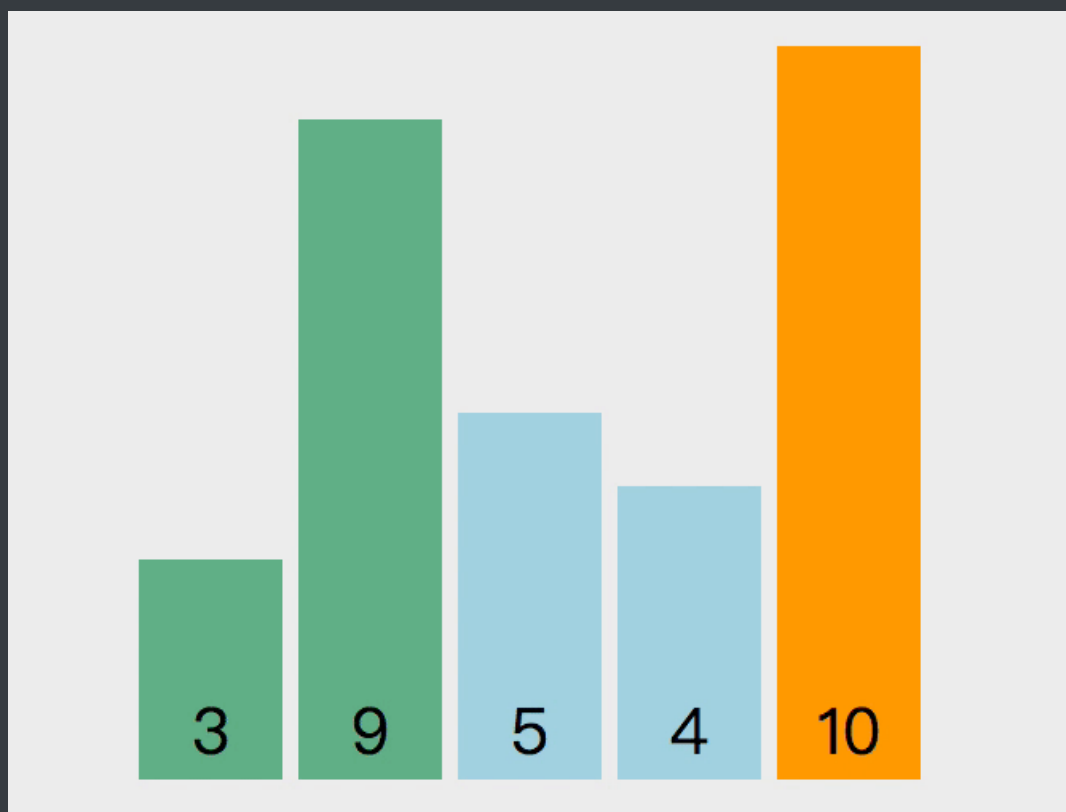
排序

以下两个函数是排序中会用到的通用函数，就不一一写了

```
function checkArray(array) {  
    if (!array || array.length <= 2) return  
}  
function swap(array, left, right) {  
    let rightValue = array[right]  
    array[right] = array[left]  
    array[left] = rightValue  
}
```

冒泡排序

冒泡排序的原理如下，从第一个元素开始，把当前元素和下一个索引元素进行比较。如果当前元素大，那么就交换位置，重复操作直到比较到最后一个元素，那么此时最后一个元素就是该数组中最大的数。下一轮重复以上操作，但是此时最后一个元素已经是最大数了，所以不需要再比较最后一个元素，只需要比较到 $\text{length} - 1$ 的位置。



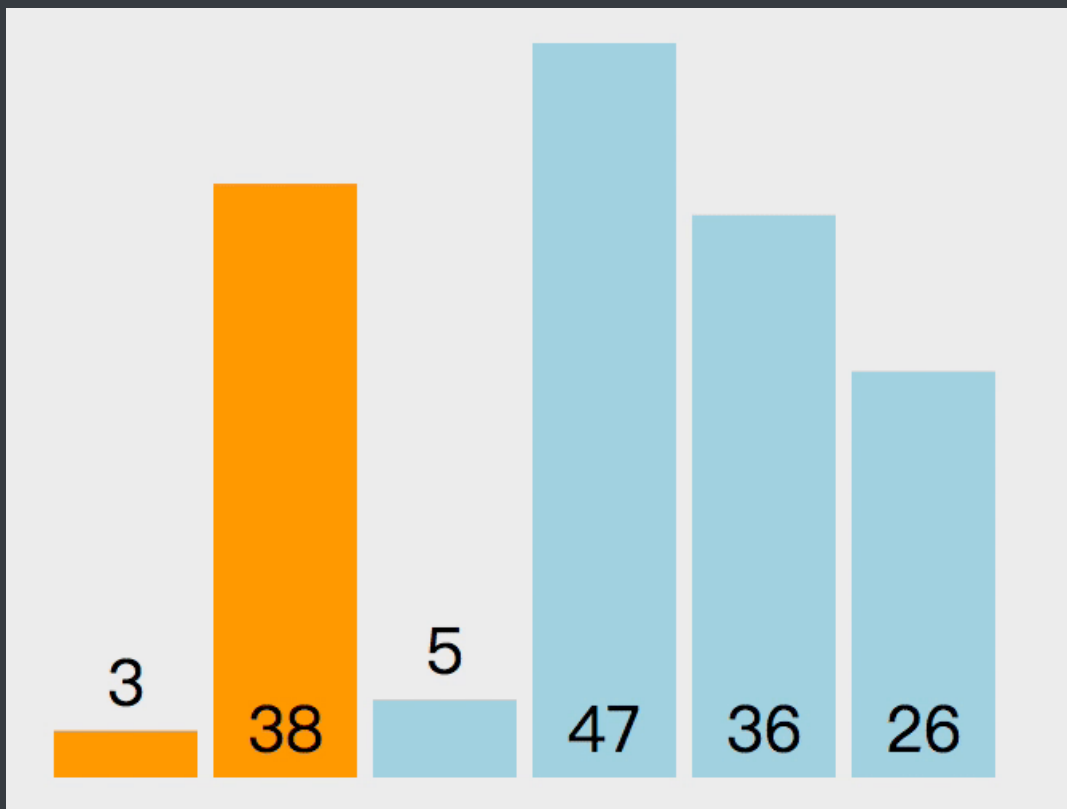
以下是实现该算法的代码

```
function bubble(array) {
  checkArray(array);
  for (let i = array.length - 1; i > 0; i--) {
    // 从 0 到 `length - 1` 遍历
    for (let j = 0; j < i; j++) {
      if (array[j] > array[j + 1]) swap(array, j, j + 1)
    }
  }
  return array;
}
```

该算法的操作次数是一个等差数列 $n + (n - 1) + (n - 2) + 1$ ，去掉常数项以后得出时间复杂度是 $O(n * n)$

插入排序

插入排序的原理如下。第一个元素默认是已排序元素，取出下一个元素和当前元素比较，如果当前元素大就交换位置。那么此时第一个元素就是当前的最小数，所以下次取出操作从第三个元素开始，向前对比，重复之前的操作。



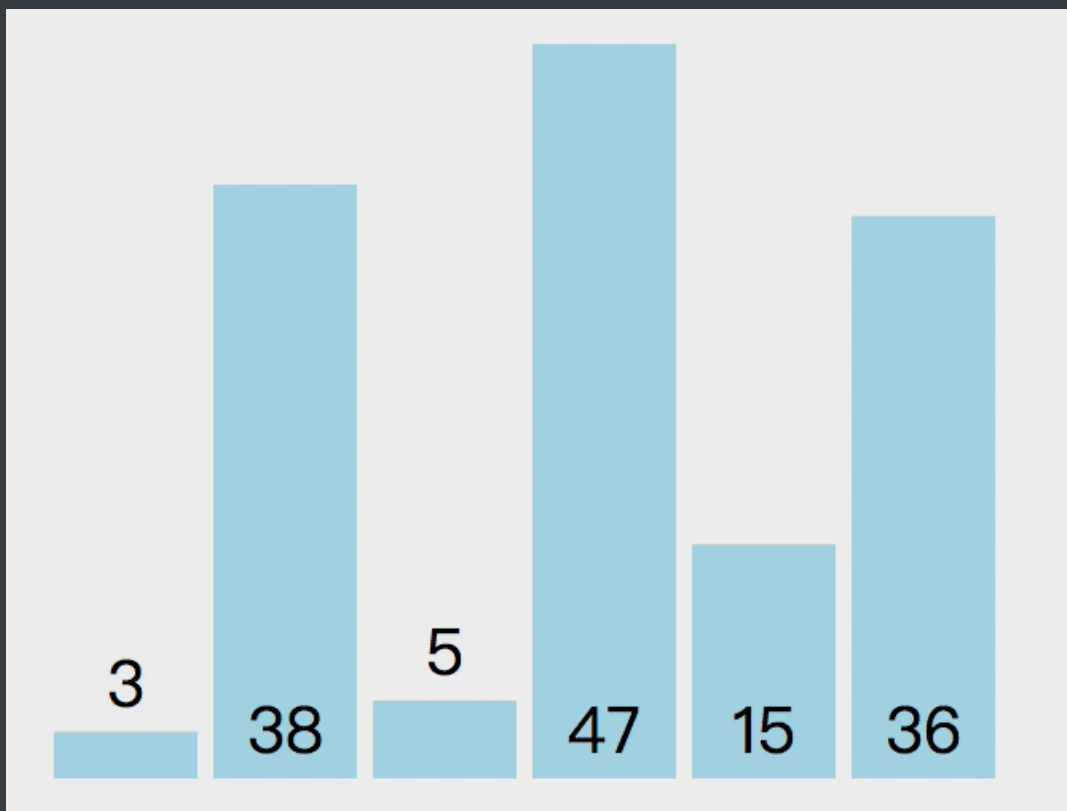
以下是实现该算法的代码

```
function insertion(array) {  
  checkArray(array);  
  for (let i = 1; i < array.length; i++) {  
    for (let j = i - 1; j >= 0 && array[j] > array[j + 1]; j--)  
      swap(array, j, j + 1);  
  }  
  return array;  
}
```

该算法的操作次数是一个等差数列 $n + (n - 1) + (n - 2) + 1$ ，去掉常数项以后得出时间复杂度是 $O(n * n)$

选择排序

选择排序的原理如下。遍历数组，设置最小值的索引为 0，如果取出的值比当前最小值小，就替换最小值索引，遍历完成后，将第一个元素和最小值索引上的值交换。如上操作后，第一个元素就是数组中的最小值，下次遍历就可以从索引 1 开始重复上述操作。



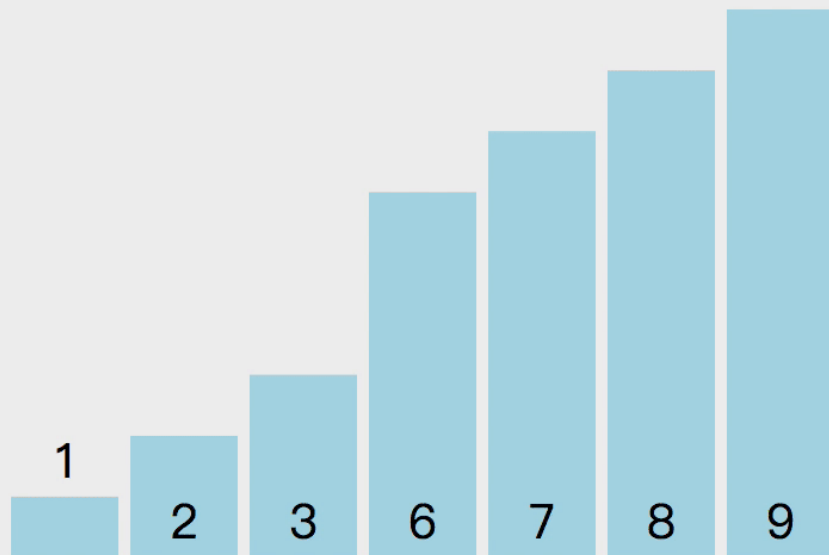
以下是实现该算法的代码

```
function selection(array) {  
  checkArray(array);  
  for (let i = 0; i < array.length - 1; i++) {  
    let minIndex = i;  
    for (let j = i + 1; j < array.length; j++) {  
      minIndex = array[j] < array[minIndex] ? j : minIndex;  
    }  
    swap(array, i, minIndex);  
  }  
  return array;  
}
```

该算法的操作次数是一个等差数列 $n + (n - 1) + (n - 2) + 1$ ，去掉常数项以后得出时间复杂度是 $O(n * n)$

归并排序

归并排序的原理如下。递归的将数组两两分开直到最多包含两个元素，然后将数组排序合并，最终合并为排序好的数组。假设我有一组数组 $[3, 1, 2, 8, 9, 7, 6]$ ，中间数索引是 3，先排序数组 $[3, 1, 2, 8]$ 。在这个左边数组上，继续拆分直到变成数组包含两个元素（如果数组长度是奇数的话，会有一个拆分数组只包含一个元素）。然后排序数组 $[3, 1]$ 和 $[2, 8]$ ，然后再排序数组 $[1, 3, 2, 8]$ ，这样左边数组就排序完成，然后按照以上思路排序右边数组，最后将数组 $[1, 2, 3, 8]$ 和 $[6, 7, 9]$ 排序。



以下是实现该算法的代码

```
function sort(array) {  
  checkArray(array);  
  mergeSort(array, 0, array.length - 1);  
  return array;  
}  
  
function mergeSort(array, left, right) {  
  // 左右索引相同说明已经只有一个数  
  if (left === right) return;  
  // 等同于 `left + (right - left) / 2`  
  // 相比 `(left + right) / 2` 来说更加安全，不会溢出  
  // 使用位运算是因为位运算比四则运算快
```

```

let mid = parseInt(left + ((right - left) >> 1));
mergeSort(array, left, mid);
mergeSort(array, mid + 1, right);

let help = [];
let i = 0;
let p1 = left;
let p2 = mid + 1;
while (p1 <= mid && p2 <= right) {
    help[i++] = array[p1] < array[p2] ? array[p1++] : array[p2++];
}
while (p1 <= mid) {
    help[i++] = array[p1++];
}
while (p2 <= right) {
    help[i++] = array[p2++];
}
for (let i = 0; i < help.length; i++) {
    array[left + i] = help[i];
}
return array;
}

```

以上算法使用了递归的思想。递归的本质就是压栈，每递归执行一次函数，就将该函数的信息（比如参数，内部的变量，执行到的行数）压栈，直到遇到终止条件，然后出栈并继续执行函数。对于以上递归函数的调用轨迹如下

```

mergeSort(data, 0, 6) // mid = 3
  mergeSort(data, 0, 3) // mid = 1
    mergeSort(data, 0, 1) // mid = 0
      mergeSort(data, 0, 0) // 遇到终止，回退到上一步
    mergeSort(data, 1, 1) // 遇到终止，回退到上一步
    // 排序 p1 = 0, p2 = mid + 1 = 1
    // 回退到 `mergeSort(data, 0, 3)` 执行下一个递归
  mergeSort(2, 3) // mid = 2

```

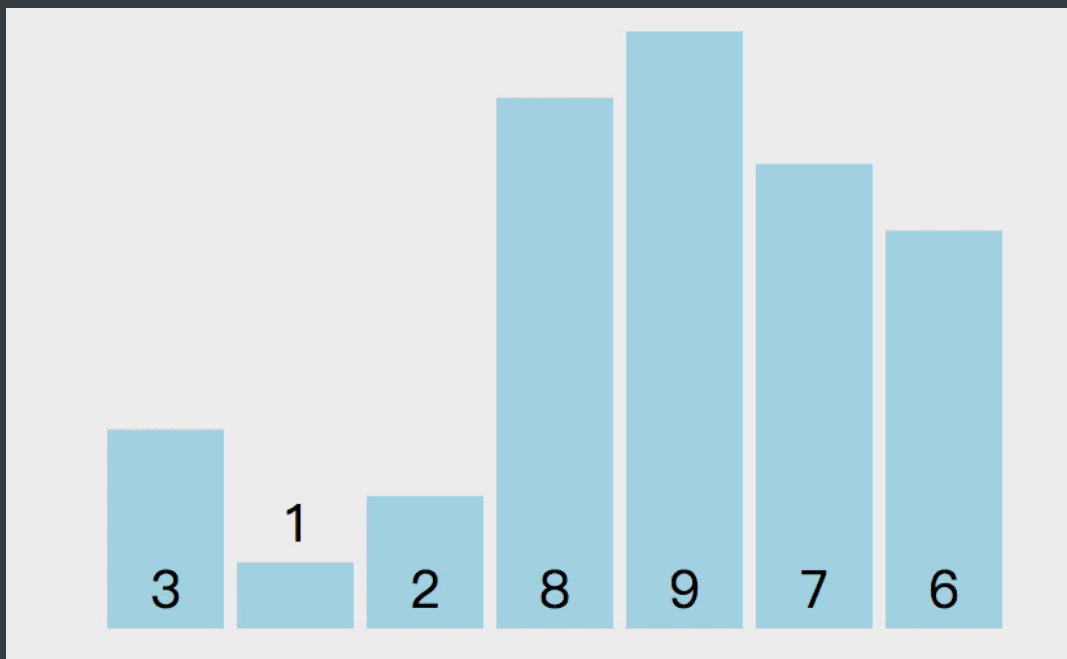


```
mergeSort(3, 3) // 遇到终止，回退到上一步
// 排序 p1 = 2, p2 = mid + 1 = 3
// 回退到 `mergeSort(data, 0, 3)` 执行合并逻辑
// 排序 p1 = 0, p2 = mid + 1 = 2
// 执行完毕回退
// 左边数组排序完毕，右边也是如上轨迹
```

该算法的操作次数是可以这样计算：递归了两次，每次数据量是数组的一半，并且最后把整个数组迭代了一次，所以得出表达式 $2T(N/2) + T(N)$ （T 代表时间，N 代表数据量）。根据该表达式可以套用 [该公式](#) 得出时间复杂度为 $O(N * \log N)$

快排

快排的原理如下。随机选取一个数组中的值作为基准值，从左至右取值与基准值对比大小。比基准值小的放数组左边，大的放右边，对比完成后将基准值和第一个比基准值大的值交换位置。然后将数组以基准值的位置分为两部分，继续递归以上操作。



以下是实现该算法的代码

```
function sort(array) {
  checkArray(array);
  quickSort(array, 0, array.length - 1);
}
```

```

    return array;
}

function quickSort(array, left, right) {
    if (left < right) {
        swap(array, , right)
        // 随机取值，然后和末尾交换，这样做比固定取一个位置的复杂度略低
        let indexs = part(array, parseInt(Math.random() * (right - left +
1)) + left, right);
        quickSort(array, left, indexs[0]);
        quickSort(array, indexs[1] + 1, right);
    }
}

function part(array, left, right) {
    let less = left - 1;
    let more = right;
    while (left < more) {
        if (array[left] < array[right]) {
            // 当前值比基准值小，`less` 和 `left` 都加一
            ++less;
            ++left;
        } else if (array[left] > array[right]) {
            // 当前值比基准值大，将当前值和右边的值交换
            // 并且不改变 `left`，因为当前换过来的值还没有判断过大小
            swap(array, --more, left);
        } else {
            // 和基准值相同，只移动下标
            left++;
        }
    }
    // 将基准值和比基准值大的第一个值交换位置
    // 这样数组就变成 `[比基准值小, 基准值, 比基准值大]`
    swap(array, right, more);
    return [less, more];
}

```

该算法的复杂度和归并排序是相同的，但是额外空间复杂度比归并排序少，只需 $O(\log N)$ ，并且相比归并排序来说，所需的常数时间也更少。

面试题

Sort Colors：该题目来自 [LeetCode](#)，题目需要我们将 $[2, 0, 2, 1, 1, 0]$ 排序成 $[0, 0, 1, 1, 2, 2]$ ，这个问题就可以使用三路快排的思想。

以下是代码实现

```
var sortColors = function(nums) {
    let left = -1;
    let right = nums.length;
    let i = 0;
    // 下标如果遇到 right, 说明已经排序完成
    while (i < right) {
        if (nums[i] == 0) {
            swap(nums, i++, ++left);
        } else if (nums[i] == 1) {
            i++;
        } else {
            swap(nums, i, --right);
        }
    }
};
```

Kth Largest Element in an Array：该题目来自 [LeetCode](#)，题目需要找出数组中第 K 大的元素，这问题也可以使用快排的思路。并且因为是找出第 K 大元素，所以在分离数组的过程中，可以找出需要的元素在哪边，然后只需要排序相应的一边数组就好。

以下是代码实现

```
var findKthLargest = function(nums, k) {
    let l = 0
    let r = nums.length - 1
```

```
// 得出第 K 大元素的索引位置
k = nums.length - k
while (l < r) {
    // 分离数组后获得比基准树大的第一个元素索引
    let index = part(nums, l, r)
    // 判断该索引和 k 的大小
    if (index < k) {
        l = index + 1
    } else if (index > k) {
        r = index - 1
    } else {
        break
    }
}
return nums[k]
};

function part(array, left, right) {
    let less = left - 1;
    let more = right;
    while (left < more) {
        if (array[left] < array[right]) {
            ++less;
            ++left;
        } else if (array[left] > array[right]) {
            swap(array, --more, left);
        } else {
            left++;
        }
    }
    swap(array, right, more);
    return more;
}
```

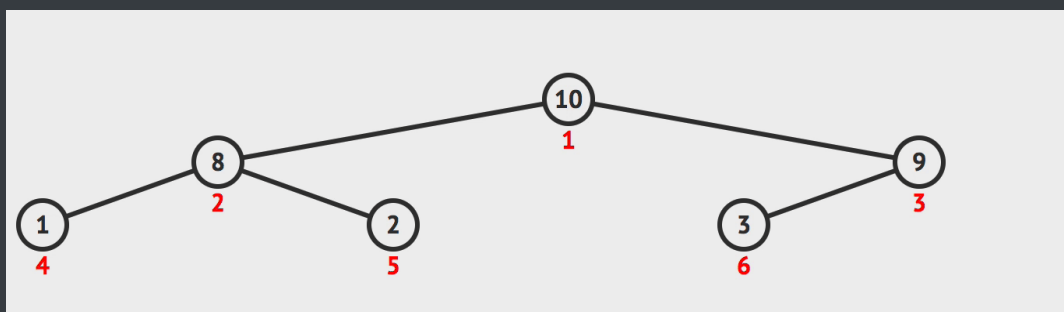
堆排序

堆排序利用了二叉堆的特性来做，二叉堆通常用数组表示，并且二叉堆是一颗完全二叉树（所有叶节点（最底层的节点）都是从左往右顺序排序，并且其他层的节点都是满的）。二叉堆又分为大根堆与小根堆。

- 大根堆是某个节点的所有子节点的值都比他小
- 小根堆是某个节点的所有子节点的值都比他大

堆排序的原理就是组成一个大根堆或者小根堆。以小根堆为例，某个节点的左边子节点索引是 $i * 2 + 1$ ，右边是 $i * 2 + 2$ ，父节点是 $(i - 1) / 2$ 。

1. 首先遍历数组，判断该节点的父节点是否比他小，如果小就交换位置并继续判断，直到他的父节点比他大
2. 重新以上操作 1，直到数组首位是最大值
3. 然后将首位和末尾交换位置并将数组长度减一，表示数组末尾已是最大值，不需要再比较大小
4. 对比左右节点哪个大，然后记住大的节点的索引并且和父节点对比大小，如果子节点大就交换位置
5. 重复以上操作 3 - 4 直到整个数组都是大根堆。



以下是实现该算法的代码

```
function heap(array) {  
  checkArray(array);  
  // 将最大值交换到首位  
  for (let i = 0; i < array.length; i++) {  
    heapInsert(array, i);  
  }  
  let size = array.length;
```

```

// 交换首位和末尾
swap(array, 0, --size);
while (size > 0) {
    heapify(array, 0, size);
    swap(array, 0, --size);
}
return array;
}

function heapInsert(array, index) {
    // 如果当前节点比父节点大, 就交换
    while (array[index] > array[parseInt((index - 1) / 2)]) {
        swap(array, index, parseInt((index - 1) / 2));
        // 将索引变成父节点
        index = parseInt((index - 1) / 2);
    }
}

function heapify(array, index, size) {
    let left = index * 2 + 1;
    while (left < size) {
        // 判断左右节点大小
        let largest =
            left + 1 < size && array[left] < array[left + 1] ? left + 1 :
left;
        // 判断子节点和父节点大小
        largest = array[index] < array[largest] ? largest : index;
        if (largest === index) break;
        swap(array, index, largest);
        index = largest;
        left = index * 2 + 1;
    }
}

```

以上代码实现了小根堆, 如果需要通过实现大根堆, 只需要把节点对比反一下就好。

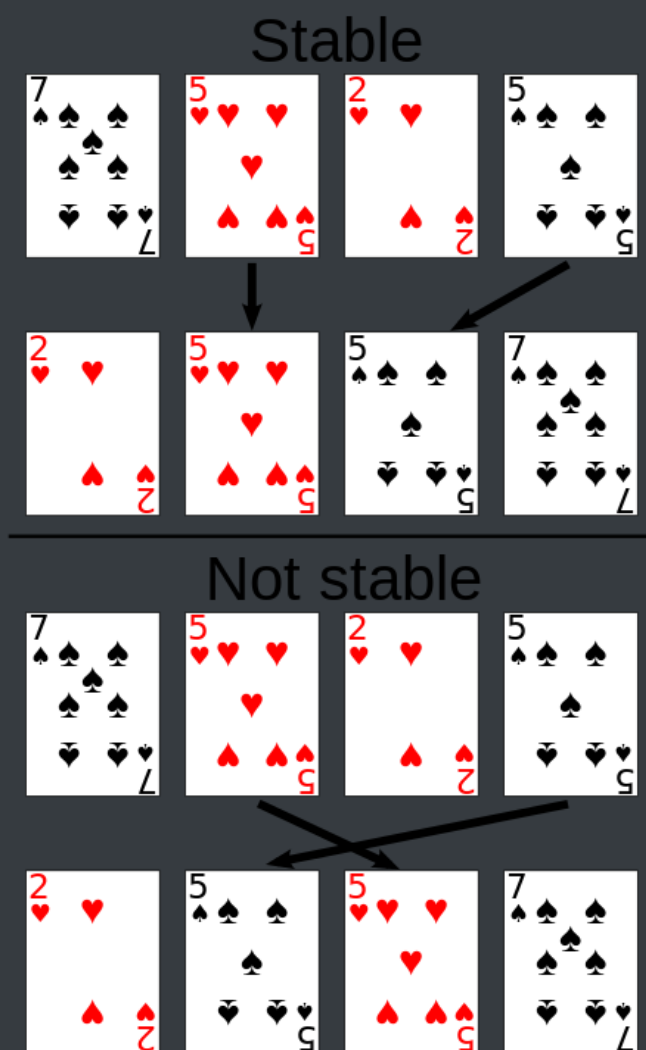
该算法的复杂度是 $O(\log N)$

系统自带排序实现

每个语言的排序内部实现都是不同的。

对于 JS 来说，数组长度大于 10 会采用快排，否则使用插入排序 [源码实现](#)。选择插入排序是因为虽然时间复杂度很差，但是在数据量很小的情况下和 $O(N * \log N)$ 相差无几，然而插入排序需要的常数时间很小，所以相对别的排序来说更快。

对于 Java 来说，还会考虑内部的元素的类型。对于存储对象的数组来说，会采用稳定性好的算法。稳定性的意思就是对于相同值来说，相对顺序不能改变。



链表

反转单向链表

该题目来自 [LeetCode](#)，题目需要将一个单向链表反转。思路很简单，使用三个变量分别表示当前节点和当前节点的前后节点，虽然这题很简单，但是却是一道面试常考题

以下是实现该算法的代码

```
var reverseList = function(head) {  
    // 判断下变量边界问题  
    if (!head || !head.next) return head  
    // 初始设置为空，因为第一个节点反转后就是尾部，尾部节点指向 null  
    let pre = null  
    let current = head  
    let next  
    // 判断当前节点是否为空  
    // 不为空就先获取当前节点的下一节点  
    // 然后把当前节点的 next 设为上一个节点  
    // 然后把 current 设为下一个节点，pre 设为当前节点  
    while(current) {  
        next = current.next  
        current.next = pre  
        pre = current  
        current = next  
    }  
    return pre  
};
```

树

二叉树的先序，中序，后序遍历

先序遍历表示先访问根节点，然后访问左节点，最后访问右节点。

中序遍历表示先访问左节点，然后访问根节点，最后访问右节点。

后序遍历表示先访问左节点，然后访问右节点，最后访问根节点。

递归实现

递归实现相当简单，代码如下

```
function TreeNode(val) {
  this.val = val;
  this.left = this.right = null;
}
var traversal = function(root) {
  if (root) {
    // 先序
    console.log(root);
    traversal(root.left);
    // 中序
    // console.log(root);
    traversal(root.right);
    // 后序
    // console.log(root);
  }
};
```

对于递归的实现来说，只需要理解每个节点都会被访问三次就明白为什么这样实现了。

非递归实现

非递归实现使用了栈的结构，通过栈的先进后出模拟递归实现。

以下是先序遍历代码实现

```
function pre(root) {
  if (root) {
    let stack = [];
    // 先将根节点 push
    stack.push(root);
```

```

// 判断栈中是否为空
while (stack.length > 0) {
    // 弹出栈顶元素
    root = stack.pop();
    console.log(root);
    // 因为先序遍历是先左后右，栈是先进后出结构
    // 所以先 push 右边再 push 左边
    if (root.right) {
        stack.push(root.right);
    }
    if (root.left) {
        stack.push(root.left);
    }
}
}
}

```

以下是中序遍历代码实现

```

function mid(root) {
    if (root) {
        let stack = [];
        // 中序遍历是先左再根最后右
        // 所以首先应该先把最左边节点遍历到底依次 push 进栈
        // 当左边没有节点时，就打印栈顶元素，然后寻找右节点
        // 对于最左边的叶节点来说，可以把它看成是两个 null 节点的父节点
        // 左边打印不出东西就把父节点拿出来打印，然后再看右节点
        while (stack.length > 0 || root) {
            if (root) {
                stack.push(root);
                root = root.left;
            } else {
                root = stack.pop();
                console.log(root);
                root = root.right;
            }
        }
    }
}

```

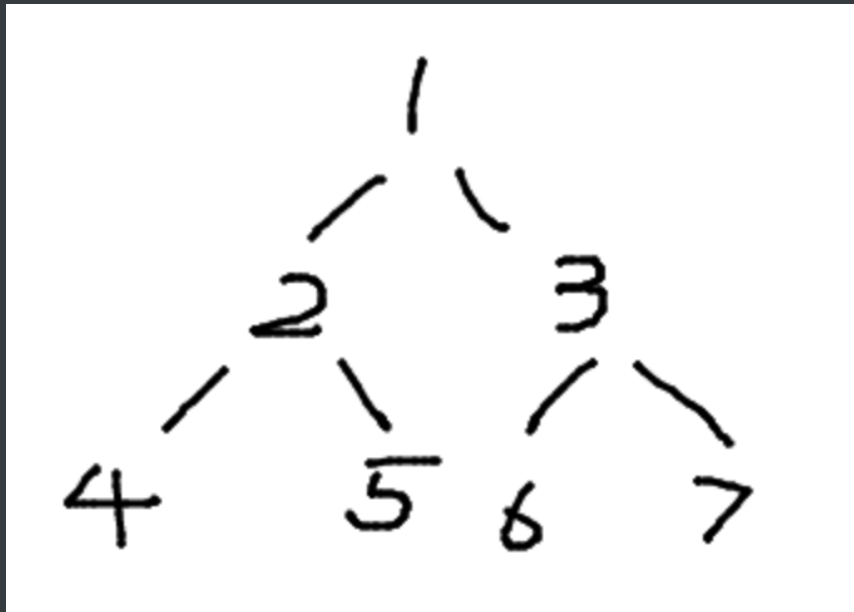
```
    }  
  }  
}  
}
```

以下是后序遍历代码实现，该代码使用了两个栈来实现遍历，相比一个栈的遍历来说要容易理解很多

```
function pos(root) {  
  if (root) {  
    let stack1 = [];  
    let stack2 = [];  
    // 后序遍历是先左再右最后根  
    // 所以对于一个栈来说，应该先 push 根节点  
    // 然后 push 右节点，最后 push 左节点  
    stack1.push(root);  
    while (stack1.length > 0) {  
      root = stack1.pop();  
      stack2.push(root);  
      if (root.left) {  
        stack1.push(root.left);  
      }  
      if (root.right) {  
        stack1.push(root.right);  
      }  
    }  
    while (stack2.length > 0) {  
      console.log(s2.pop());  
    }  
  }  
}
```

中序遍历的前驱后继节点

实现这个算法的前提是节点有一个 `parent` 的指针指向父节点，根节点指向 `null`。



如图所示，该树的中序遍历结果是 4，2，5，1，6，3，7

前驱节点

对于节点 2 来说，他的前驱节点就是 4，按照中序遍历原则，可以得出以下结论

1. 如果选取的节点的左节点不为空，就找该左节点最右的节点。对于节点 1 来说，他有左节点 2，那么节点 2 的最右节点就是 5
2. 如果左节点为空，且目标节点是父节点的右节点，那么前驱节点为父节点。对于节点 5 来说，没有左节点，且是节点 2 的右节点，所以节点 2 是前驱节点
3. 如果左节点为空，且目标节点是父节点的左节点，向上寻找到第一个是父节点的右节点的节点。对于节点 6 来说，没有左节点，且是节点 3 的左节点，所以向上寻找到节点 1，发现节点 3 是节点 1 的右节点，所以节点 1 是节点 6 的前驱节点

以下是算法实现

```
function predecessor(node) {  
  if (!node) return  
  // 结论 1  
  if (node.left) {  
    return getRight(node.left)  
  }  
}
```

```

    } else {
        let parent = node.parent
        // 结论 2 3 的判断
        while(parent && parent.right === node) {
            node = parent
            parent = node.parent
        }
        return parent
    }
}

function getRight(node) {
    if (!node) return
    node = node.right
    while(node) node = node.right
    return node
}

```

后继节点

对于节点 2 来说，他的后继节点就是 5，按照中序遍历原则，可以得出以下结论

1. 如果有右节点，就找到该右节点的最左节点。对于节点 1 来说，他有右节点 3，那么节点 3 的最左节点就是 6
2. 如果没有右节点，就向上遍历直到找到一个节点是父节点的左节点。对于节点 5 来说，没有右节点，就向上寻找到节点 2，该节点是父节点 1 的左节点，所以节点 1 是后继节点

以下是算法实现

```

function successor(node) {
    if (!node) return
    // 结论 1
    if (node.right) {
        return getLeft(node.right)
    } else {

```

```
// 结论 2
let parent = node.parent
// 判断 parent 为空
while(parent && parent.left === node) {
  node = parent
  parent = node.parent
}
return parent
}
}
function getLeft(node) {
  if (!node) return
  node = node.left
  while(node) node = node.left
  return node
}
```

树的深度

树的最大深度：该题目来自 [Leetcode](#)，题目要求求出一颗二叉树的最大深度

以下是算法实现

```
var maxDepth = function(root) {
  if (!root) return 0
  return Math.max(maxDepth(root.left), maxDepth(root.right)) + 1
};
```

对于该递归函数可以这样理解：一旦没有找到节点就会返回 0，每弹出一次递归函数就会加一，树有三层就会得到3。

动态规划

动态规划背后的基本思想非常简单。就是将一个问题拆分为子问题，一般来说这些子问题都是非常相似的，那么我们可以通过只解决一次每个子问题来达到减少计算量的目的。

一旦得出每个子问题的解，就存储该结果以便下次使用。

斐波那契数列

斐波那契数列就是从 0 和 1 开始，后面的数都是前两个数之和

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89....

那么显而易见，我们可以通过递归的方式来完成求解斐波那契数列

```
function fib(n) {  
  if (n < 2 && n >= 0) return n  
  return fib(n - 1) + fib(n - 2)  
}  
fib(10)
```

以上代码已经可以完美的解决问题。但是以上解法却存在很严重的性能问题，当 n 越大的时候，需要的时间是指数增长的，这时候就可以通过动态规划来解决这个问题。

动态规划的本质其实就是两点

1. 自底向上分解子问题
2. 通过变量存储已经计算过的解

根据上面两点，我们的斐波那契数列的动态规划思路也就出来了

1. 斐波那契数列从 0 和 1 开始，那么这就是这个子问题的最底层
2. 通过数组来存储每一位所对应的斐波那契数列的值

```
function fib(n) {  
  let array = new Array(n + 1).fill(null)  
  array[0] = 0  
  array[1] = 1  
  for (let i = 2; i <= n; i++) {  
    array[i] = array[i - 1] + array[i - 2]  
  }  
  return array[n]  
}  
fib(10)
```

0 - 1背包问题

该问题可以描述为：给定一组物品，每种物品都有自己的重量和价格，在限定的总重量内，我们如何选择，才能使得物品的总价格最高。每个问题只能放入至多一次。

假设我们有以下物品

物品 ID / 重量	价值
1	3
2	7
3	12

对于一个总容量为 5 的背包来说，我们可以放入重量 2 和 3 的物品来达到背包内的物品总价值最高。

对于这个问题来说，子问题就两个，分别是放物品和不放物品，可以通过以下表格来理解子问题

物品 ID / 剩余容量	0	1	2	3	4	5
1	0	3	3	3	3	3
2	0	3	7	10	10	10
3	0	3	7	12	15	19

直接来分析能放三种物品的情况，也就是最后一行

- 当容量少于 3 时，只取上一行对应的数据，因为当前容量不能容纳物品 3
- 当容量为 3 时，考虑两种情况，分别为放入物品 3 和不放物品 3
 - 不放物品 3 的情况下，总价值为 10
 - 放入物品 3 的情况下，总价值为 12，所以应该放入物品 3
- 当容量为 4 时，考虑两种情况，分别为放入物品 3 和不放物品 3
 - 不放物品 3 的情况下，总价值为 10
 - 放入物品 3 的情况下，和放入物品 1 的价值相加，得出总价值为 15，所以应该放入物品 3
- 当容量为 5 时，考虑两种情况，分别为放入物品 3 和不放物品 3
 - 不放物品 3 的情况下，总价值为 10
 - 放入物品 3 的情况下，和放入物品 2 的价值相加，得出总价值为 19，所以应该放入物品 3

以下代码对照上表更容易理解

```
/**
 * @param {*} w 物品重量
 * @param {*} v 物品价值
 * @param {*} C 总容量
 * @returns
 */
function knapsack(w, v, C) {
  let length = w.length
  if (length === 0) return 0

  // 对照表格，生成的二维数组，第一维代表物品，第二维代表背包剩余容量
```

```

// 第二维中的元素代表背包物品总价值
let array = new Array(length).fill(new Array(C + 1).fill(null))

// 完成底部子问题的解
for (let i = 0; i <= C; i++) {
  // 对照表格第一行， array[0] 代表物品 1
  // i 代表剩余总容量
  // 当剩余总容量大于物品 1 的重量时，记录下背包物品总价值，否则价值为 0
  array[0][i] = i >= w[0] ? v[0] : 0
}

// 自底向上开始解决子问题，从物品 2 开始
for (let i = 1; i < length; i++) {
  for (let j = 0; j <= C; j++) {
    // 这里求解子问题，分别为不放当前物品和放当前物品
    // 先求不放当前物品的背包总价值，这里的值也就是对应表格中上一行对应的值
    array[i][j] = array[i - 1][j]
    // 判断当前剩余容量是否可以放入当前物品
    if (j >= w[i]) {
      // 可以放入的话，就比大小
      // 放入当前物品和不放入当前物品，哪个背包总价值大
      array[i][j] = Math.max(array[i][j], v[i] + array[i - 1][j -
w[i]])
    }
  }
}
return array[length - 1][C]
}

```

最长递增子序列

最长递增子序列意思是在一组数字中，找出最长一串递增的数字，比如

0, 3, 4, 17, 2, 8, 6, 10

对于以上这串数字来说，最长递增子序列就是 0, 3, 4, 8, 10，可以通过以下表格更清晰的理解

数字	0	3	4	17	2	8	6	10
长度	1	2	3	4	2	4	4	5

通过以上表格可以很清晰的发现一个规律，找出刚好比当前数字小的数，并且在小的数组成的长度基础上加一。

这个问题的动态思路解法很简单，直接上代码

```
function lis(n) {
  if (n.length === 0) return 0
  // 创建一个和参数相同大小的数组，并填充值为 1
  let array = new Array(n.length).fill(1)
  // 从索引 1 开始遍历，因为数组已经所有都填充为 1 了
  for (let i = 1; i < n.length; i++) {
    // 从索引 0 遍历到 i
    // 判断索引 i 上的值是否大于之前的值
    for (let j = 0; j < i; j++) {
      if (n[i] > n[j]) {
        array[i] = Math.max(array[i], 1 + array[j])
      }
    }
  }
  let res = 1
  for (let i = 0; i < array.length; i++) {
    res = Math.max(res, array[i])
  }
  return res
}
```

字符串相关

在字符串相关算法中，Trie 树可以解决解决很多问题，同时具备良好的空间和时间复杂度，比如以下问题

- 词频统计

- 前缀匹配

如果你对于 Trie 树还不怎么了解，可以前往 [这里](#) 阅读

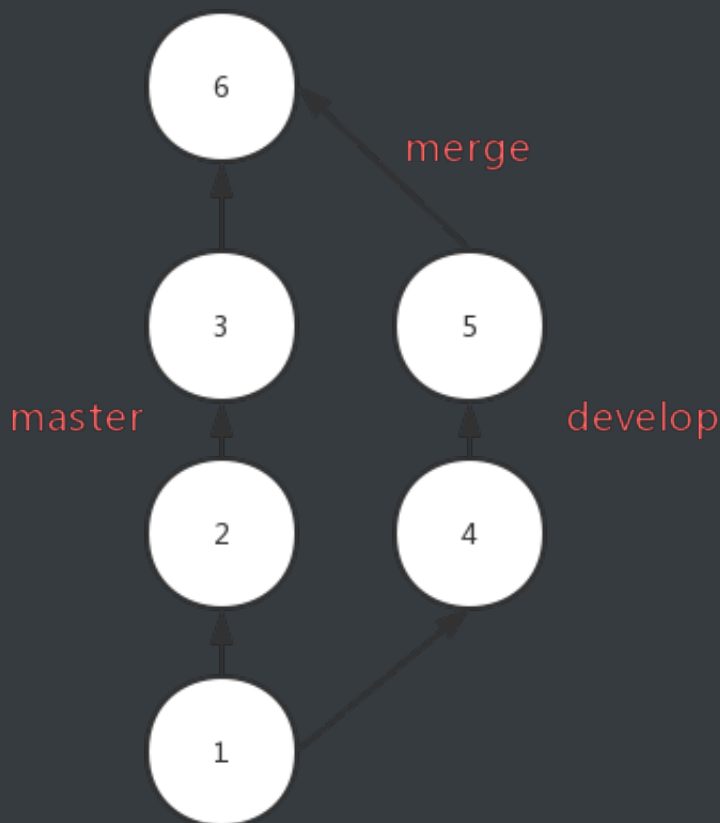
Git 章节

本文不会介绍 Git 的基本操作，会对一些高级操作进行说明。

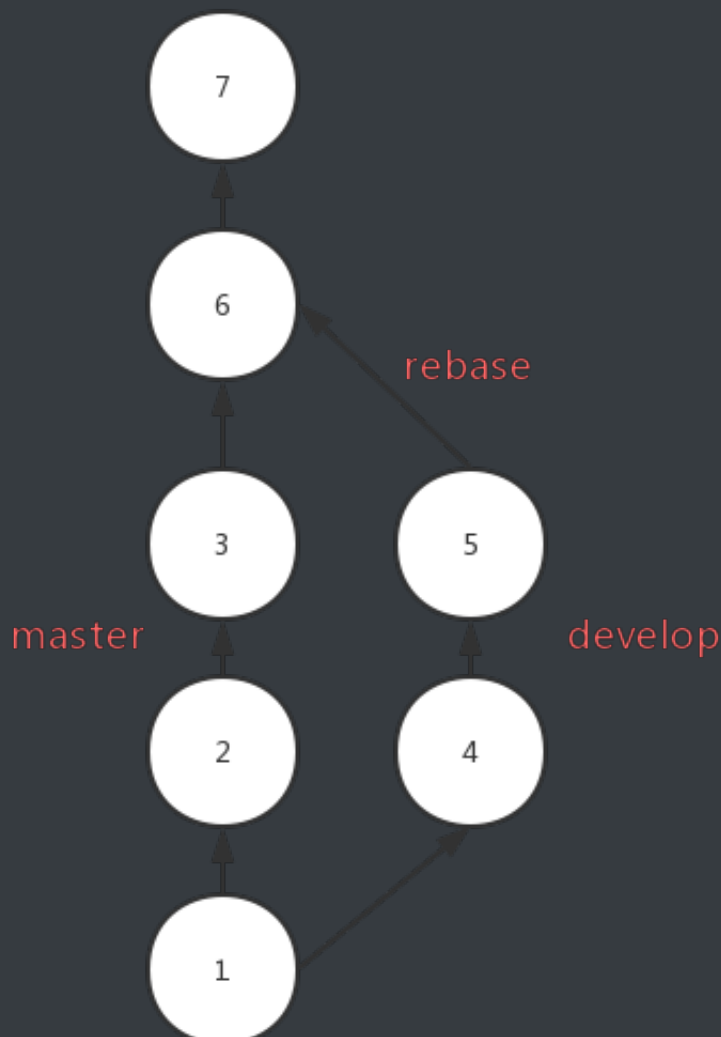
Rebase 合并

该命令可以让和 merge 命令得到的结果基本是一致的。

通常使用 merge 操作将分支上的代码合并到 master 中，分支样子如下所示



使用 rebase 后，会将 develop 上的 commit 按顺序移到 master 的第三个 commit 后面，分支样子如下所示



Rebase 对比 merge，优势在于合并后的结果很清晰，只有一条线，劣势在于如果一旦出现冲突，解决冲突很麻烦，可能要解决多个冲突，但是 merge 出现冲突只需要解决一次。

使用 rebase 应该在需要被 rebase 的分支上操作，并且该分支是本地分支。如果 develop 分支需要 rebase 到 master 上去，那么应该如下操作

```
## branch develop
git rebase master
git checkout master
## 用于将 `master` 上的 HEAD 移动到最新的 commit
git merge develop
```

stash

stash 用于临时保存工作目录的改动。开发中可能会遇到代码写一半需要切分支打包的问题，如果这时候你不想 commit 的话，就可以使用该命令。

```
git stash
```

使用该命令可以暂存你的工作目录，后面想恢复工作目录，只需要使用

```
git stash pop
```

这样你之前临时保存的代码又回来了

reflog

reflog 可以看到 HEAD 的移动记录，假如之前误删了一个分支，可以通过 git reflog 看到移动 HEAD 的哈希值

```
:
37d9aca (HEAD -> master) HEAD@{0}: merge new: Fast-forward
07244ac HEAD@{1}: checkout: moving from new to master
37d9aca (HEAD -> master) HEAD@{2}: commit: remove
```

从图中可以看出，HEAD 的最后一次移动行为是 merge 后，接下来分支 new 就被删除了，那么我们可以通过以下命令找回 new 分支

```
git checkout 37d9aca
git checkout -b new
```

PS: reflog 记录是时效的，只会保存一段时间内的记录。

Reset

如果你想删除刚写的 commit，就可以通过以下命令实现

```
git reset --hard HEAD^
```

但是 reset 的本质并不是删除了 commit，而是重新设置了 HEAD 和它指向的 branch。

职业章节

你是否时常会焦虑时间过的很快，没时间学习，本文将会分享一些个人的见解。

花时间补基础，读文档

在工作中我们时常会花很多时间去 debug，但是你是否发现很多问题最终只是你基础不扎实或者文档没有仔细看。

基础是你技术的基石，一定要花时间打好基础，而不是追各种新的技术。一旦你的基础扎实，学习各种新的技术也肯定不在话下，因为新的技术，究其根本都是相通的。

文档同样也是一门技术的基础。一个优秀的库，开发人员肯定已经把如何使用这个库都写在文档中了，仔细阅读文档一定会是少写 bug 的最省事路子。

学会搜索

如果你还在使用百度搜索编程问题，请尽快抛弃这个垃圾搜索引擎。同样一个关键字，使用百度和谷歌，谷歌基本完胜的。即使你使用中文在谷歌中搜索，得到的结果也往往是谷歌占优，所以如果你想迅速的通过搜索引擎来解决问题，那一定是谷歌。

学点英语

说到英语，一定是大家所最不想听的。其实我一直认为程序员学习英语是简单的，因为我们工作中是一直接触着英语，并且看懂技术文章，文档所需要的单词量是极少的。我时常在群里看到大家发出一个问题的截图问什么原因，其实在截图中英语已经很明白的说明了问题的所在，如果你的英语过关，完全不需要浪费时间来提问和搜索。所以我认为学点英语也是节省时间中很重要的一点。

那么如何去学习呢，chrome 装个翻译插件，直接拿英文文档或文章读，不会的就直接划词翻译，然后记录下这个单词并背诵。每天花半小时看点英文文档和文章，坚持两个月，你的英语水平不说别的，看文档和文章绝对不会有难题了。这一定是一个很划算的个人时间投资，花点时间学习英语，能为你将来的技术之路铺平很多坎。

画个图，想一想再做

你是否遇到过这种问题，需求一下来，看一眼，然后马上就按照设计稿开始做了，可能中间出个问题导致你需要返工。

如果你存在这样的问题，我很推荐在看到设计稿和需求的时候花点时间想一想，画一画。考虑一下设计稿中是否可以找到可以拆分出来的复用组件，是否存在之前写过的组件。该如何组织这个界面，数据的流转是怎么样。然后画一下这个页面的需求，最后再动手做。

利用好下班时间学习

说到下班时间，那可能就有人说了公司很迟下班，这其实是国内很普遍的情况。但是我认为正常的加班是可以的，但是强制的加班就是在损耗你的身体和前途。

可以这么说，大部分的 996 公司，加班的这些时间并不会增加你的技术，无非就是在写一些重复的业务逻辑。也许你可以拿到更多的钱，但是代价是身体还有前途。程序员是靠技术吃饭的，如果你长久呆在一个长时间加班的公司，不能增长你的技术还要吞噬你的下班学习时间，那么你一定废掉的。如果你遇到了这种情况，只能推荐尽快跳槽到非 996 的公司。

那么如果你有足够的下班时间，一定要花上 1，2 小时去学习，上班大家基本都一样，技术的精进就是看下班以后的那几个小时了。如果你能利用好下班时间来学习，坚持下去，时间一定会给你很好的答复。

列好 ToDo

我喜欢规划好一段时间内要做的事情，并且要把事情拆分为小点。给 ToDo 列好优先级，紧急的优先级最高。相同优先级的我喜欢先做简单的，因为这样一旦完成就能划掉一个，提高成就感。

反思和整理

每周末都会花上点时间整理下本周记录的笔记和看到的不错文章。然后考虑下本周完成的工作和下周准备要完成的工作。